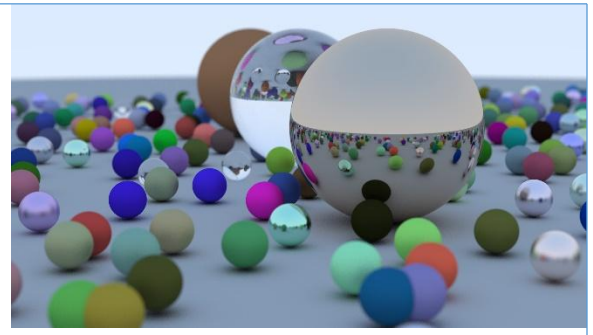
Comp4422



Computer Graphics

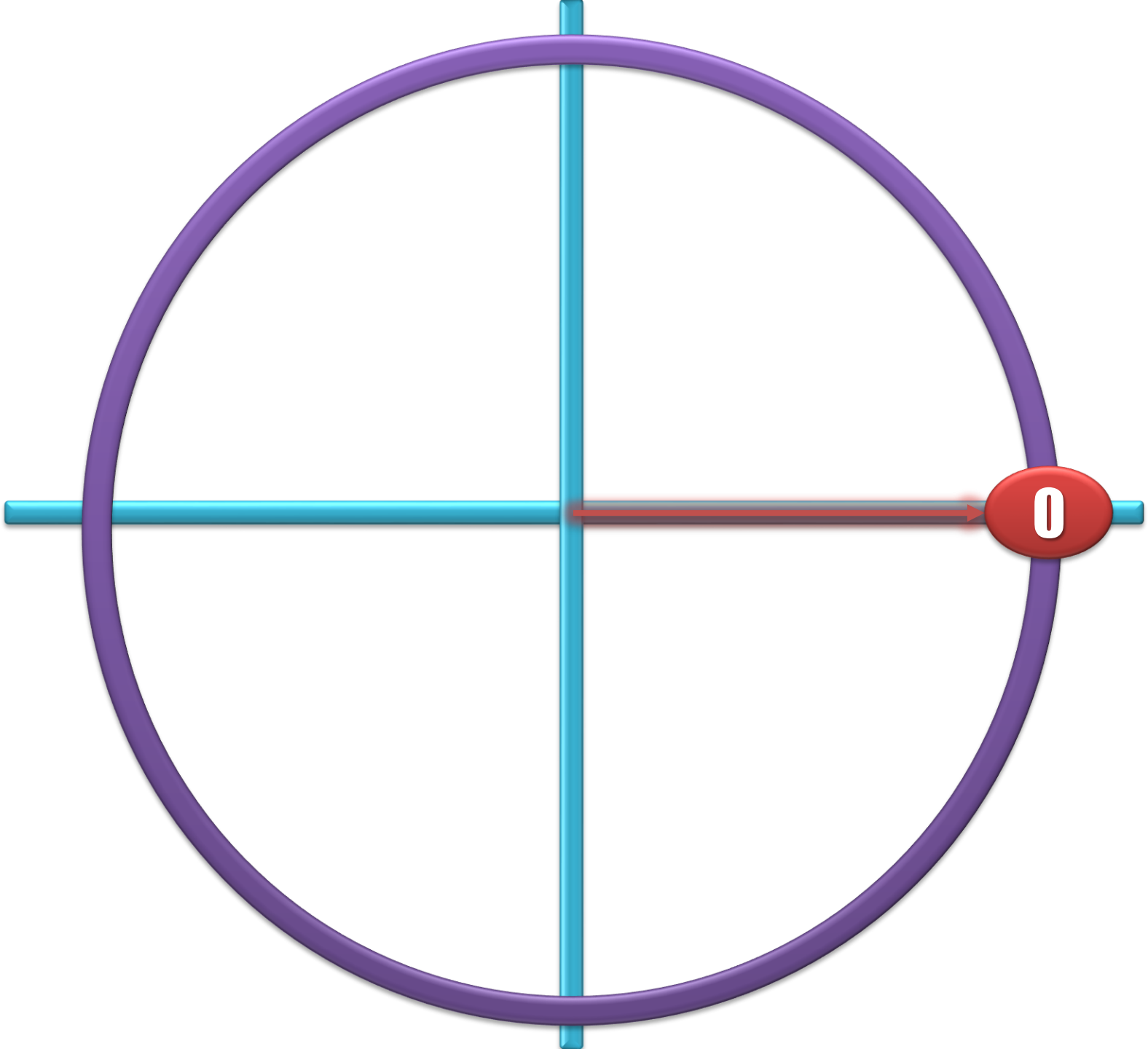
Lecture 03:

Geometric Transformations



What you will learn…

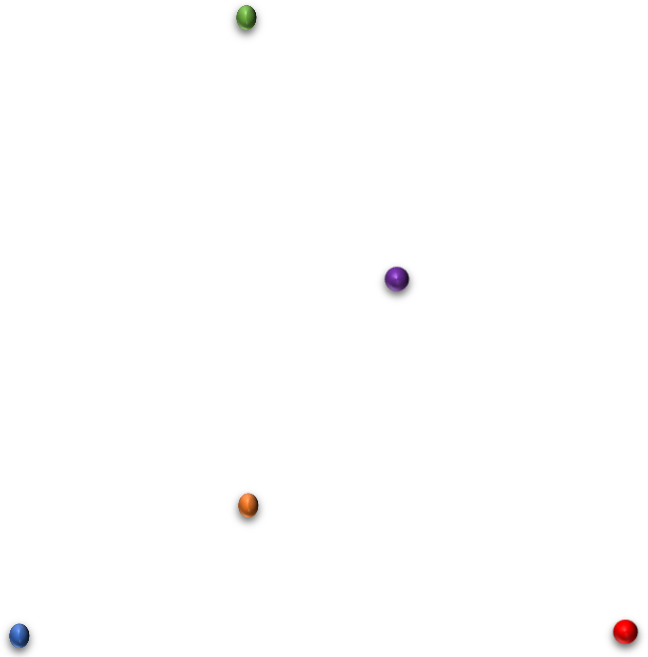
- Coordinate Free Geometry
- Coordinate Systems
- Basic Transformations
- Homogeneous Coordinates
- Interpolations


0

Coordinate Free

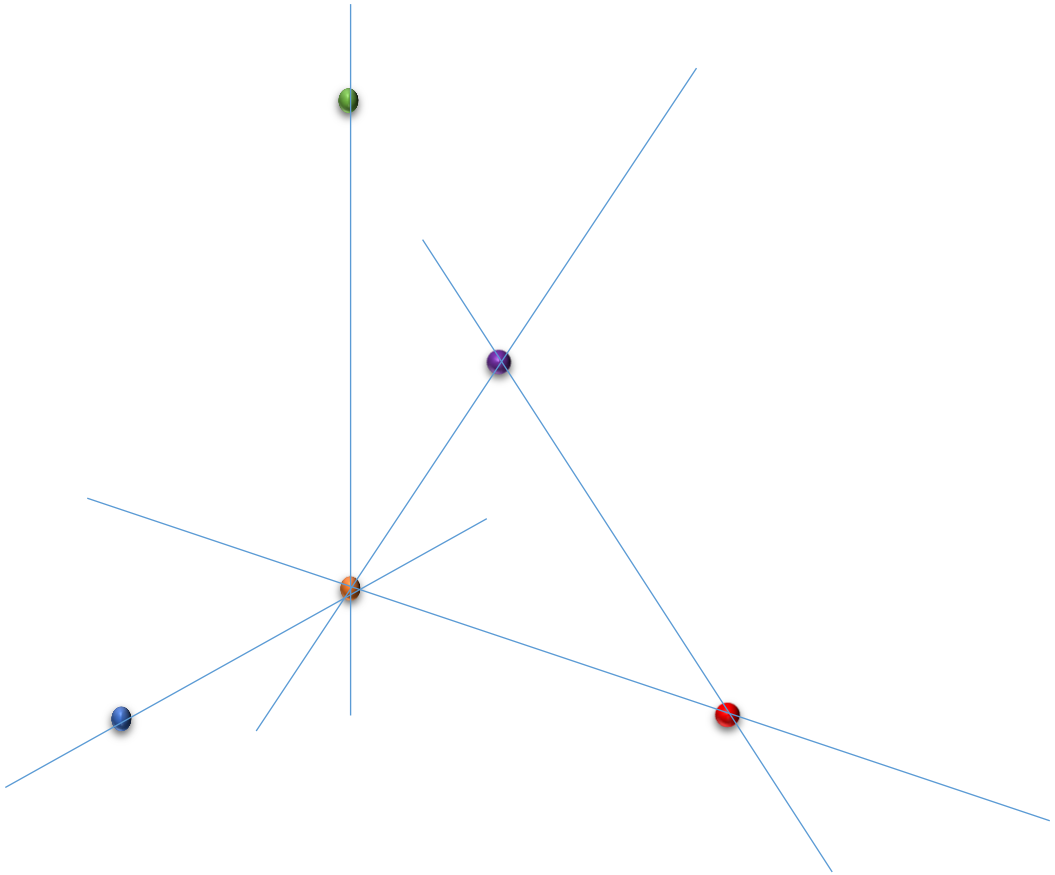
Basics: Points vs Vectors

- A point is … a position or a location in space.
- Many points … form a shape or feature in space
- This is independent of the dimensions of the space



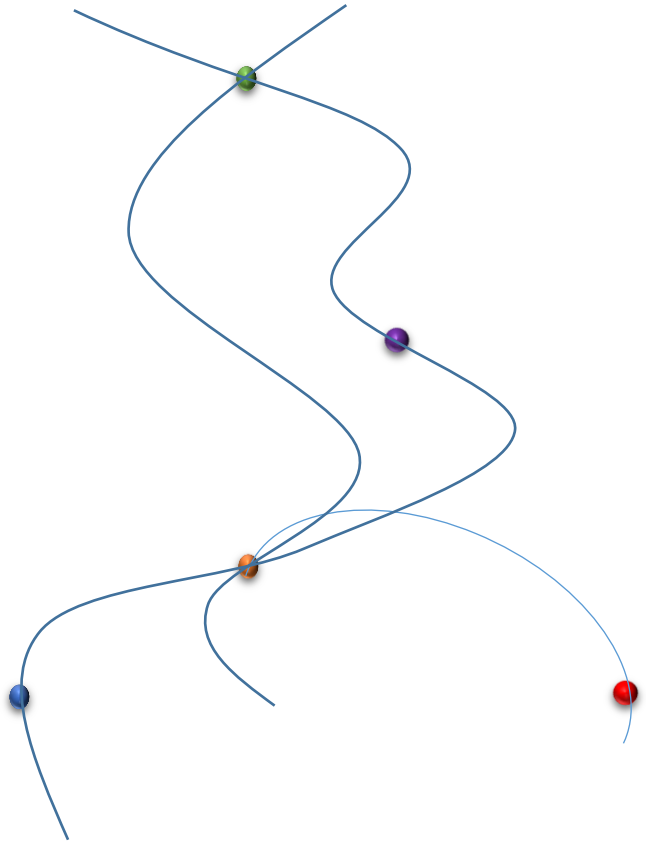
Basics: Points vs Vectors

- Relative placement of points in space can be measured with lines.



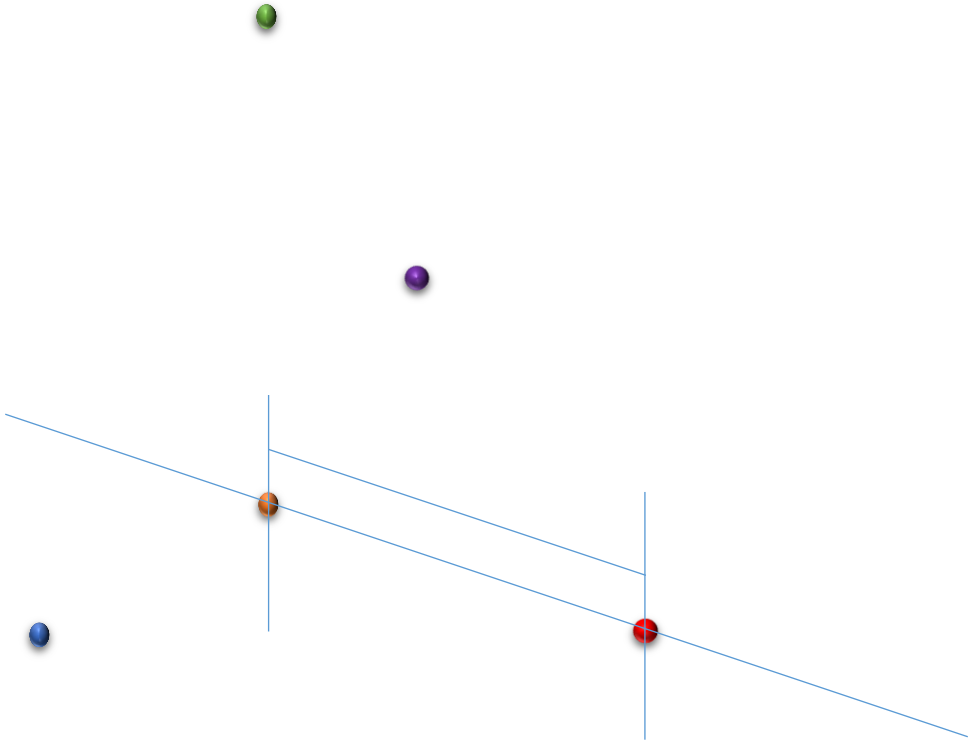
Basics: Points vs Vectors

- Relative placement of points in space can be measured with lines.
- We can also use curves!
- But, that is a bit arbitrary.
- We need more structure.



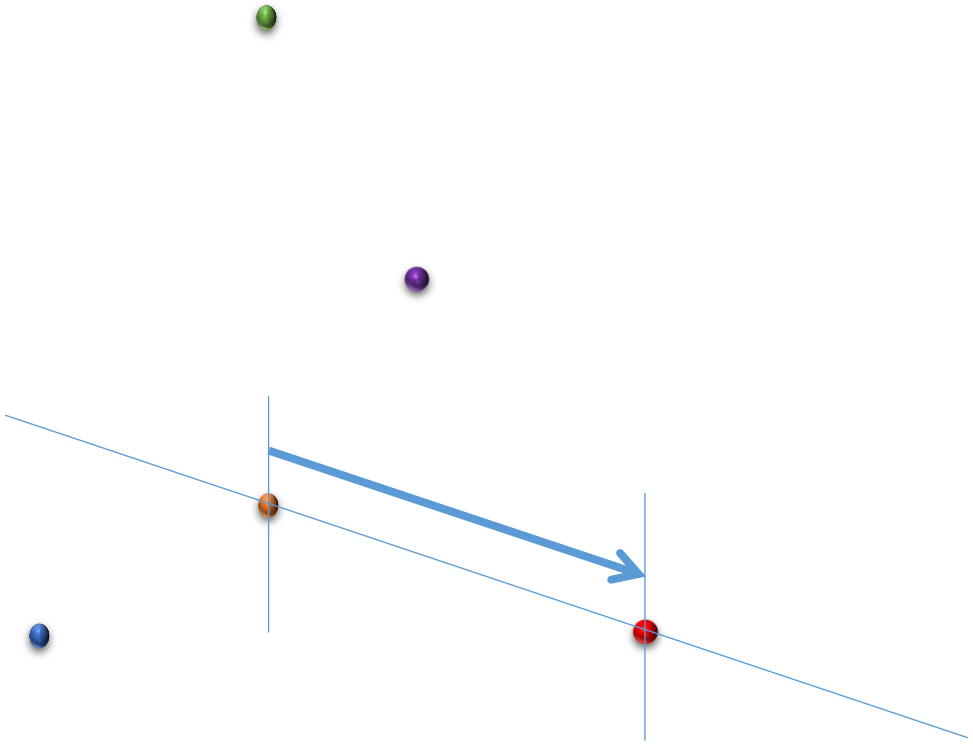
Basics: Points vs Vectors

- In absence of any other structure (e.g. spherical) we can maintain a linear displacement
- So, we can measure the relative distance between two points, but …



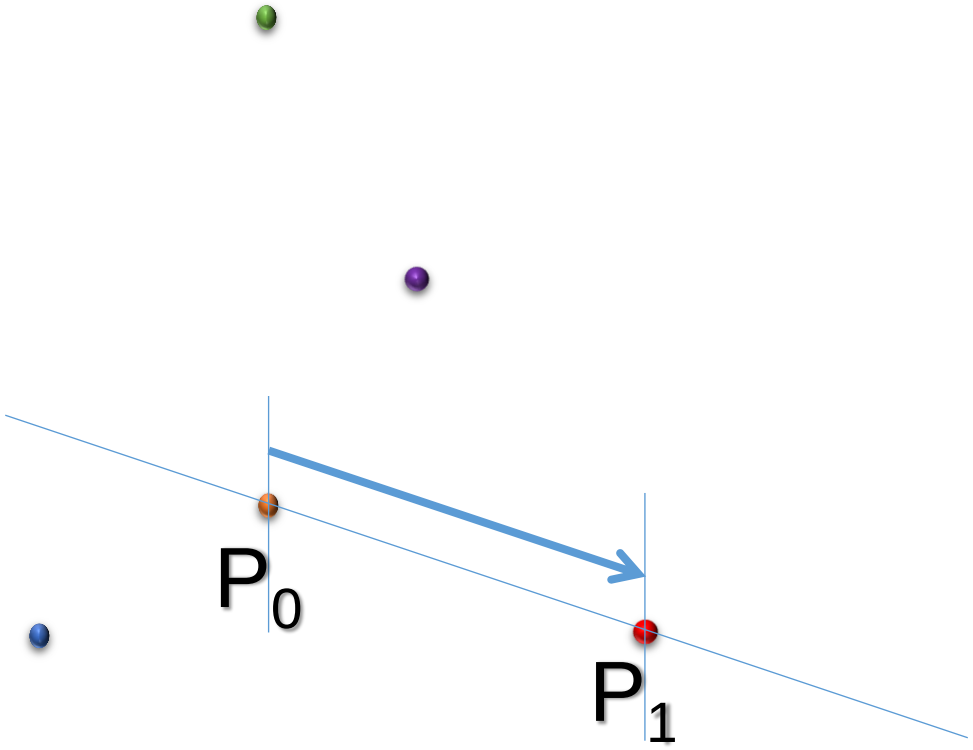
Basics: Points vs Vectors

- We need an order …



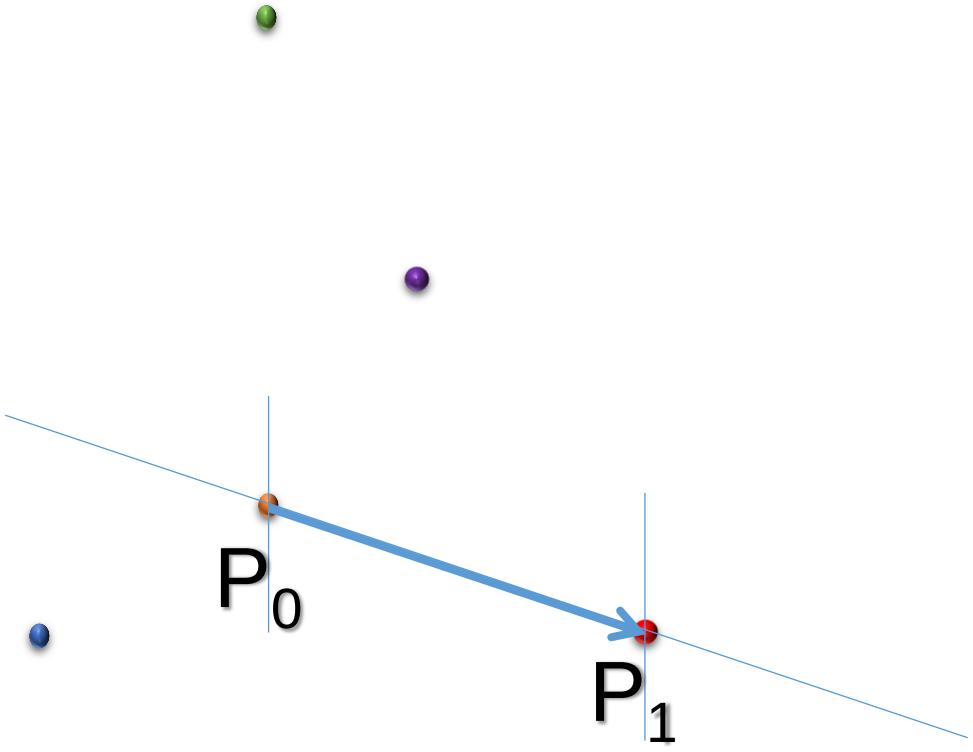
Basics: Points vs Vectors

- In other words:
  - We start at P_0
  - We end at P_1



Basics: Points vs Vectors

- In other words:
  - We start at P_0
  - We end at P_1
- This leads to the idea of a vector
  - We start at P_0
  - Move in the direction of P_1



Basics: Points vs Vectors

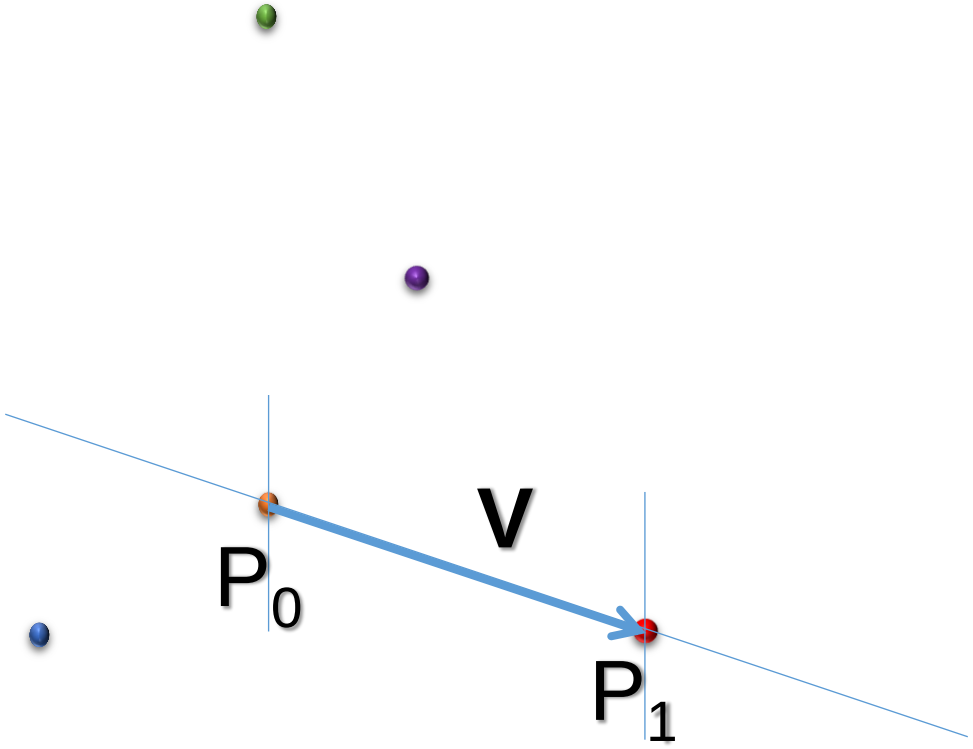
- So, we say that a vector is

$$\mathbf{V} = P_1 - P_0$$

- In other words:

$$P_1 = P_0 + \mathbf{V}$$

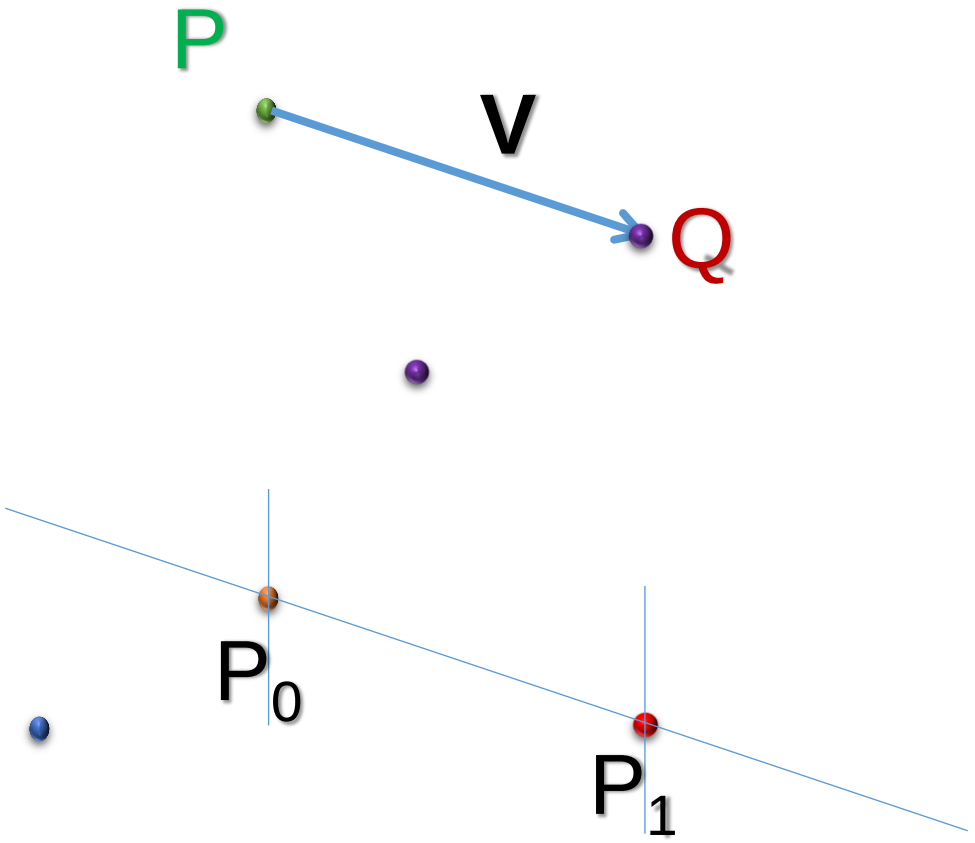
- That is, if we start at P_0 we can get to P_1 after moving along $\mathbf{V}$.



Basics: Points vs Vectors

- More amazing, we can use the same **V** to get from any other point in space **P** to another point **Q**

$$Q = P + \mathbf{V}$$

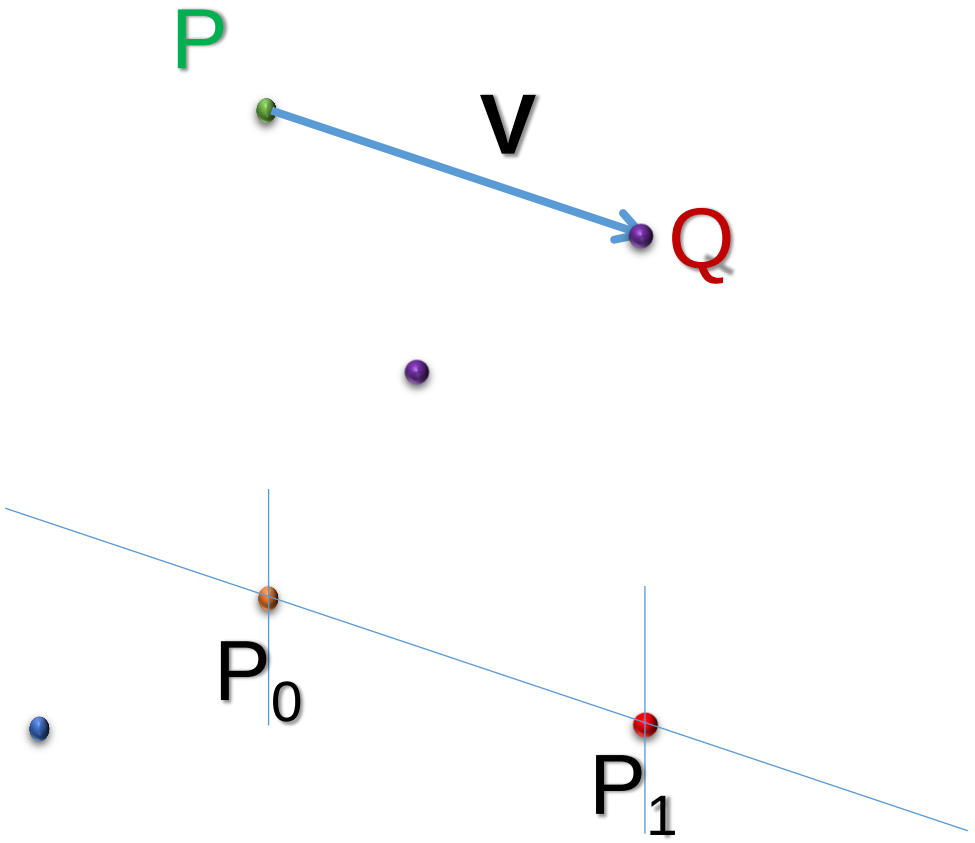


Basics: Points vs Vectors

- More amazing, we can use the same **V** to get from any other point in space **P** to another point **Q**

$$Q = P + \mathbf{V}$$

- That means, we can move **V** to any point in space



Point-Vector Algebra

- This suggests a point-vector algebra:

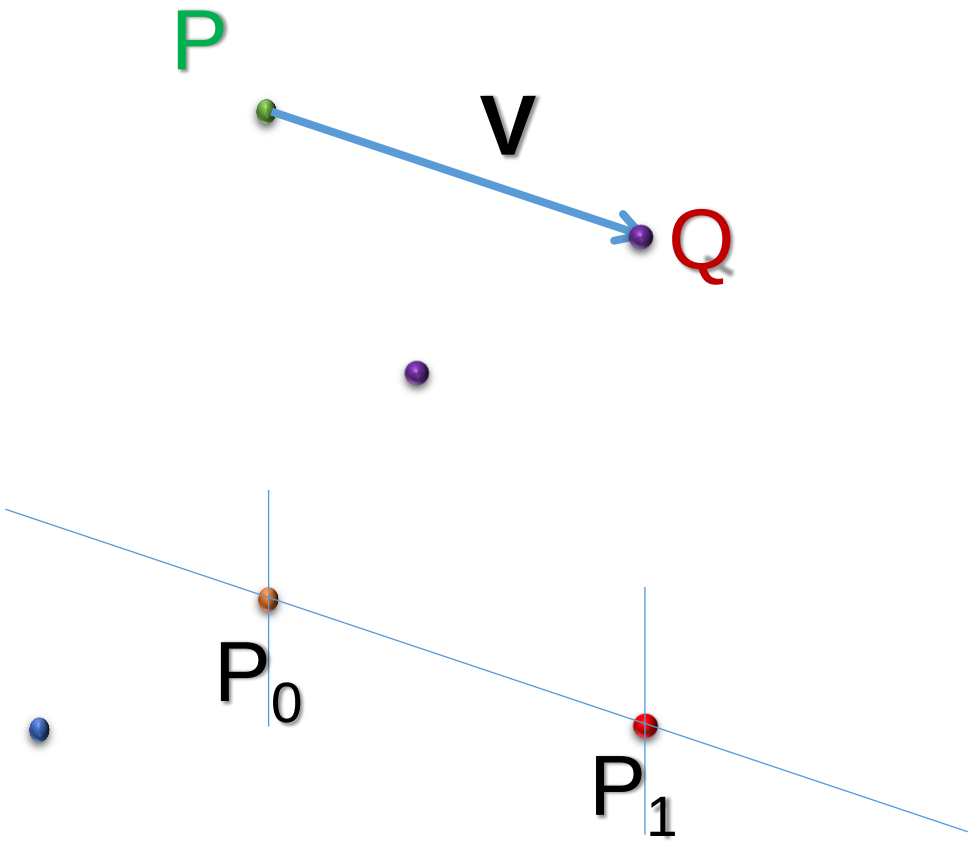
$$Q = P + \mathbf{V}$$

- That means:

Point = Point + Vector

- **Hence,**

Vector = Point - Point



Multiple Points

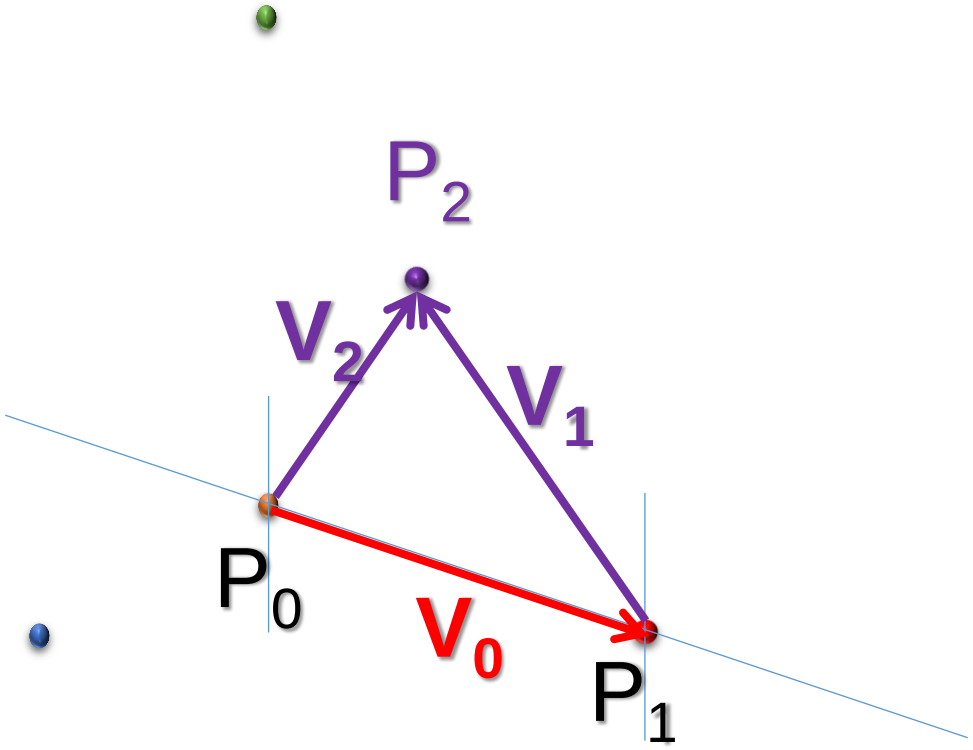
- Instead of intermediate operations, we can use vectors between points

$$P_2 = P_0 + V_0 + V_1$$

- That means:

$$P_2 - P_0 = V_0 + V_1$$

$$V_2 = V_0 + V_1$$



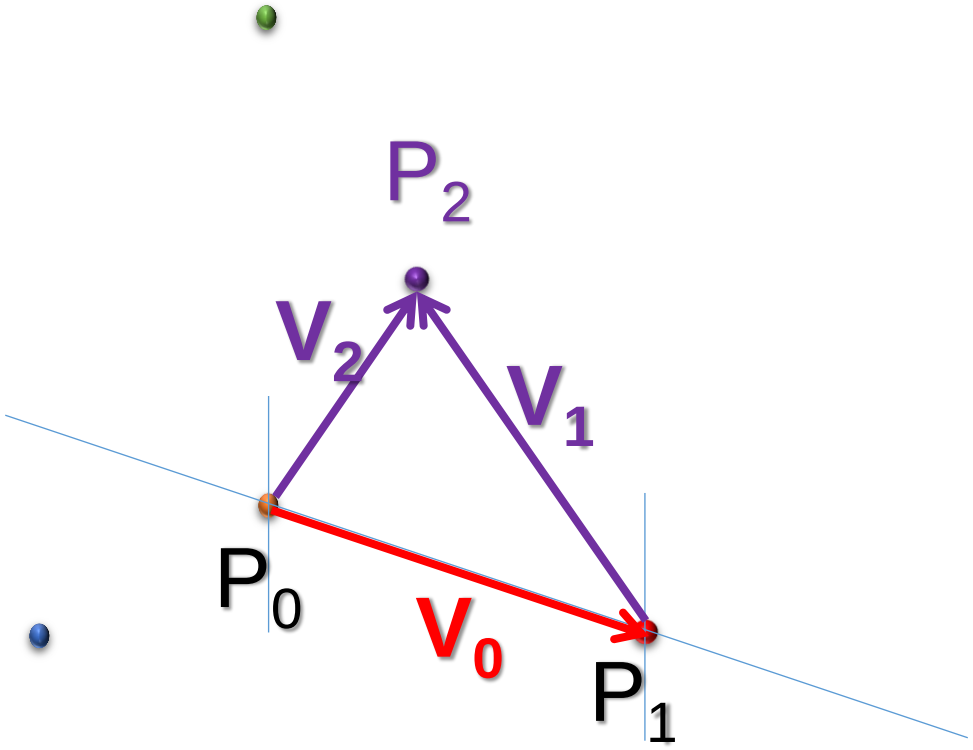
Vector Algebra

- This leads to vector algebra:
- Vector addition

$$\mathbf{V_2} = \mathbf{V_0} + \mathbf{V_1}$$

- Vector subtraction

$$\mathbf{V_2} - \mathbf{V_0} = \mathbf{V_1}$$



Vector Normalization

- Now, let's make **V** unit length:

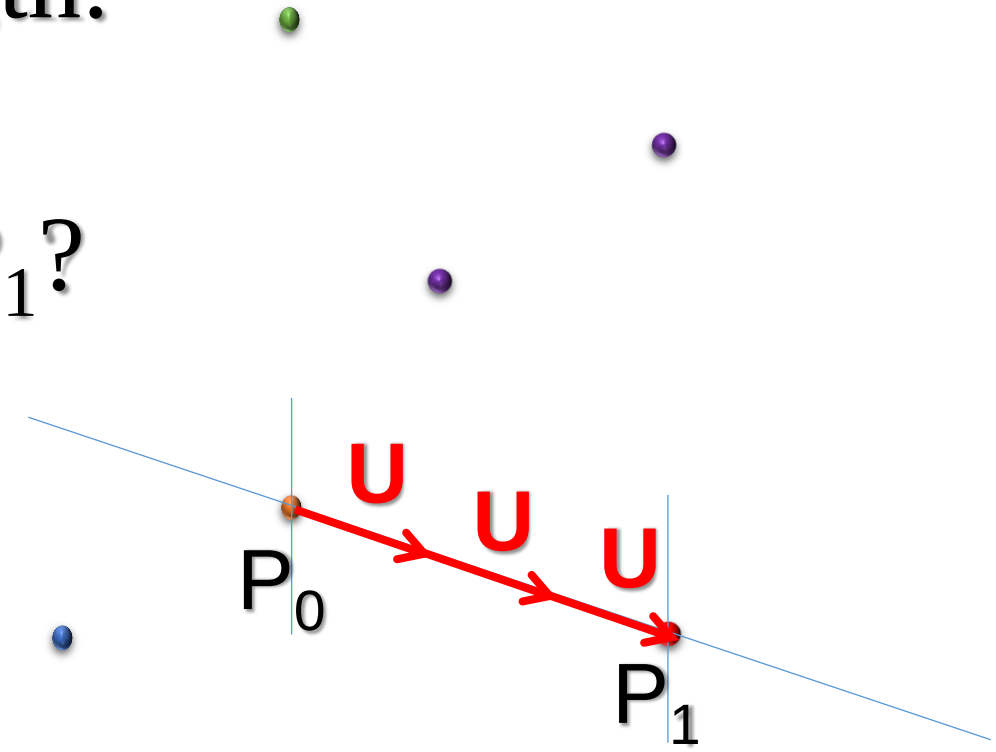
$$\|\mathbf{U}\| = 1$$

- How do we get from P_0 to P_1?

$$P_1 = P_0 + 3\mathbf{U}$$

- This means that:

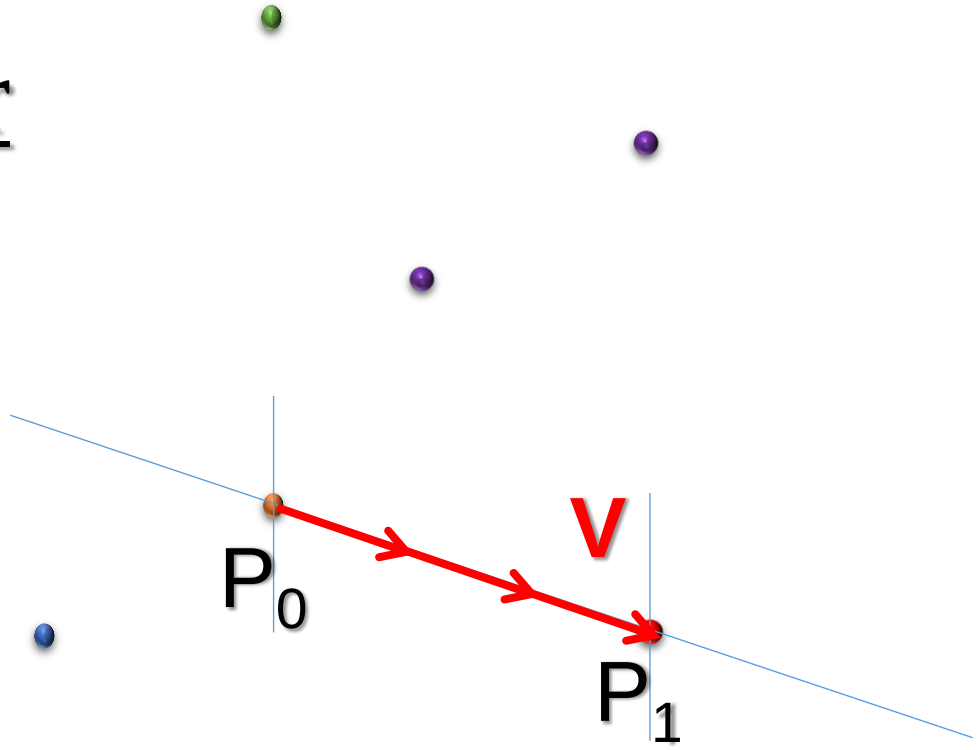
$$P_1 - P_0 = \mathbf{V} = 3\mathbf{U}$$



Scalar multiplication

- In general, we can multiply any unit vector **U** by a scalar number *s*
- This scales **U** indefinitely

$$\mathbf{V} = s\mathbf{U}$$



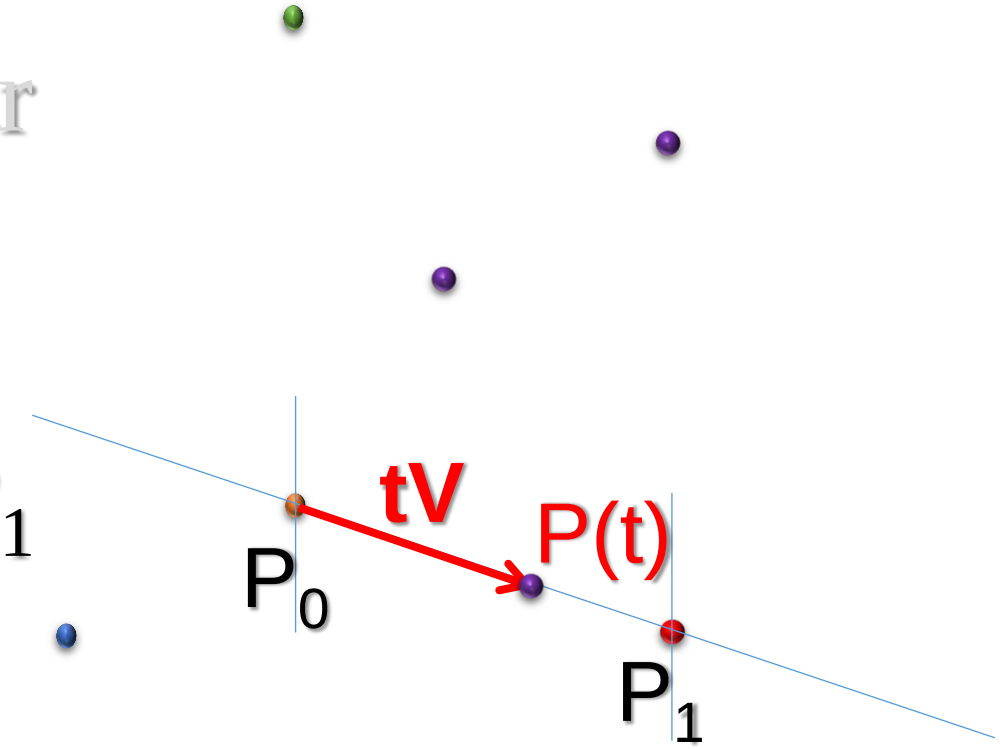
Any point on a line

- In general, we can multiply any unit vector U by a scalar number *s*

$$\mathbf{V} = s\mathbf{U}$$

- Hence, a point on a line $\overline{P_0P_1}$

$$\mathbf{P}(t) = P_0 + t\mathbf{V}$$



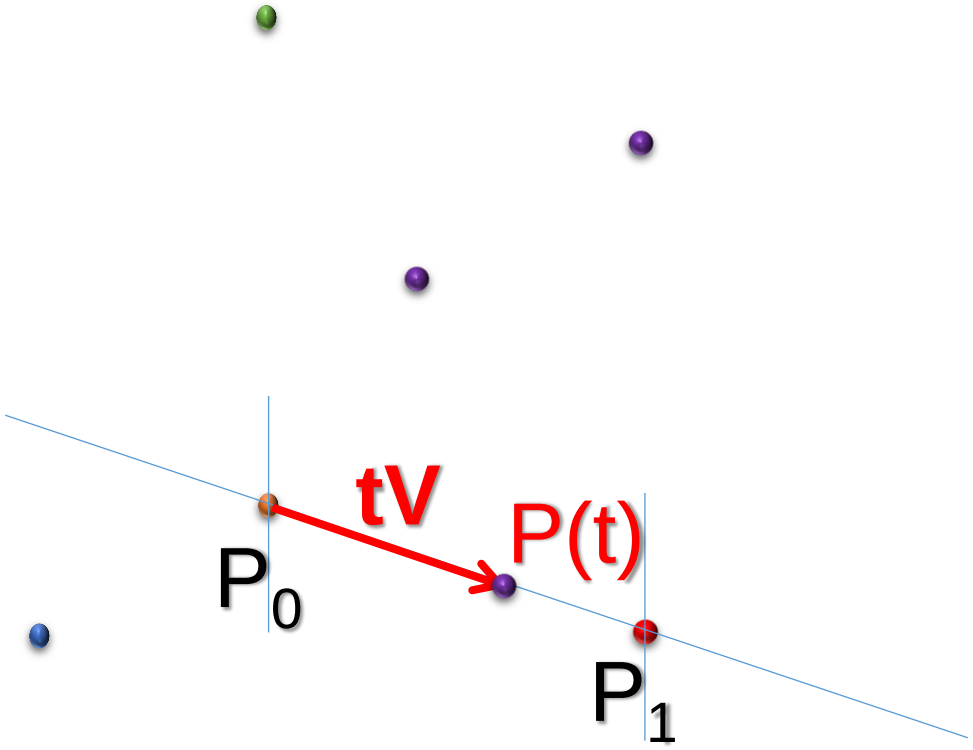
Any point on a line

- But,

$$V = P_1 - P_0$$

- So, any point on line $\overline{P_0P_1}$

$$P(t) = P_0 + t\,(P_1 - P_0)$$



Linear Interpolation

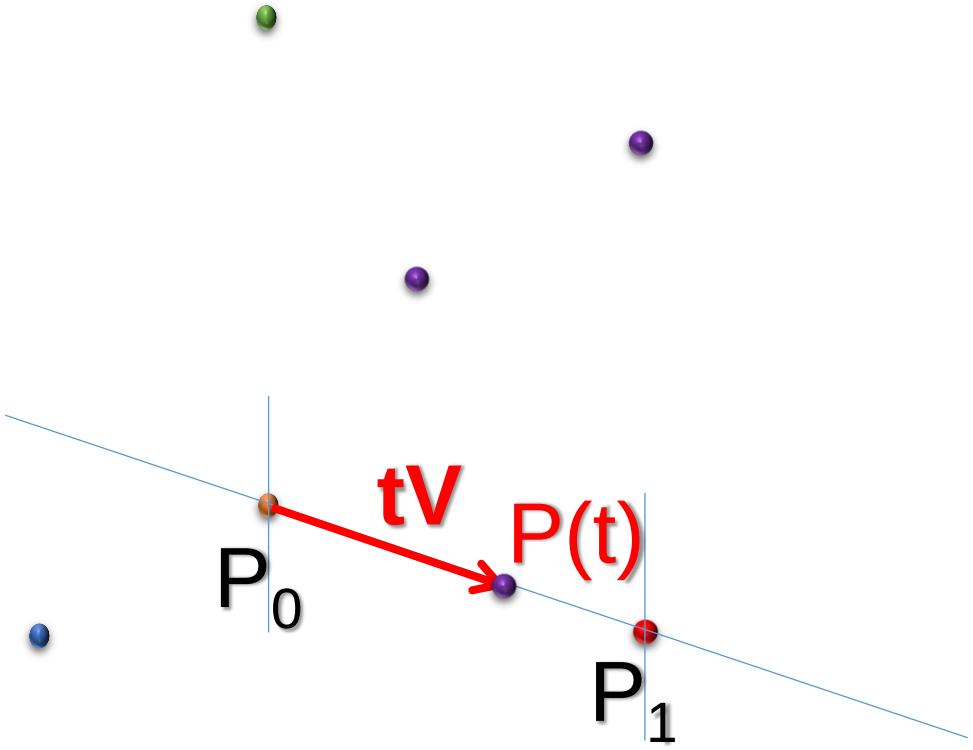
- But,

$$V = P_1 - P_0$$

- Linear Interpolation!

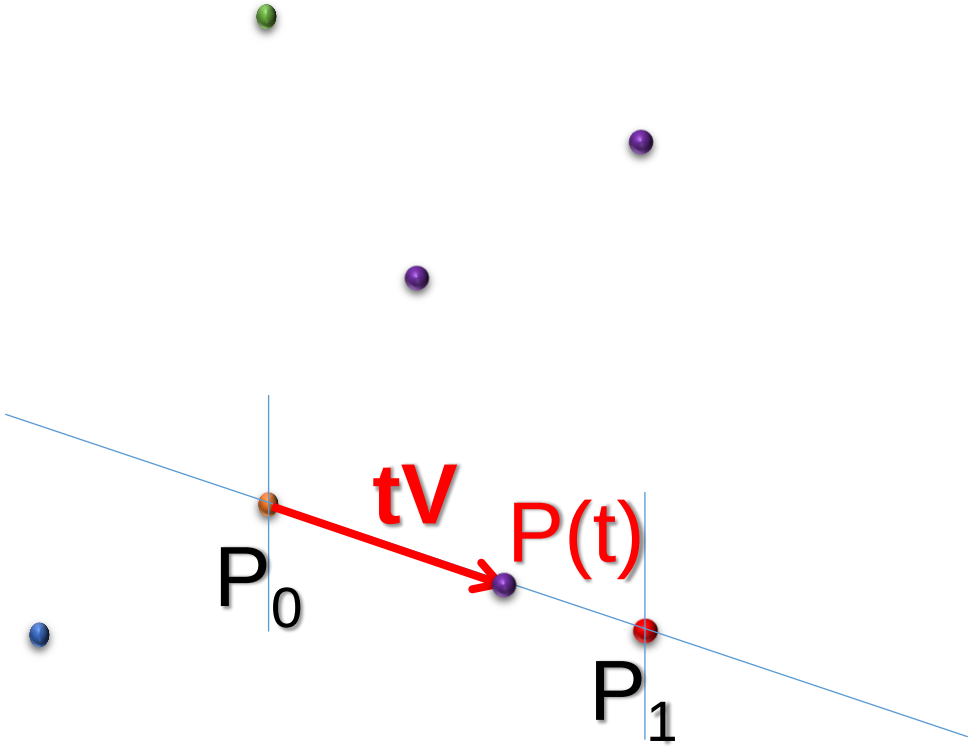
$$P(t) = P_0 + t\,(P_1 - P_0)$$

- Any point on a line is determined by a single scalar t! and two points!



No coordinates!

- So far all operations were strictly geometric!
- No coordinates!
- But, there are more!

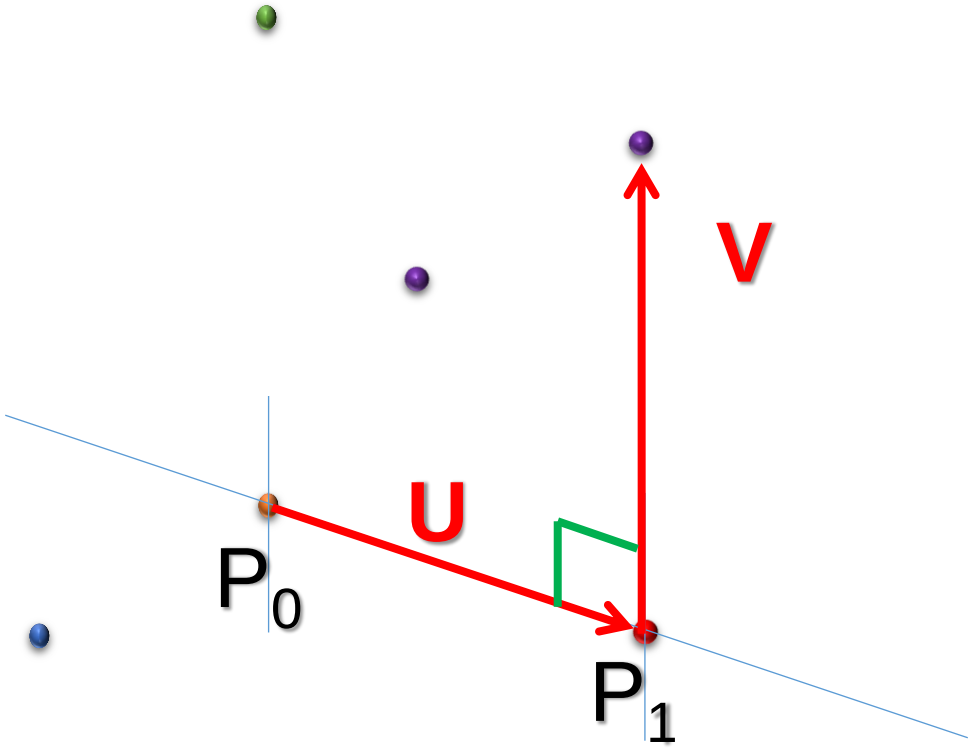


Perpendicularity

- Two vectors U and V are ⊥

$$\mathbf{U} \bullet \mathbf{V} = 0$$

- What does this mean geometrically?

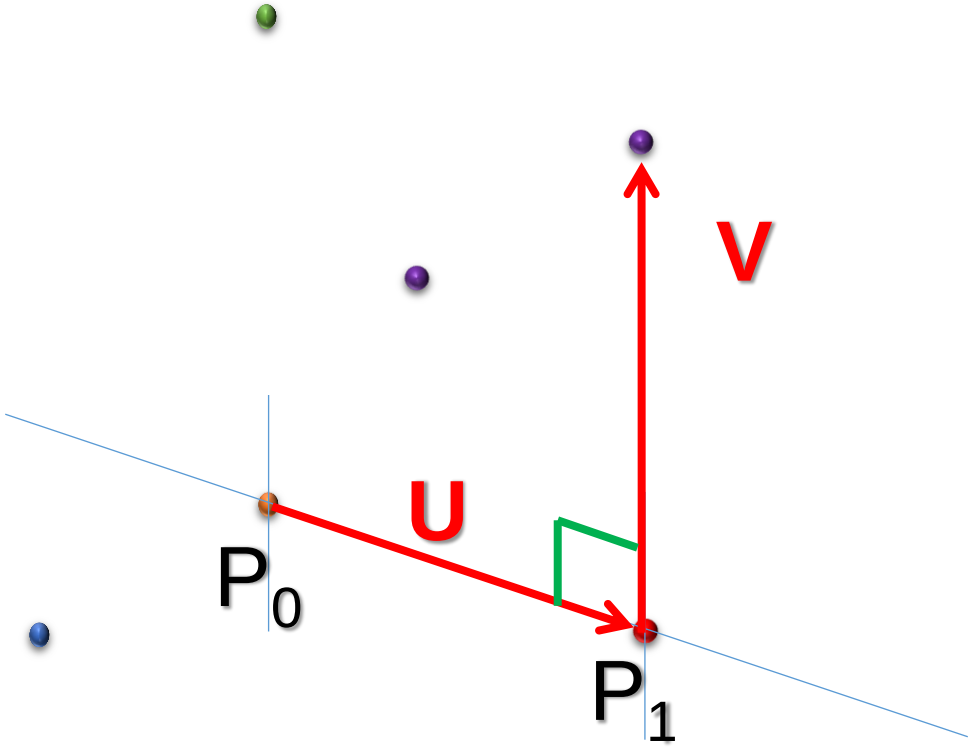


Perpendicularity

- Two vectors U and V are $\perp$

$$U \bullet V = 0$$

- Geometrically, it means that that V has no shadow on U and U has no shadow on V
- This means, PROJECTION



Vector Projections

- For two vectors U and V

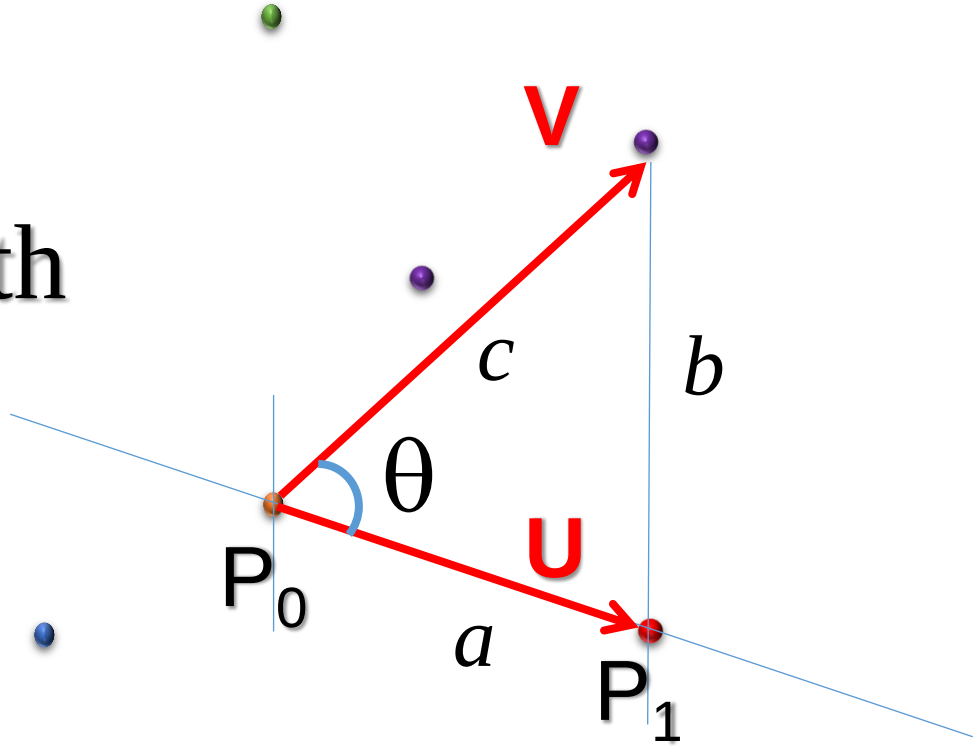
$$\mathbf{U} \bullet \mathbf{V} = |U||V| \cos \theta$$

- When U and V are unit length

$$\mathbf{U} \bullet \mathbf{V} = \cos \theta$$

- Why? Right angle triangles

$$\frac{a}{c} = \cos\theta \rightarrow a = c\cos\theta$$



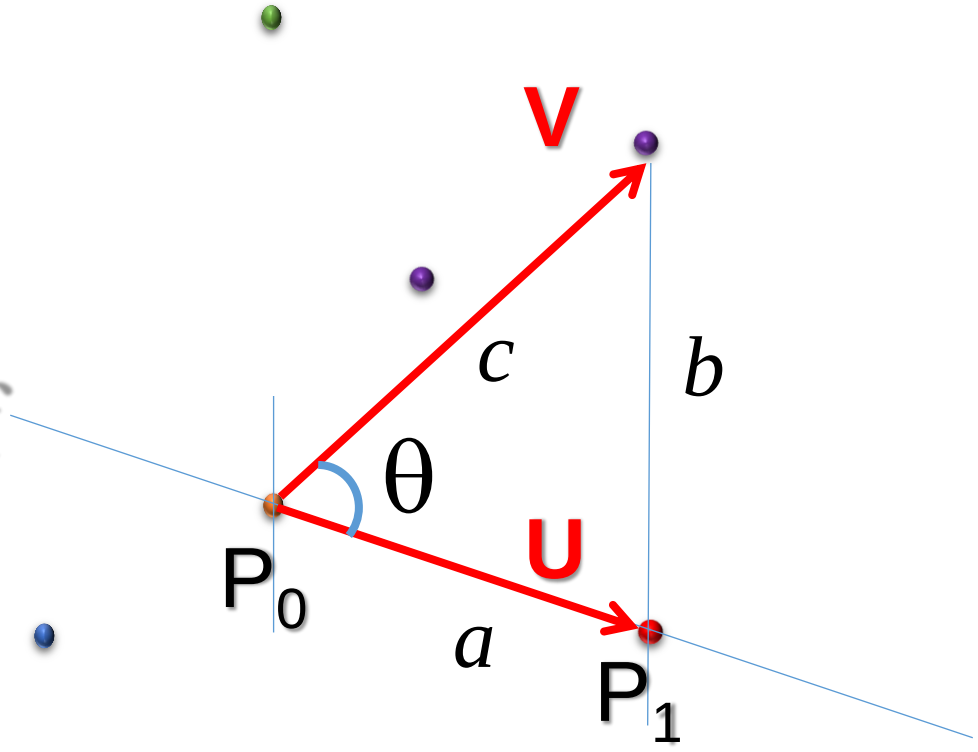
Vector Projections

- Right angle triangles

$$\frac{a}{c} = \cos\theta \rightarrow a = c\cos\theta$$

- In other words, if U is the unit vector, the projection of V onto U is:

$$\mathbf{V'} = (|\mathbf{V}|\cos\theta)\mathbf{U}$$



Vector Dot Product

- For two vectors U and V

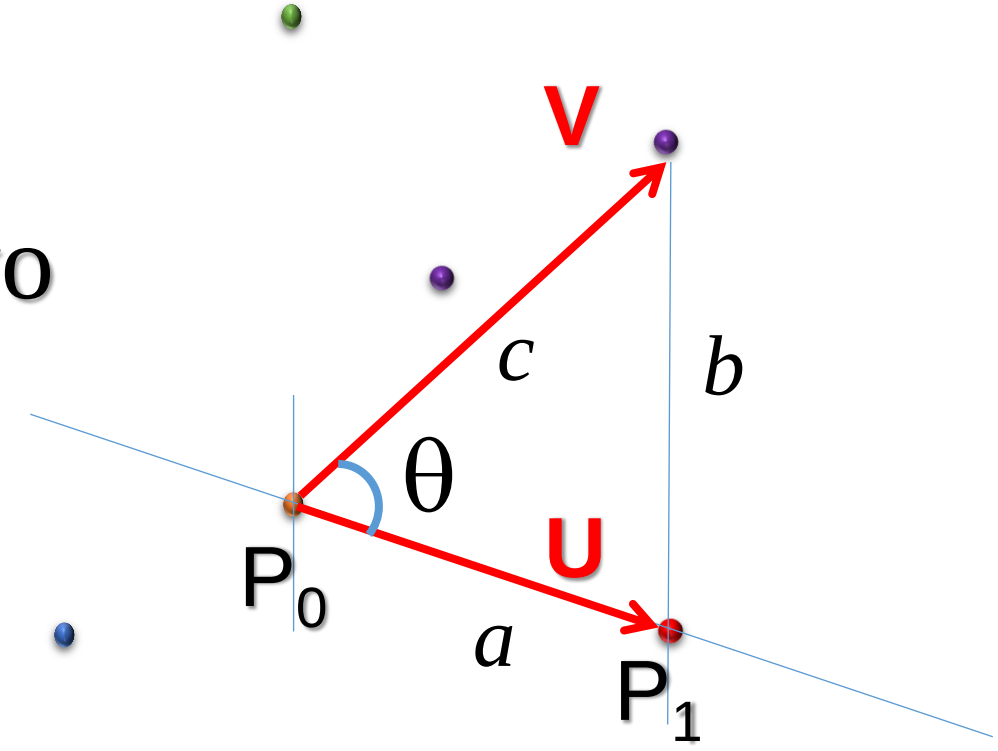
$$\mathbf{U} \bullet \mathbf{V} = |U||V| \cos \theta$$

- The dot product between two vectors is a Projection!

$$\mathbf{U} \bullet \mathbf{V} = (|\mathbf{V}| \cos \theta)\mathbf{U}$$

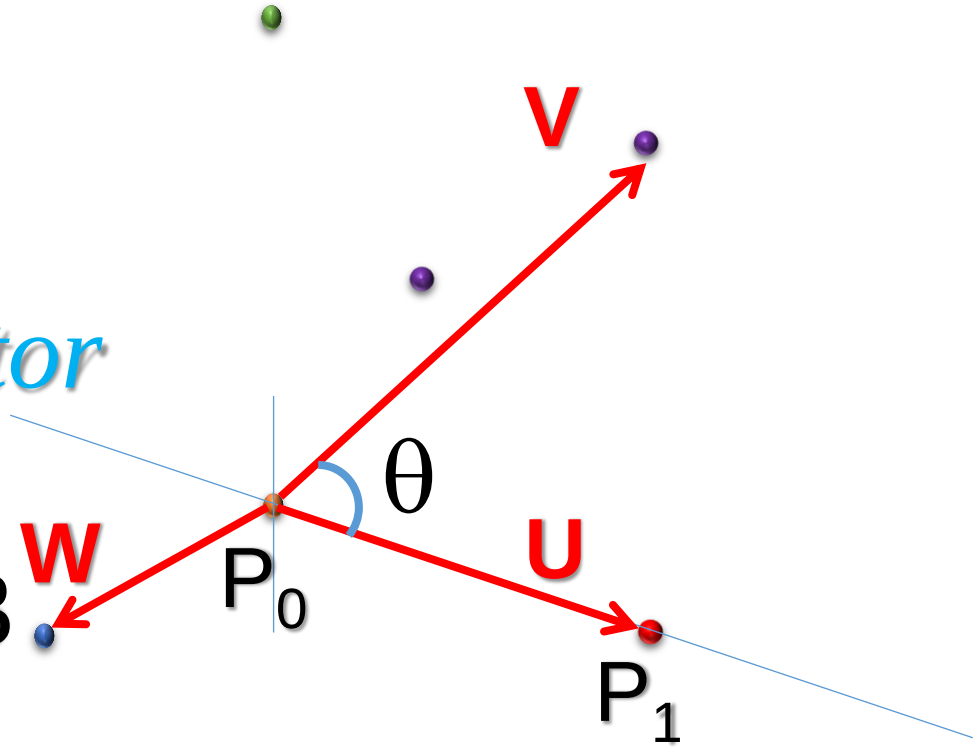
$$\mathbf{U} \bullet \mathbf{V} = (|\mathbf{U}| \cos \theta)\mathbf{V}$$

Unit vectors!



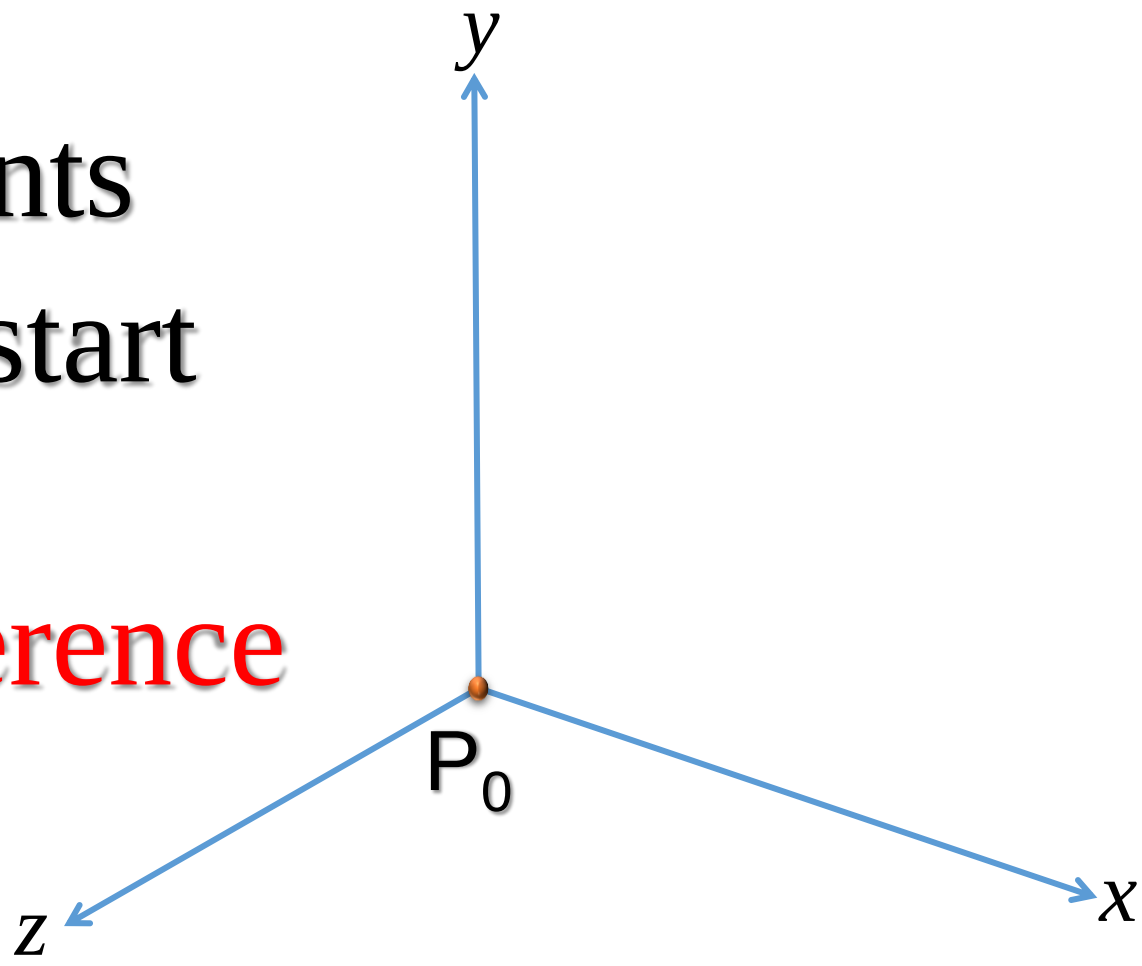
Vector Cross Product

- For two vectors U and V
  $$\mathbf{U} \times \mathbf{V} = (|\mathbf{U}||\mathbf{V}| \sin\theta)\mathbf{W}$$
- The cross product between two vectors is a *Normal Vector* to the plane of U and V.
- Joseph Louis Lagrange 1773



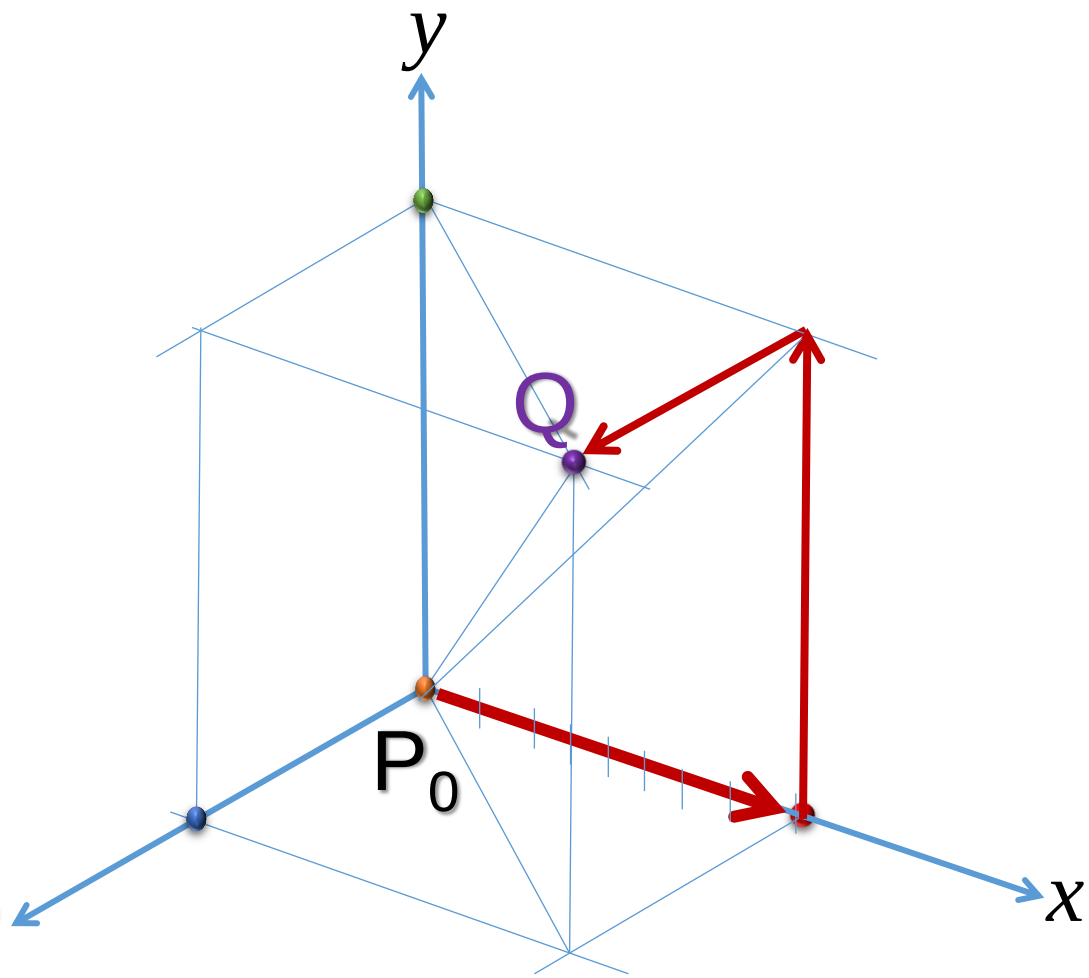
Space Reference

- We started with points
- Hence, space must start with a point P_0
- We call this the reference point



Set up Measurements

- We need
  - Three non-collinear vectors
  - Units of measure
- We call these numbers coordinates

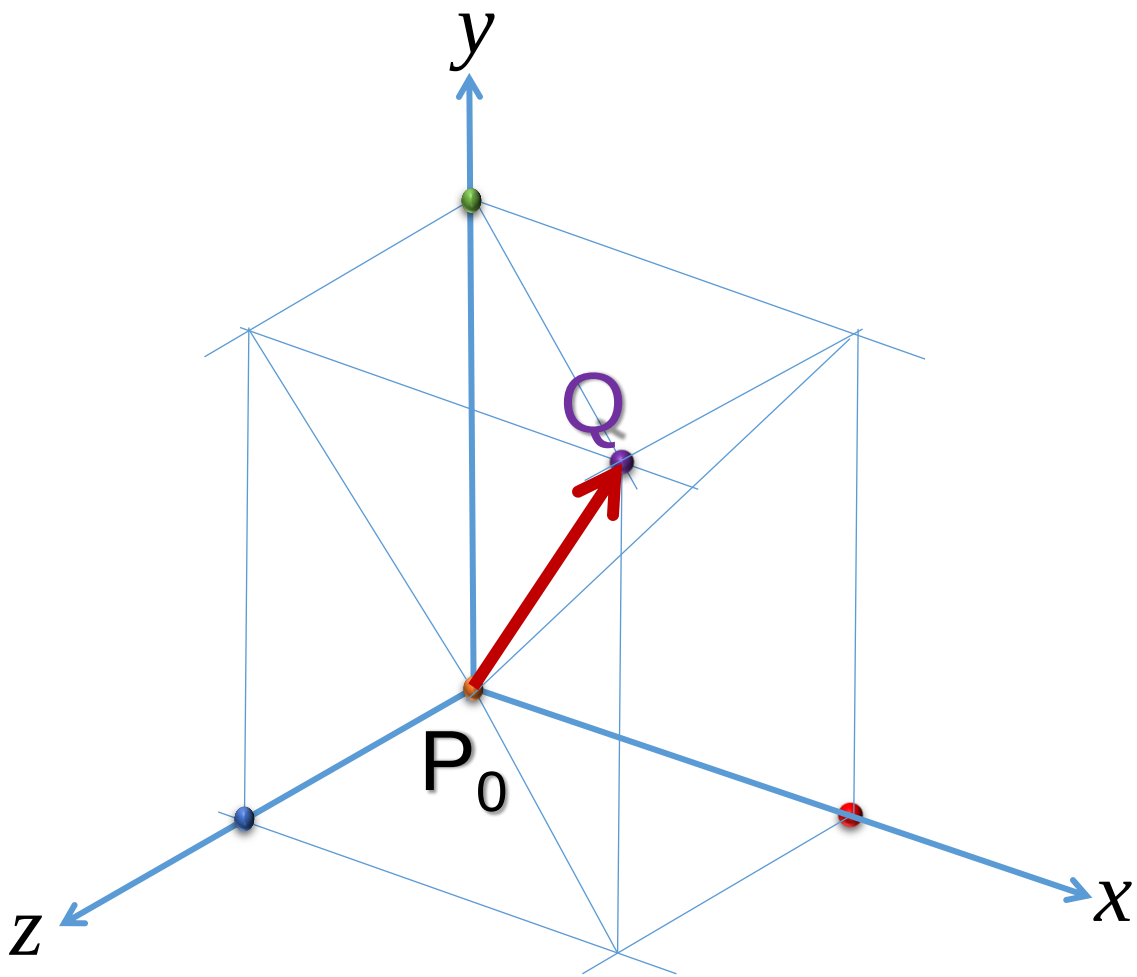


Position Vector PQ

- Vector

$$PQ = Q - P_0$$

- A position vector!



PQ Projection

- Onto the x-axis, $\|\mathbf{u}\|=1$

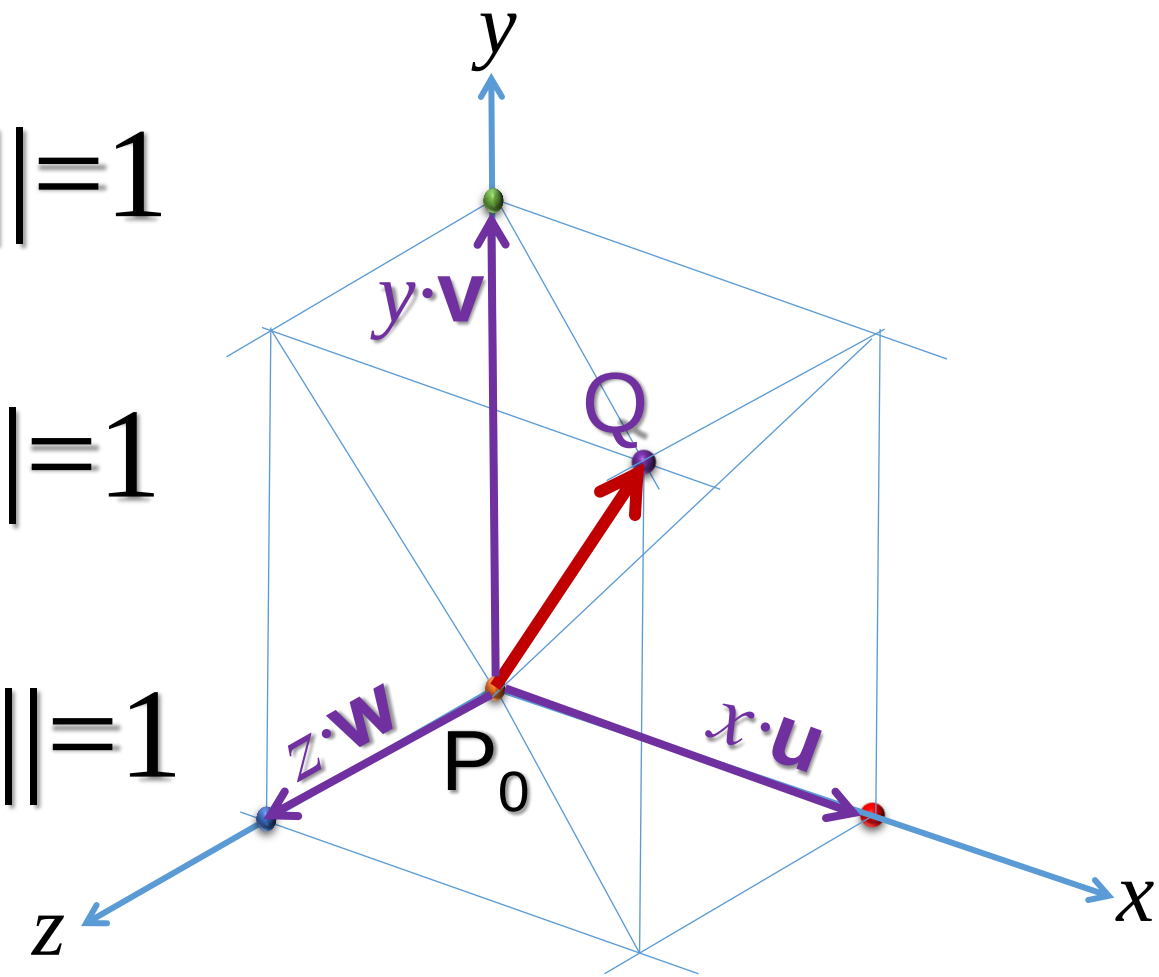
$$PQ \bullet \mathbf{u} = x$$

- Onto the y-axis, $\|\mathbf{v}\|=1$

$$PQ \bullet \mathbf{v} = y$$

- Onto the z-axis, $\|\mathbf{w}\|=1$

$$PQ \bullet \mathbf{w} = z$$



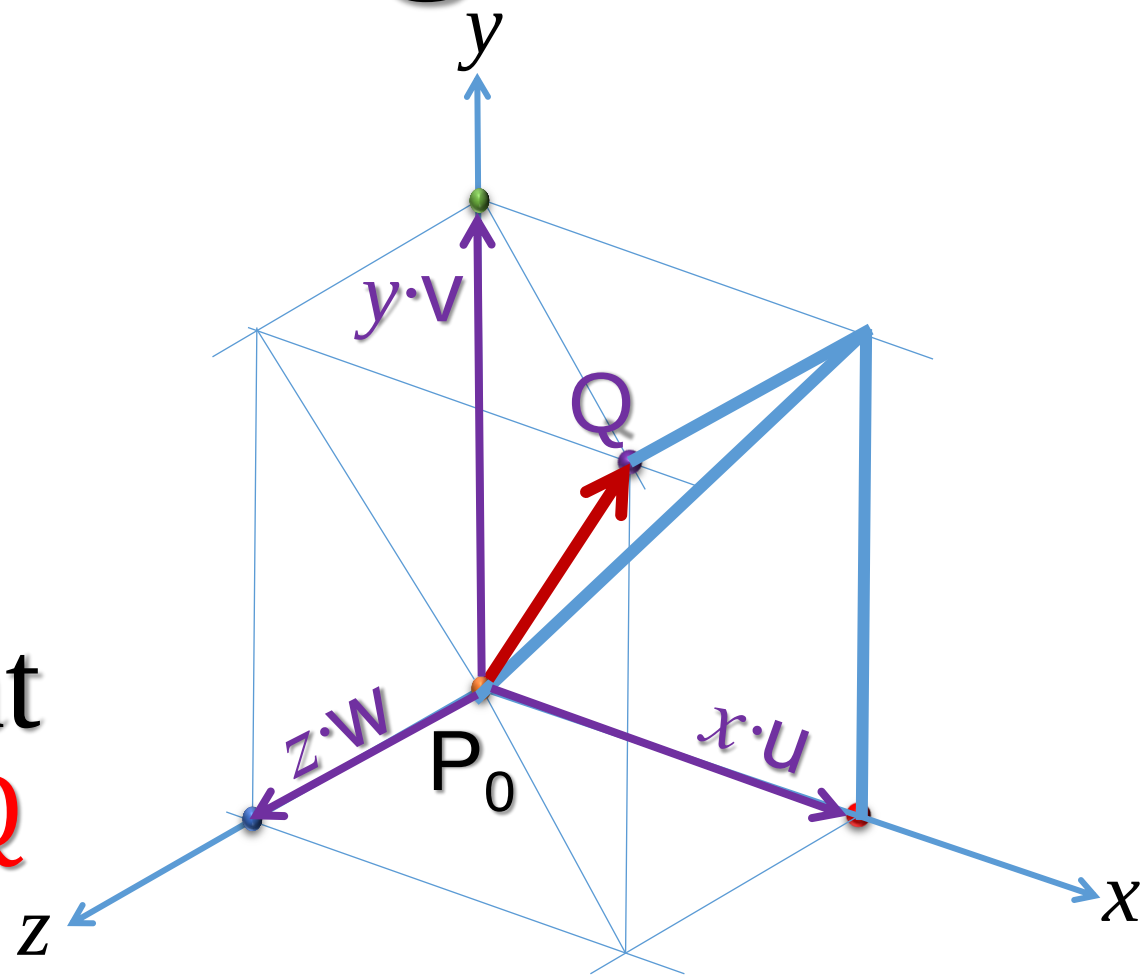
Unit vectors along axes

- Axes projections

$$PQ \bullet u = x$$
$$PQ \bullet v = y$$
$$PQ \bullet w = z$$

- This also means that

$$x\,\mathbf{u} + y\,\mathbf{v} + z\,\mathbf{w} = PQ$$



Note: all coordinate free!

- From

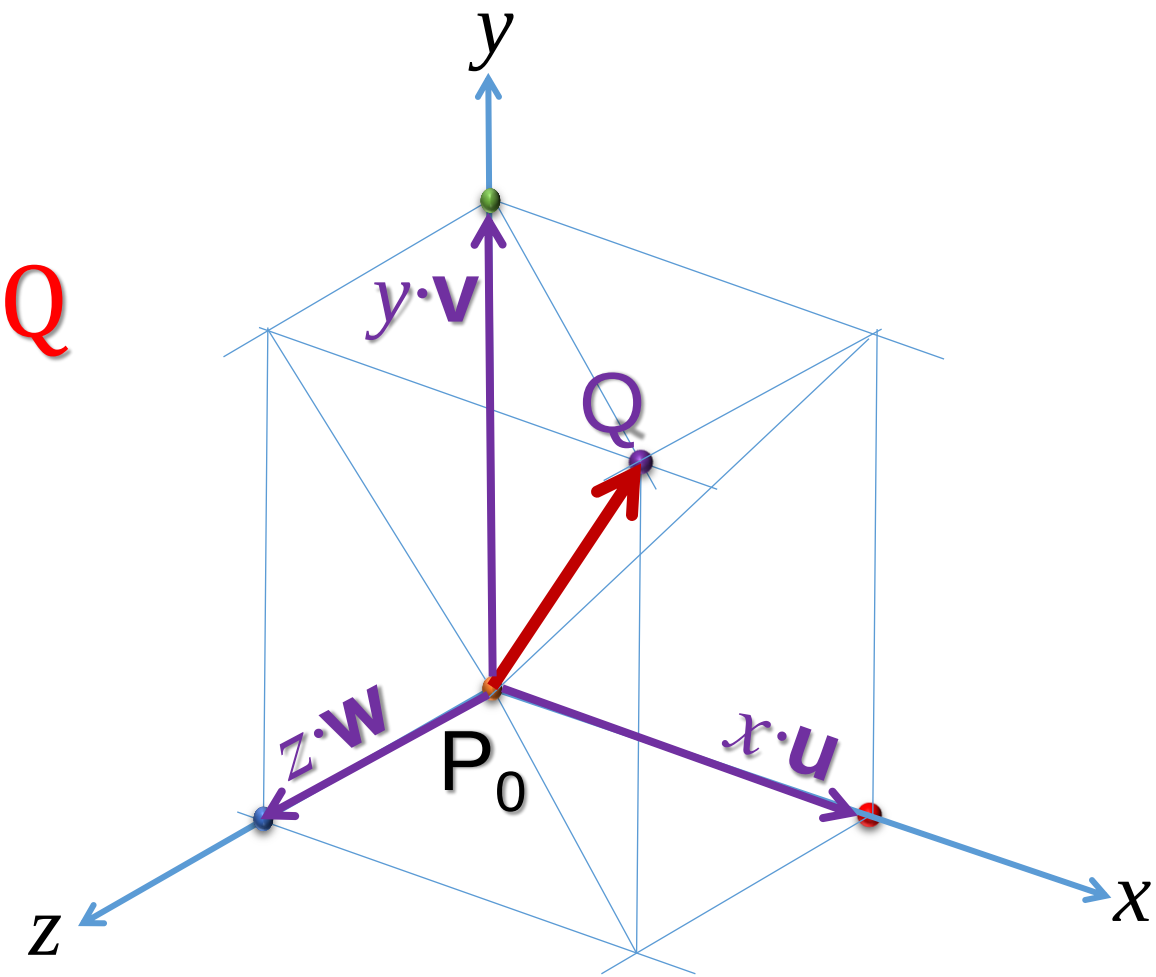
$$x\,\mathbf{u} + y\,\mathbf{v} + z\,\mathbf{w} = \mathrm{PQ}$$

We say that:

x represents PQ along **u**

y represents PQ along **v**

z represents PQ along **w**



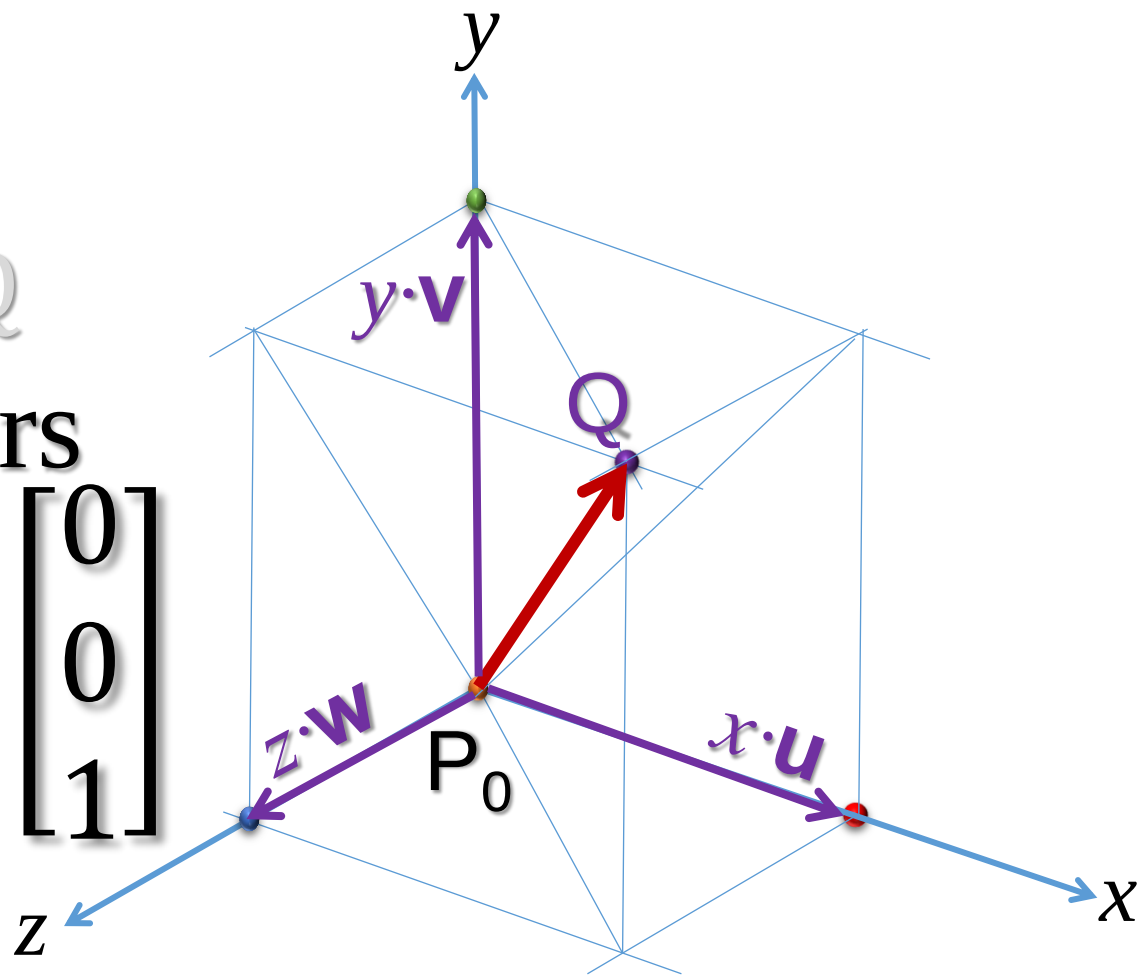
Canonical Unit Vectors

- This also means that

$$x\,\mathbf{u} + y\,\mathbf{v} + z\,\mathbf{w} = \mathrm{PQ}$$

- Canonical Unit Vectors

$$\mathbf{u} = \begin{bmatrix}1\\0\\0\end{bmatrix} \quad \mathbf{v} = \begin{bmatrix}0\\1\\0\end{bmatrix} \quad \mathbf{w} = \begin{bmatrix}0\\0\\1\end{bmatrix}$$



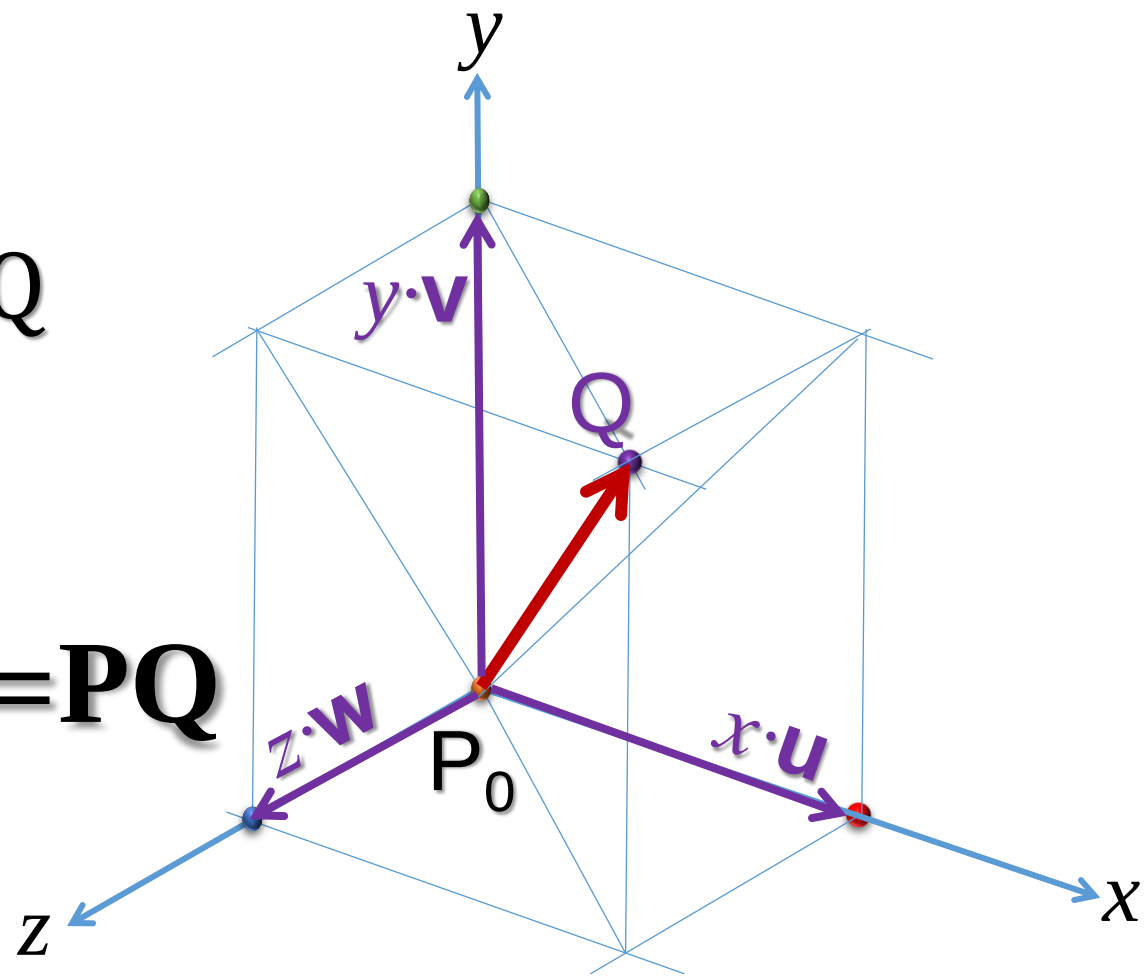
Canonical Unit Vectors

- Then,

$$x\,\mathbf{u} + y\,\mathbf{v} + z\,\mathbf{w} = \mathrm{PQ}$$

- Gives

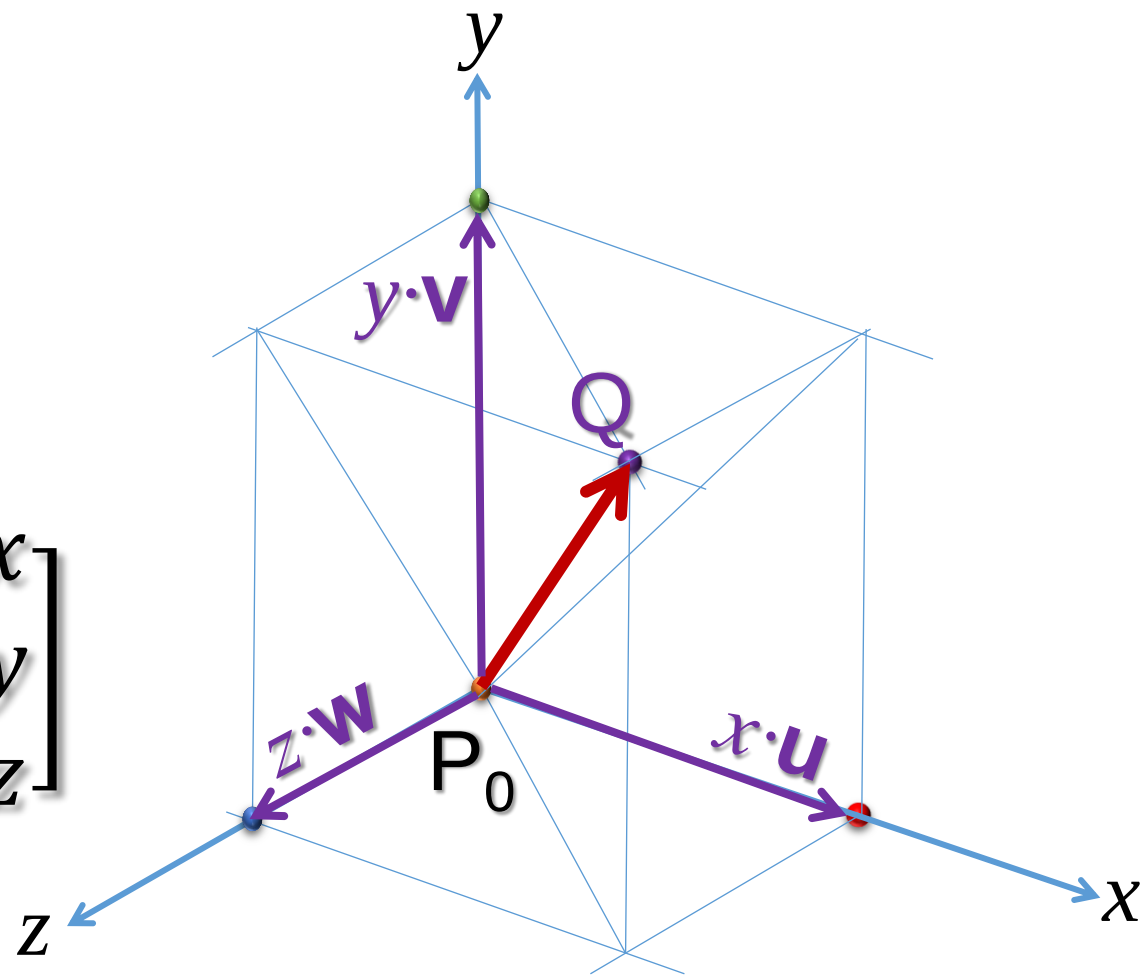
$$x\begin{bmatrix}1\\0\\0\end{bmatrix} + y\begin{bmatrix}0\\1\\0\end{bmatrix} + z\begin{bmatrix}0\\0\\1\end{bmatrix} = \mathbf{PQ}$$



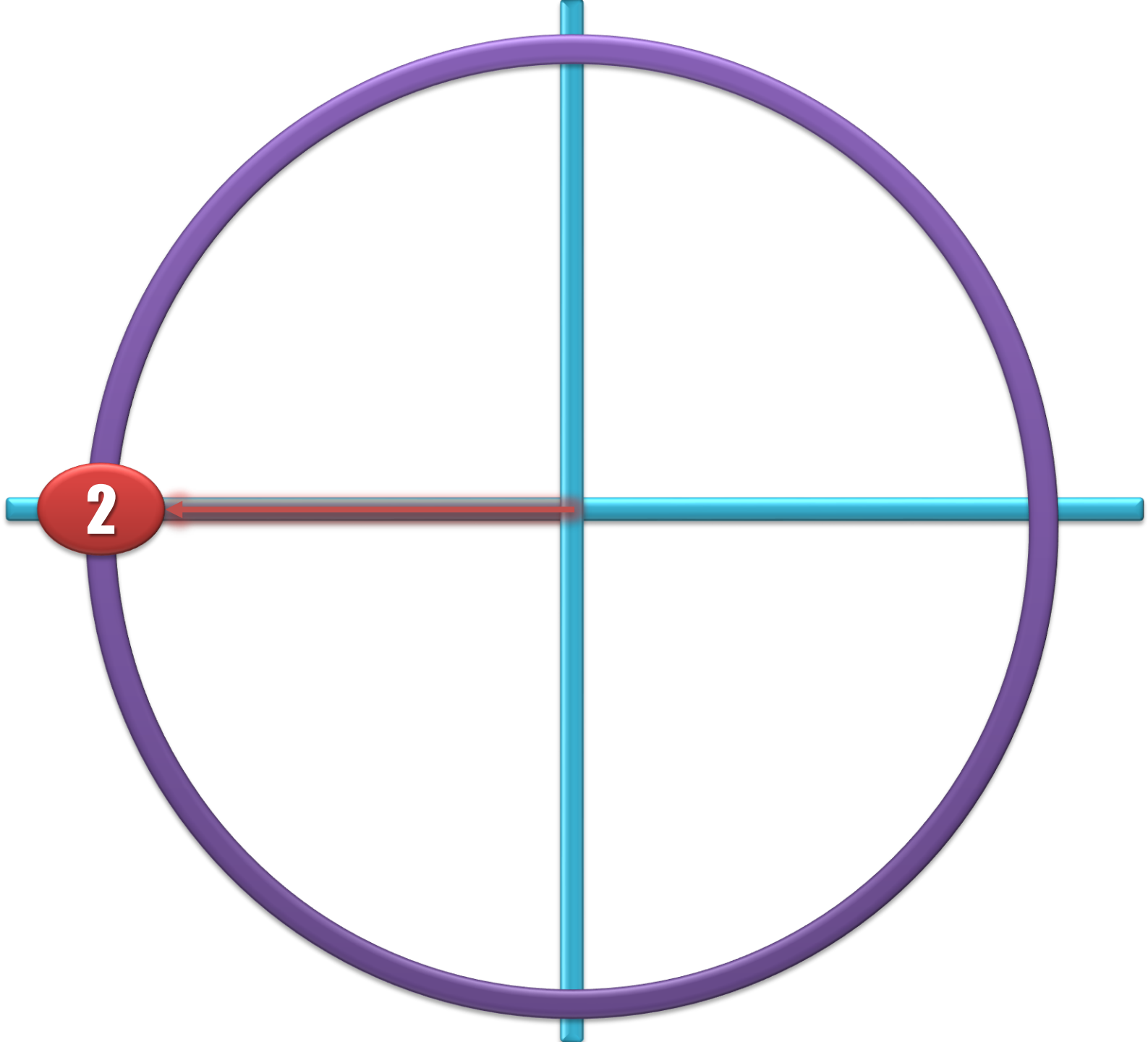
Spatial Coordinates

- Then,
  $x\,\mathbf{u} + y\,\mathbf{v} + z\,\mathbf{w} = \mathrm{PQ}$
- Gives

$$x\begin{bmatrix}1\\0\\0\end{bmatrix} + y\begin{bmatrix}0\\1\\0\end{bmatrix} + z\begin{bmatrix}0\\0\\1\end{bmatrix} = \begin{bmatrix}x\\y\\z\end{bmatrix}$$



We say that $[x, y, z]^T$ are the coordinates of PQ


2

Linear Operations

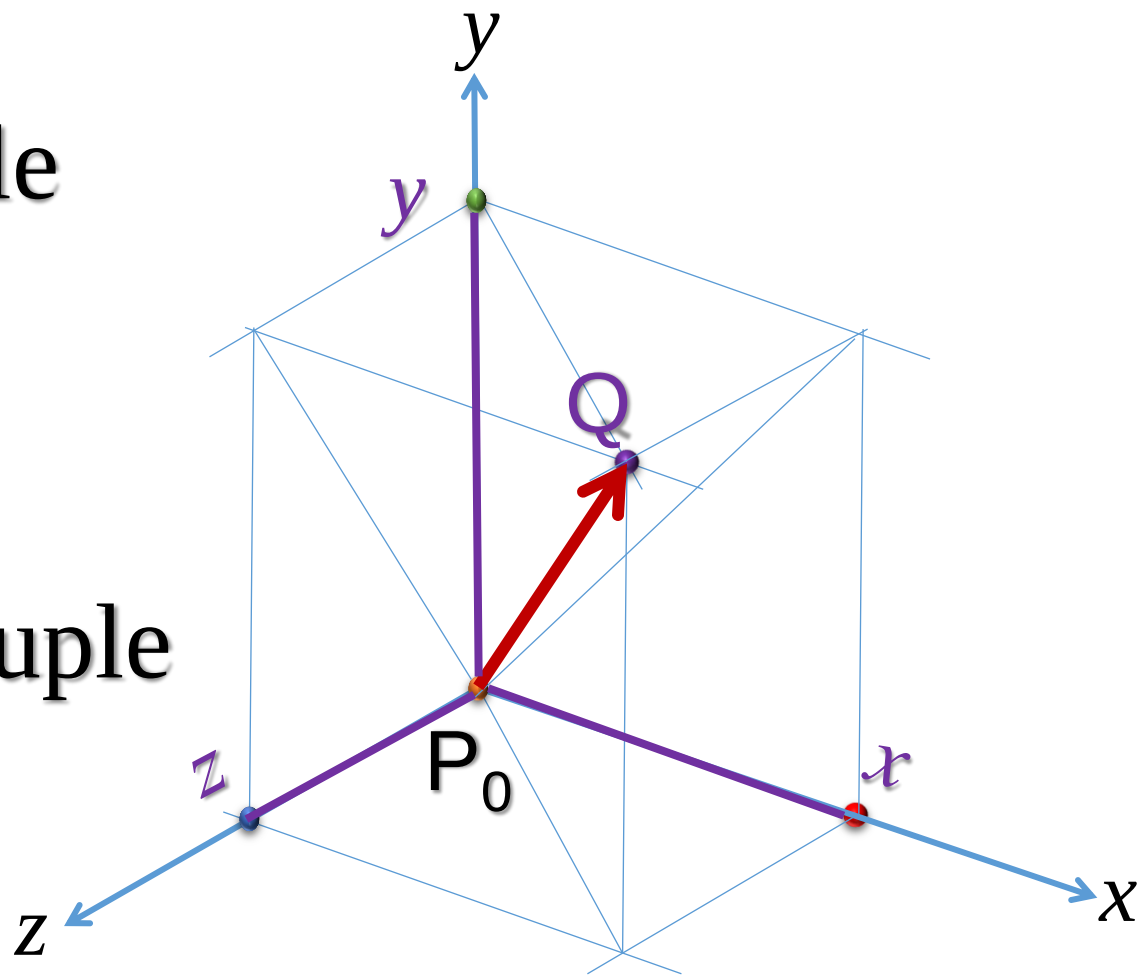
Representations of PQ

- An ordered vertical tuple

$$\mathbf{PQ}_v = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

- An ordered horizontal tuple

$$\mathbf{PQ}_h = [x \quad y \quad z]$$



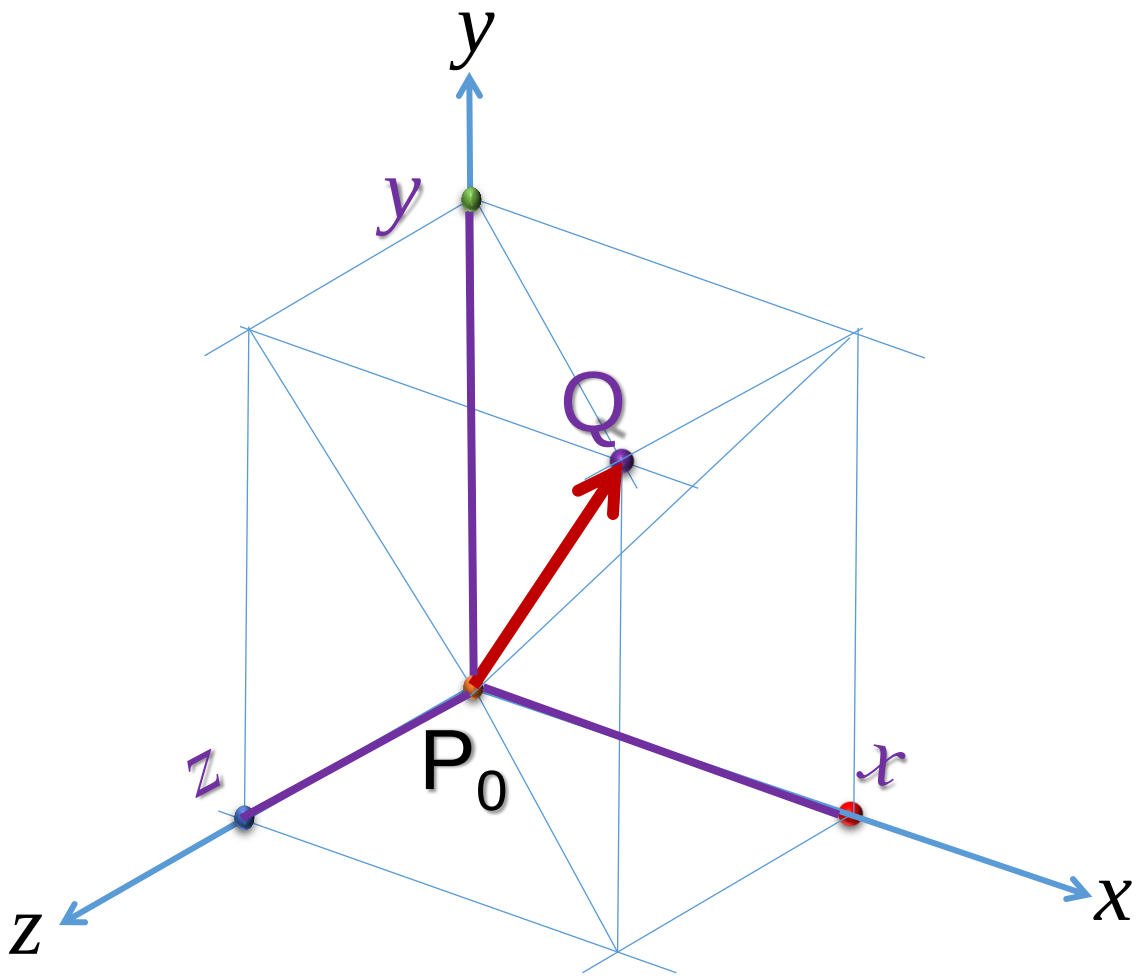
When is PQv ≡ PQh?

- If

$$\mathbf{PQ}_v = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

$$\mathbf{PQ}_h = [x \quad y \quad z]$$

Then, $\mathbf{PQ}_v \equiv \mathbf{PQ}_h^T$

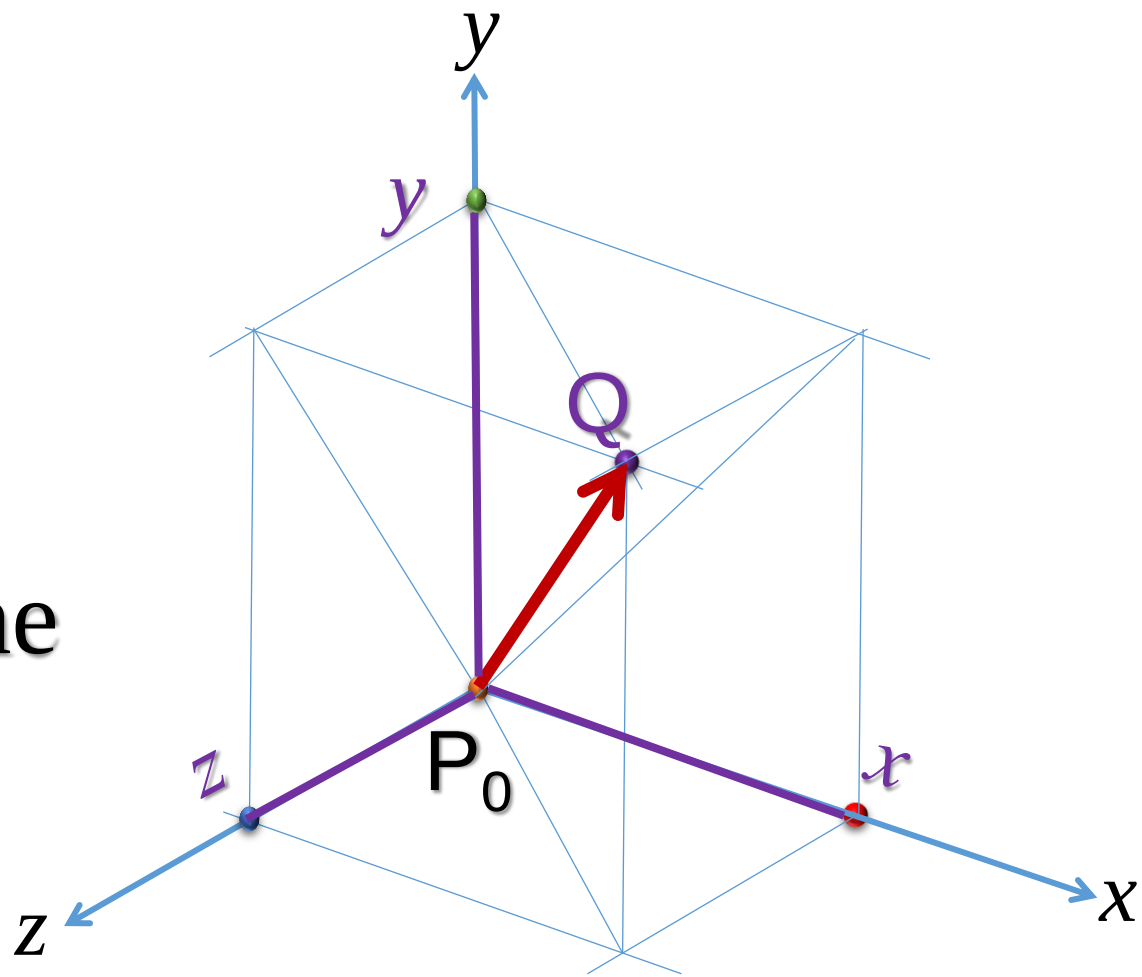


n-Tuples and Orientations

- Note: $\mathbf{PQ}_v \equiv \mathbf{PQ}_h^T$

$$[x \quad y \quad z]^T = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

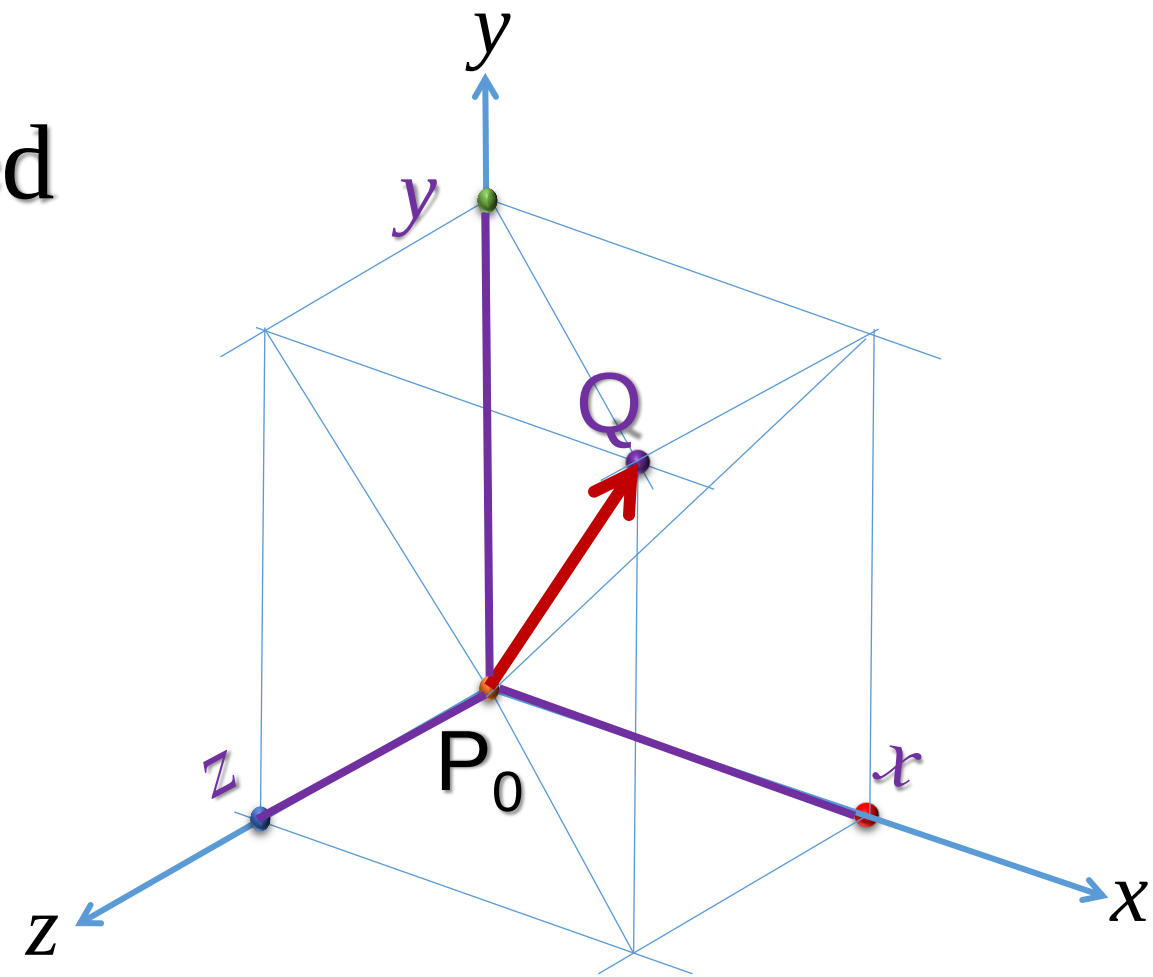
- We have not changed the order or representation!
- We only changed the tuple orientation!



Operational Procedures

- Tuple orientation is used in transformation procedures!

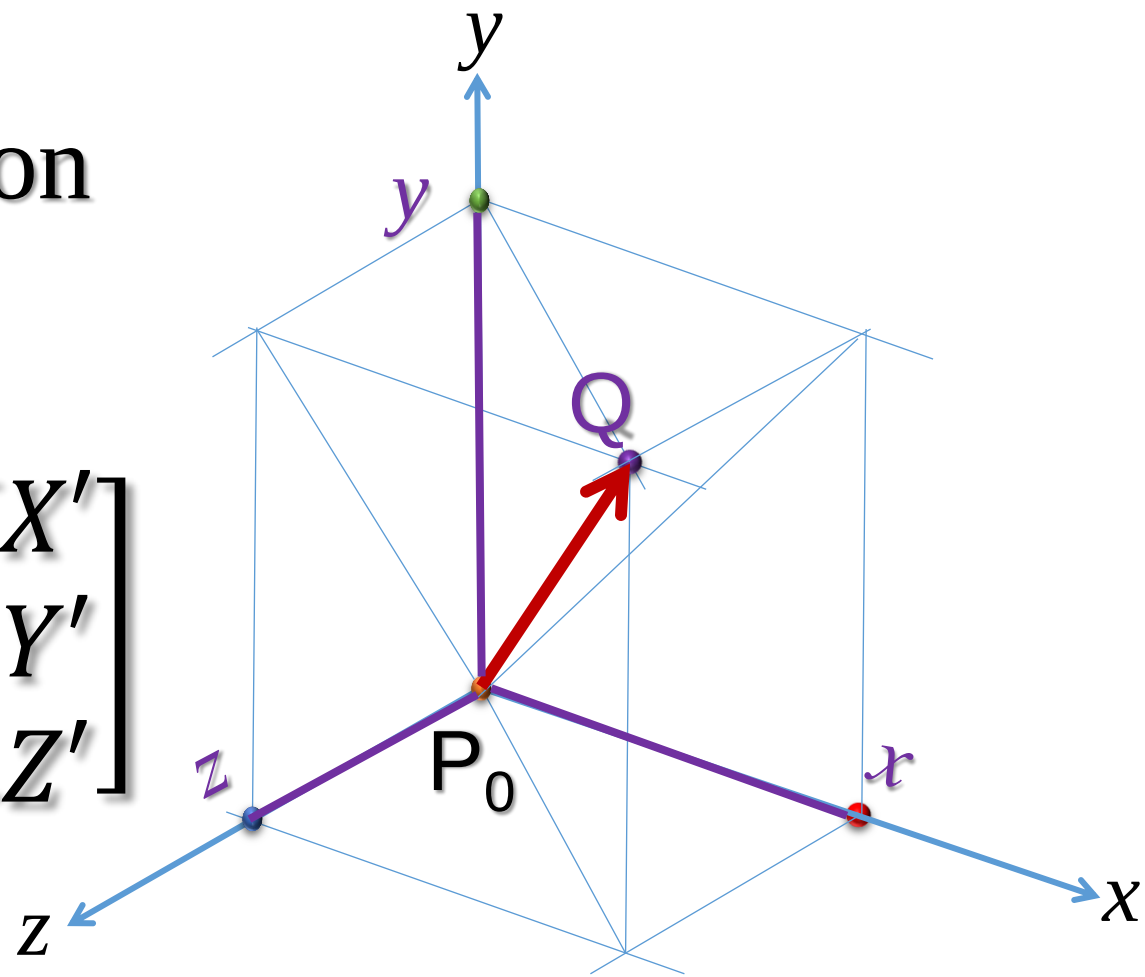
$$[x \quad y \quad z]^T = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$



Coordinate Transforms

- Transform coordinates on the **left**

$$\begin{bmatrix} u_0 & v_0 & w_0 \\ u_1 & v_1 & w_1 \\ u_2 & v_2 & w_2 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} X' \\ Y' \\ Z' \end{bmatrix}$$



- What is X', Y', Z' ?

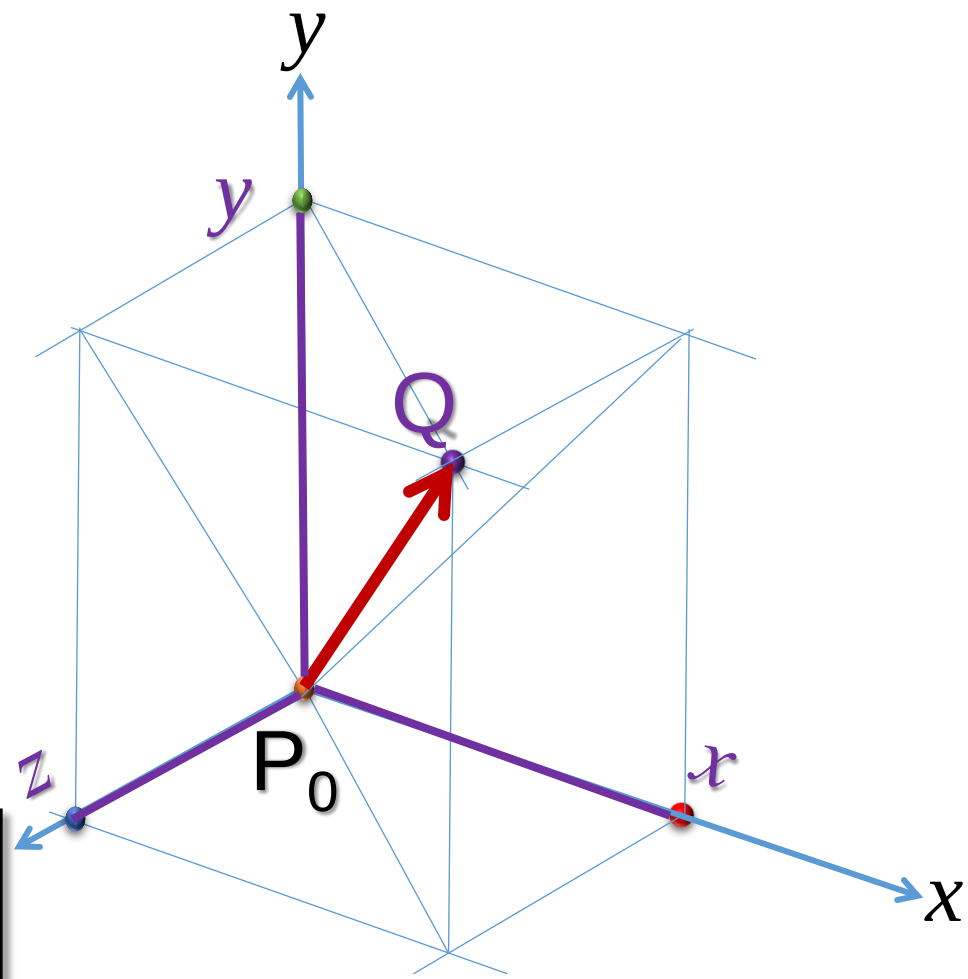
Coordinate Transforms

- Starting with

$$\begin{bmatrix} u_0 & v_0 & w_0 \\ u_1 & v_1 & w_1 \\ u_2 & v_2 & w_2 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} X' \\ Y' \\ Z' \end{bmatrix}$$

- What is X', Y', Z' ?

$$\begin{bmatrix} X' \\ Y' \\ Z' \end{bmatrix} = X \begin{bmatrix} u_0 \\ u_1 \\ u_2 \end{bmatrix} + Y \begin{bmatrix} v_0 \\ v_1 \\ v_2 \end{bmatrix} + Z \begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix}$$

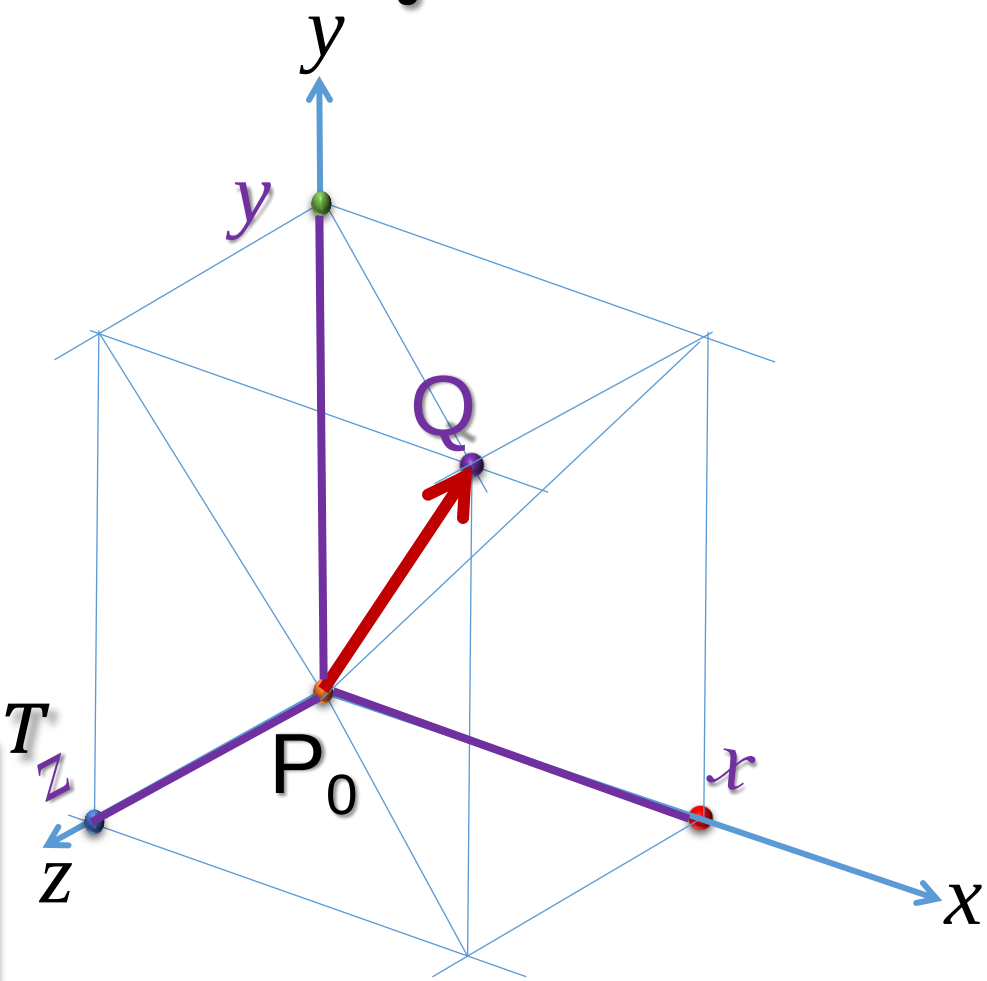


Equivalently, Transpose

- Starting with

$$\begin{bmatrix} u_0 & v_0 & w_0 \\ u_1 & v_1 & w_1 \\ u_2 & v_2 & w_2 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} X' \\ Y' \\ Z' \end{bmatrix}$$

$$\left(\begin{bmatrix} u_0 & v_0 & w_0 \\ u_1 & v_1 & w_1 \\ u_2 & v_2 & w_2 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} \right)^T = \begin{bmatrix} X' \\ Y' \\ Z' \end{bmatrix}^T$$



Transpose

- From

$$\left(\begin{bmatrix} u_0 & v_0 & w_0 \\ u_1 & v_1 & w_1 \\ u_2 & v_2 & w_2 \end{bmatrix}\begin{bmatrix} X \\ Y \\ Z \end{bmatrix}\right)^T = \begin{bmatrix} X' \\ Y' \\ Z' \end{bmatrix}^T$$

- We get

$$\begin{bmatrix} X & Y & Z \end{bmatrix}\begin{bmatrix} u_0 & u_1 & u_2 \\ v_0 & v_1 & v_2 \\ w_0 & w_1 & w_2 \end{bmatrix} = \begin{bmatrix} X' & Y' & Z' \end{bmatrix}$$

Transpose

- From

$$[X \quad Y \quad Z]\begin{bmatrix} u_0 & u_1 & u_2 \\ v_0 & v_1 & v_2 \\ w_0 & w_1 & w_2 \end{bmatrix} = [X' \quad Y' \quad Z']$$

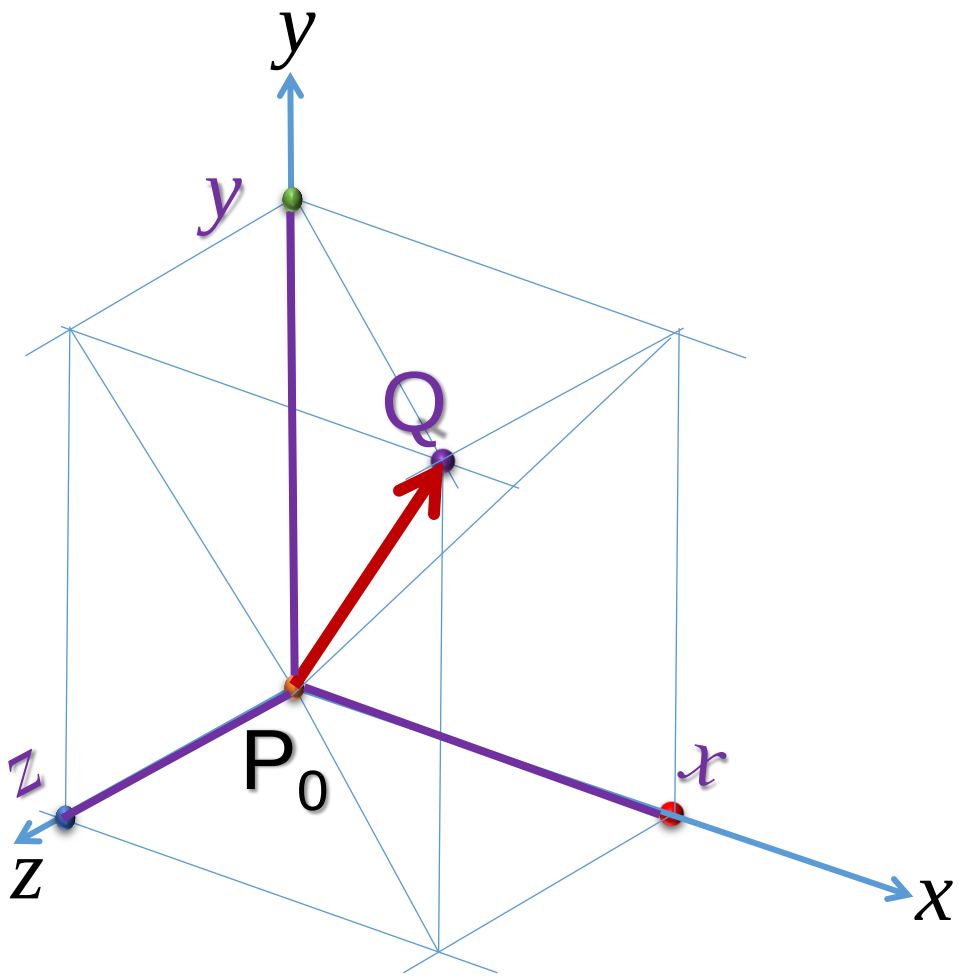
- Which is the same as

$$X\,[u_0 \quad u_1 \quad u_2\,] + Y\,[v_0 \quad v_1 \quad v_2] + Z\,[w_0 \quad w_1 \quad w_2] = [X' \quad Y' \quad Z']$$

Symbolically,

- Starting with

$$\underbrace{\begin{bmatrix} u_0 & v_0 & w_0 \\ u_1 & v_1 & w_1 \\ u_2 & v_2 & w_2 \end{bmatrix}}_{\mathbf{M}} \underbrace{\begin{bmatrix} X \\ Y \\ Z \end{bmatrix}}_{\mathbf{Q}} = \underbrace{\begin{bmatrix} X' \\ Y' \\ Z' \end{bmatrix}}_{\mathbf{Q}'}$$



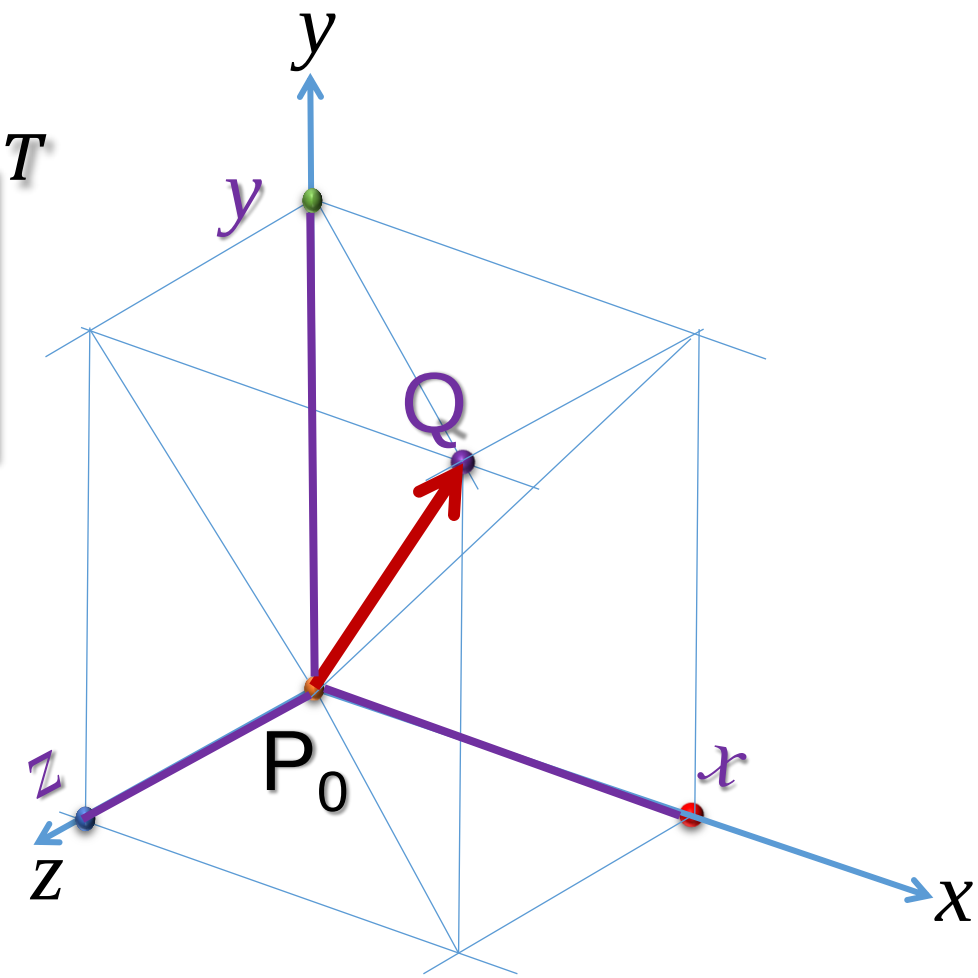
Symbolically,

- Starting with

$$\left(\begin{bmatrix} u_0 & v_0 & w_0 \\ u_1 & v_1 & w_1 \\ u_2 & v_2 & w_2 \end{bmatrix}\begin{bmatrix} X \\ Y \\ Z \end{bmatrix}\right)^T = \begin{bmatrix} X' \\ Y' \\ Z' \end{bmatrix}^T$$

$$(\mathbf{M}\ \mathbf{Q})^T = (\mathbf{Q}')^T$$

$$\mathbf{Q}^T\mathbf{M}^T = (\mathbf{Q}')^T$$



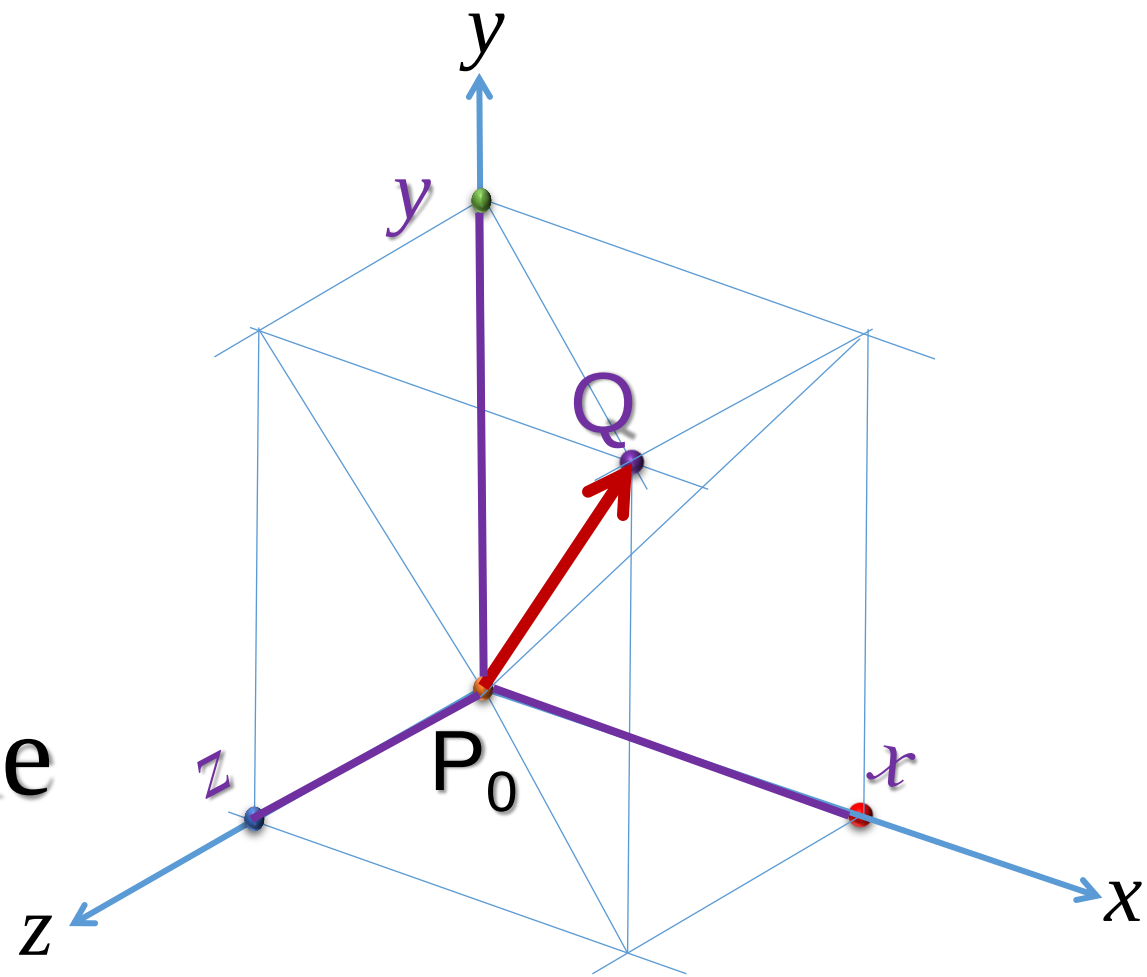
General Convention

- Horizontal tuples

$$[x \quad y \quad z]^T = \mathbf{Q}^T$$

- Matrix operator on the right

$$\mathbf{Q}^{\mathbf{T}}\,\mathbf{M}^{\mathbf{T}} = \mathbf{Q}'^{\mathbf{T}}$$



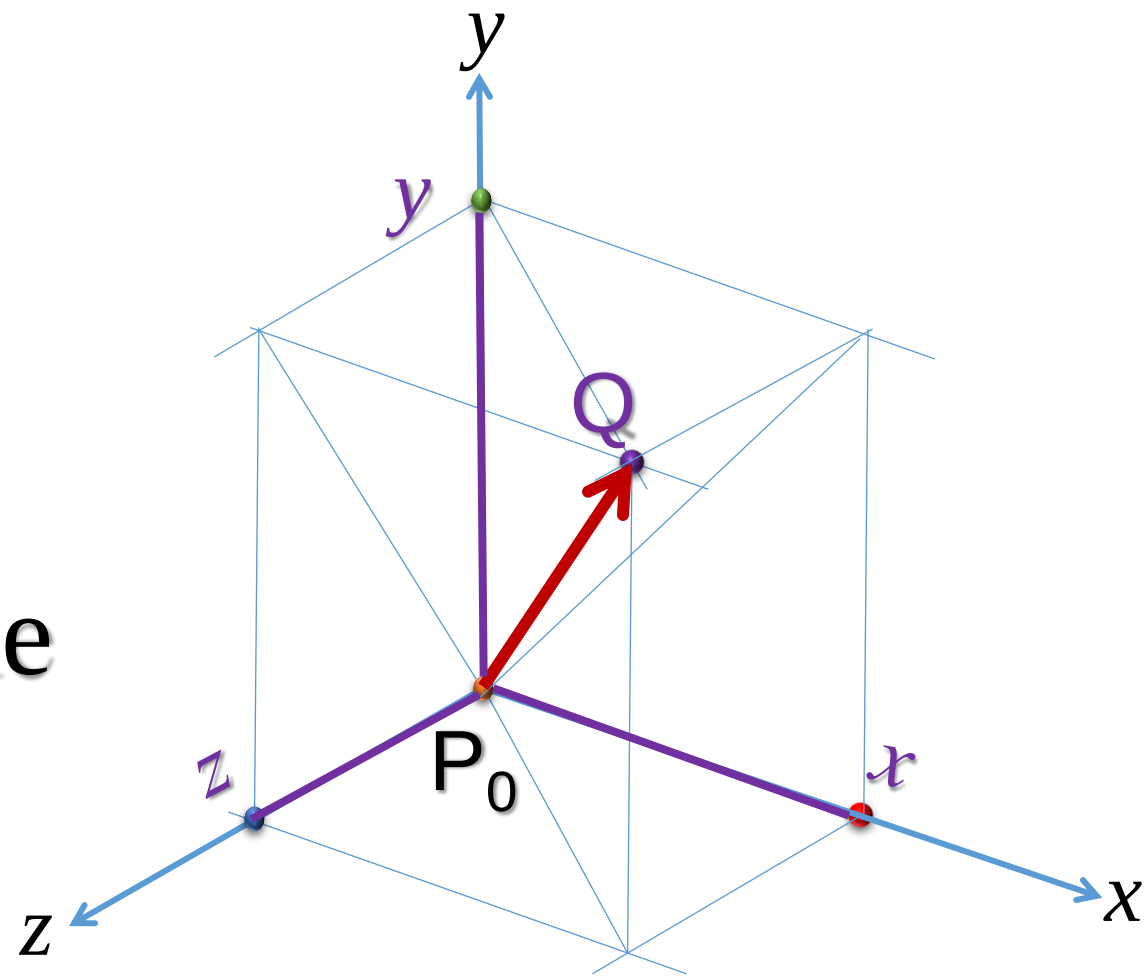
General Convention

- Vertical tuples

$$\mathbf{Q} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

- Matrix operator on the left

$$\mathbf{M}\,\mathbf{Q} = \mathbf{Q}'$$



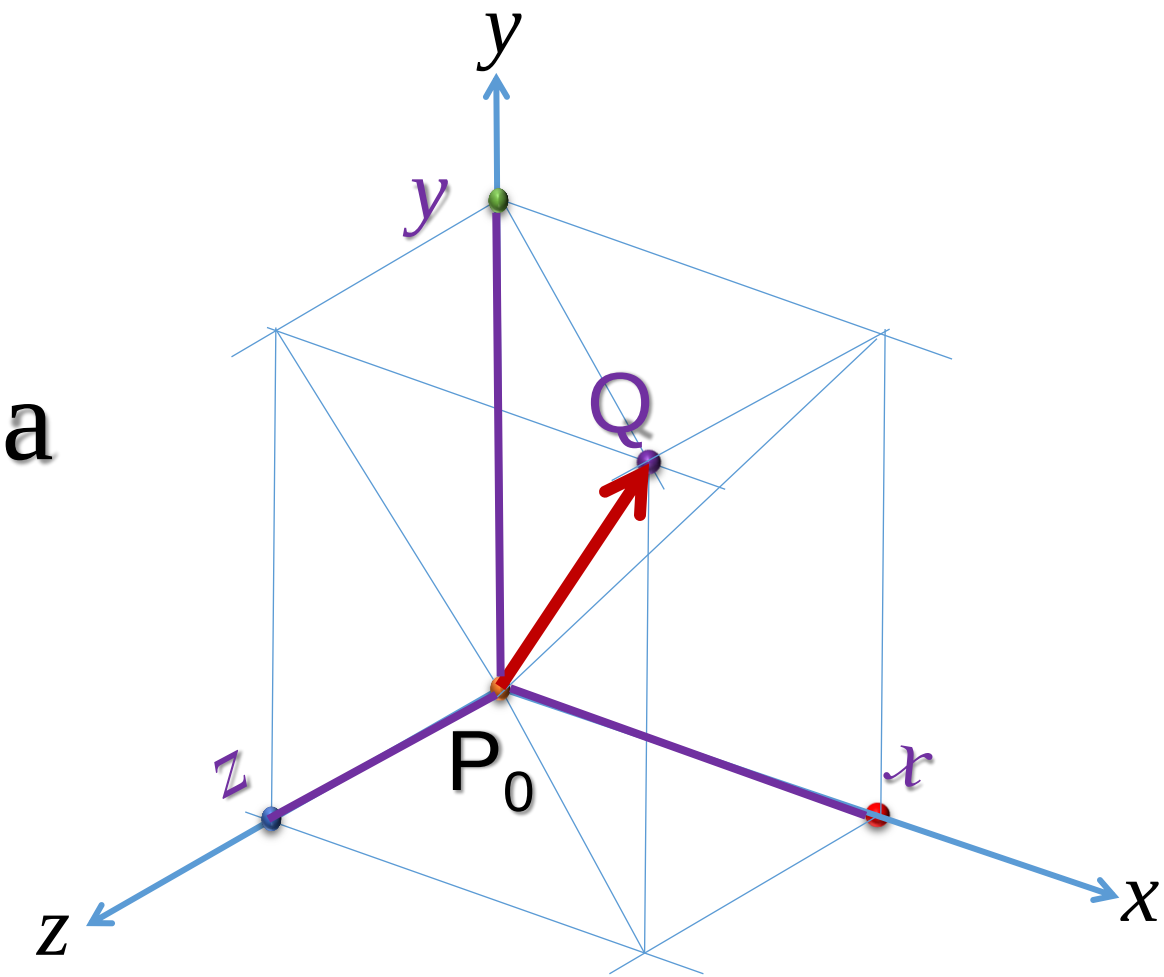
Notice also the Matrix

- For vertical tuples

$$\mathbf{M}\,\mathbf{Q} = \mathbf{Q}'$$

- The matrix **M** forms a basis for Q:

$$\left[\begin{array}{c|c|c} u_0 & v_0 & w_0 \\ u_1 & v_1 & w_1 \\ u_2 & v_2 & w_2 \end{array}\right]$$



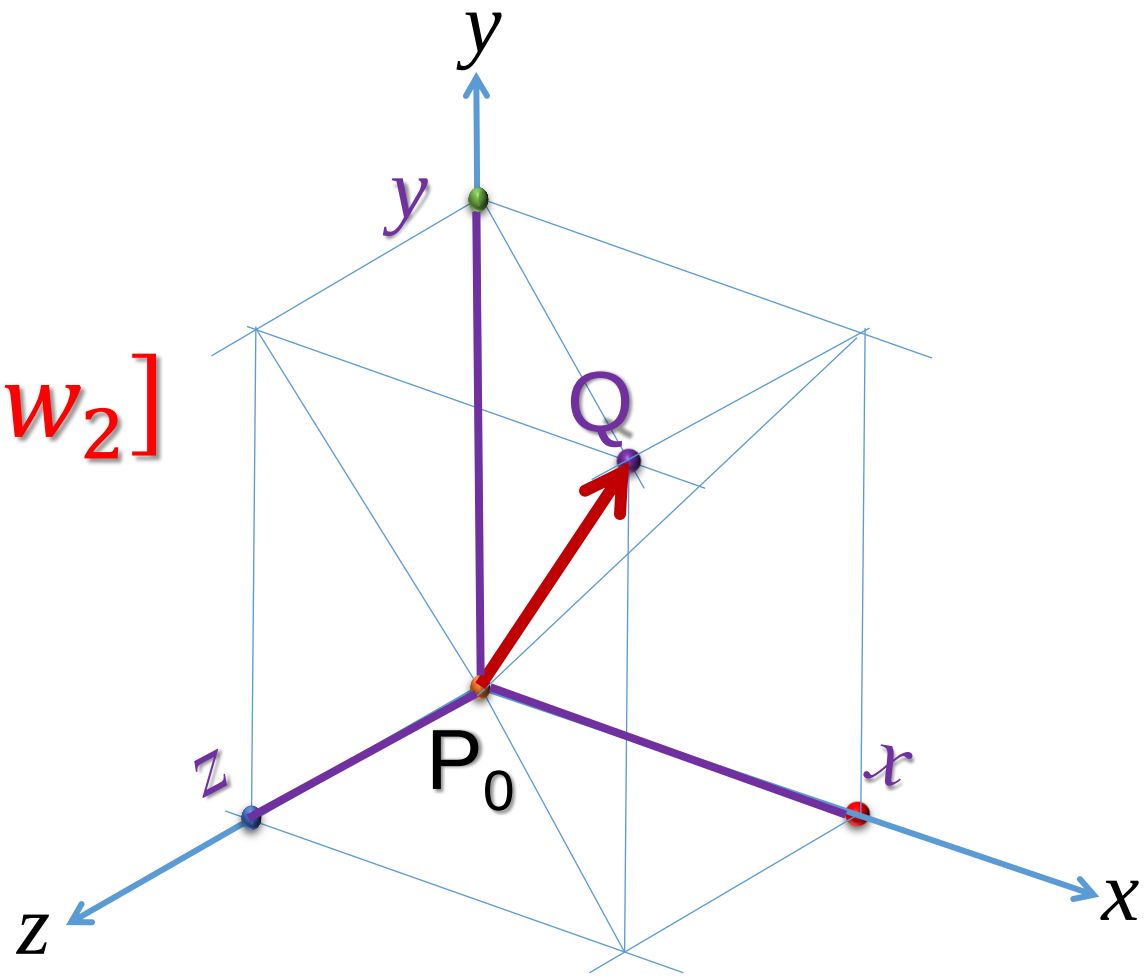
Hence, the matrix format

- Column Major Arrays!

$$\mathbf{M} = [u_0 \quad u_1 \quad u_2 \ldots w_2]$$

- The matrix M

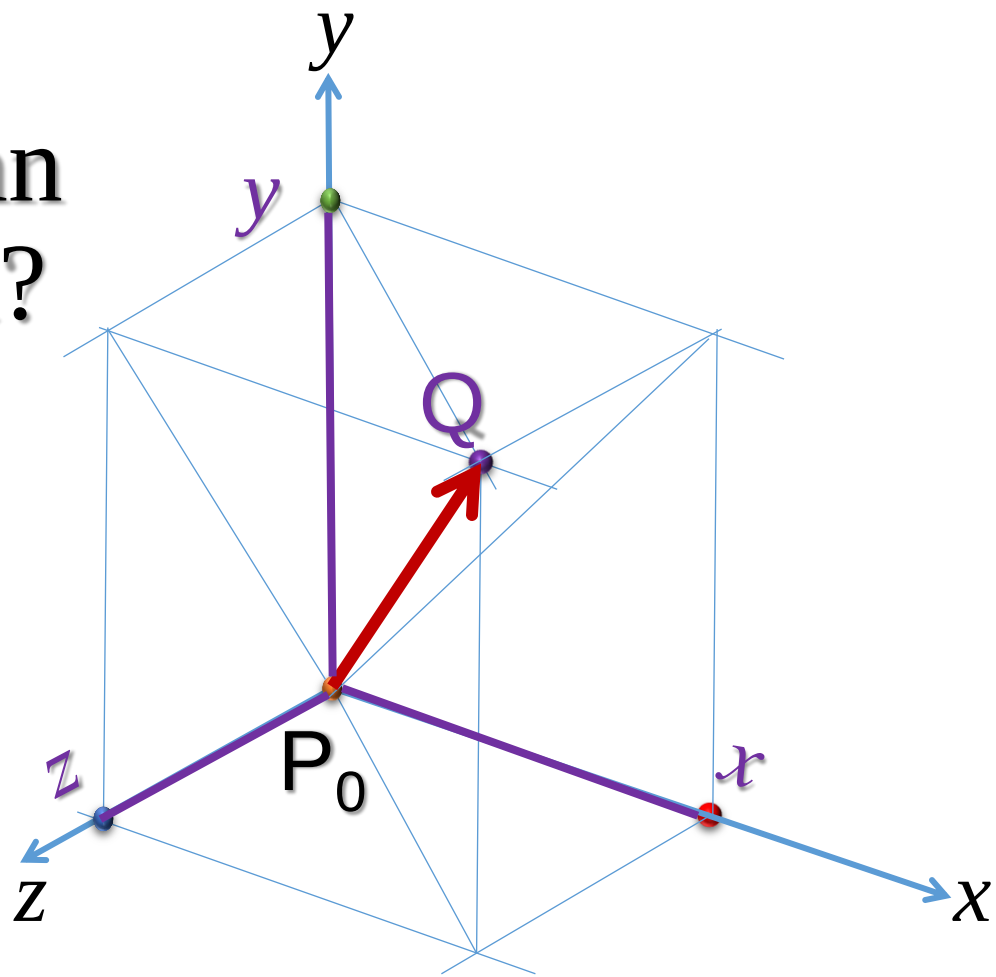
$$\left[\begin{array}{c|c|c} u_0 & v_0 & w_0 \\ u_1 & v_1 & w_1 \\ u_2 & v_2 & w_2 \end{array}\right]$$



Column Major

- Question: What is a column major array representation?
- Answer: store the column elements first

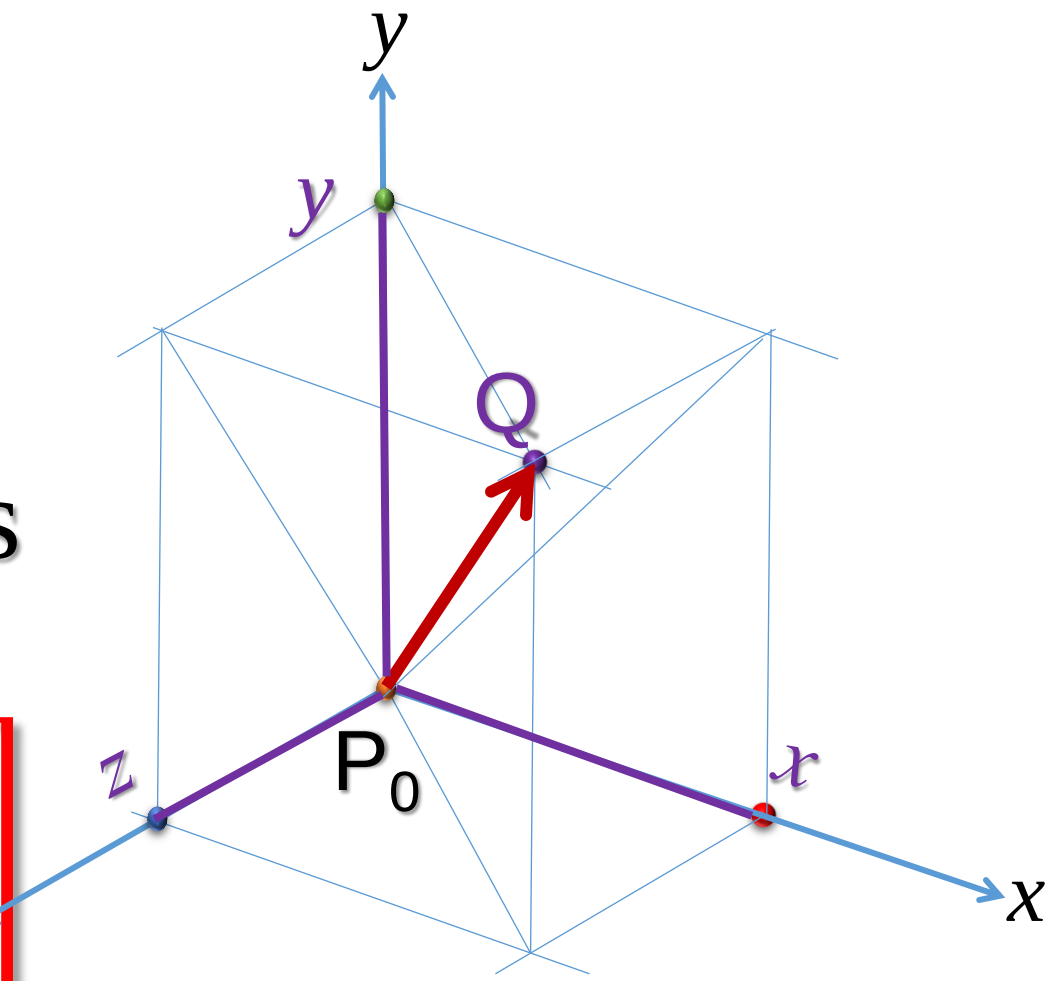
$$\mathbf{A} = \begin{bmatrix} a_{00} & \cdots & a_{0m} \\ \vdots & \ddots & \vdots \\ a_{n0} & \cdots & a_{nm} \end{bmatrix}$$



Column Major

- Question: What is a column major array representation?
- Answer: if a matrix A is

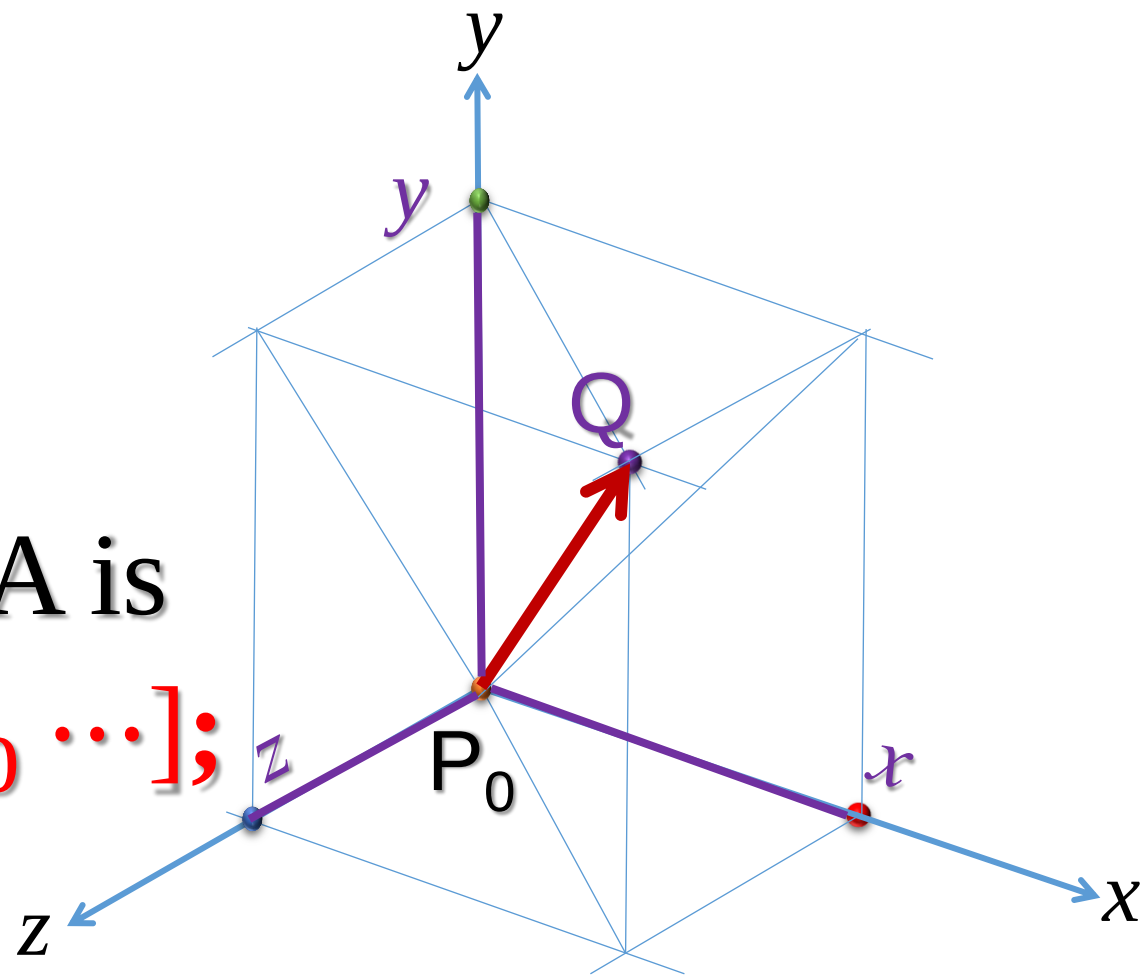
$$\mathbf{A} = \begin{bmatrix} a_{00} & \cdots & a_{0m} \\ \vdots & \ddots & \vdots \\ a_{n0} & \cdots & a_{nm} \end{bmatrix}$$



Column Major

- Question: What is a column major array representation?
- Answer: the array of A is

$$\mathbf{var\ A} = [a_{00}, a_{10}, a_{20} \cdots];$$



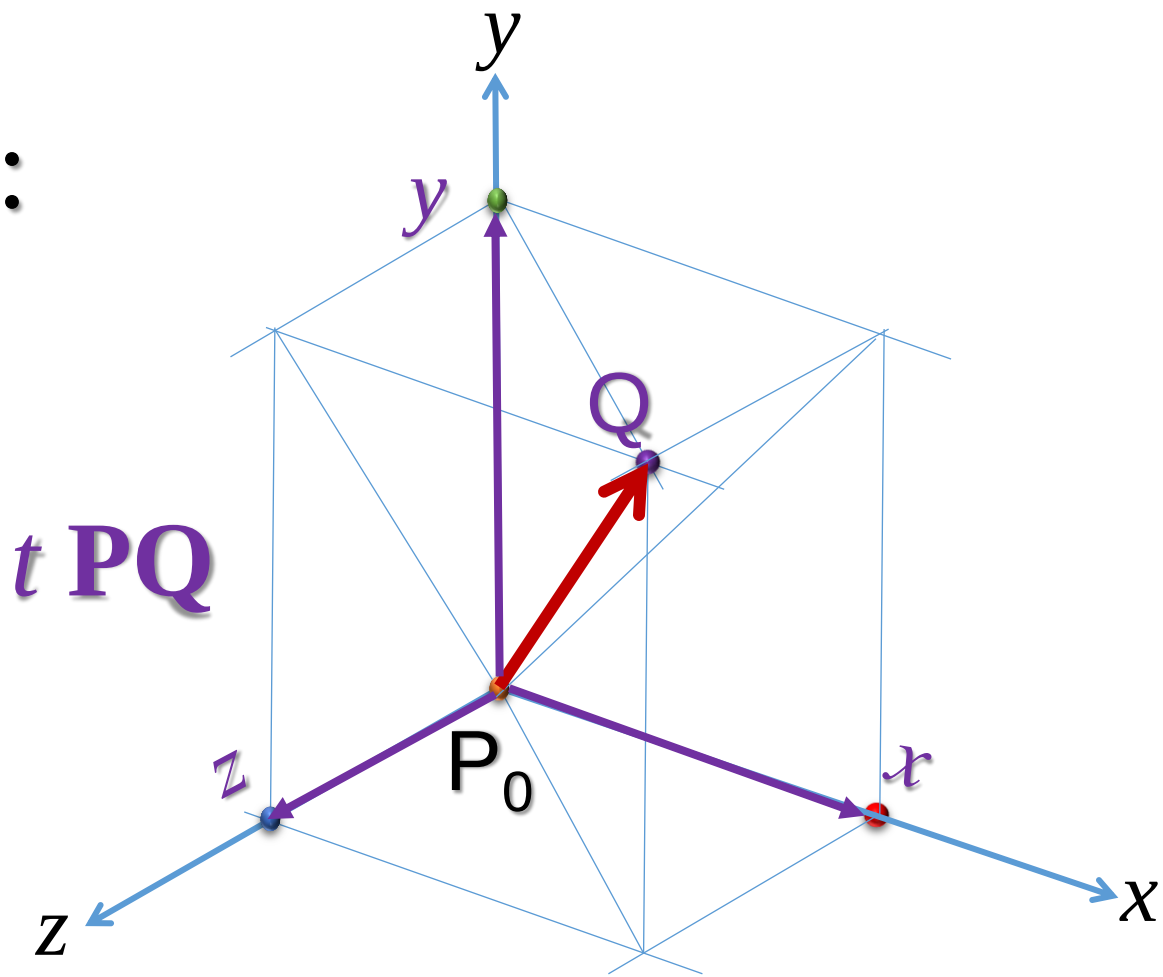
Summary

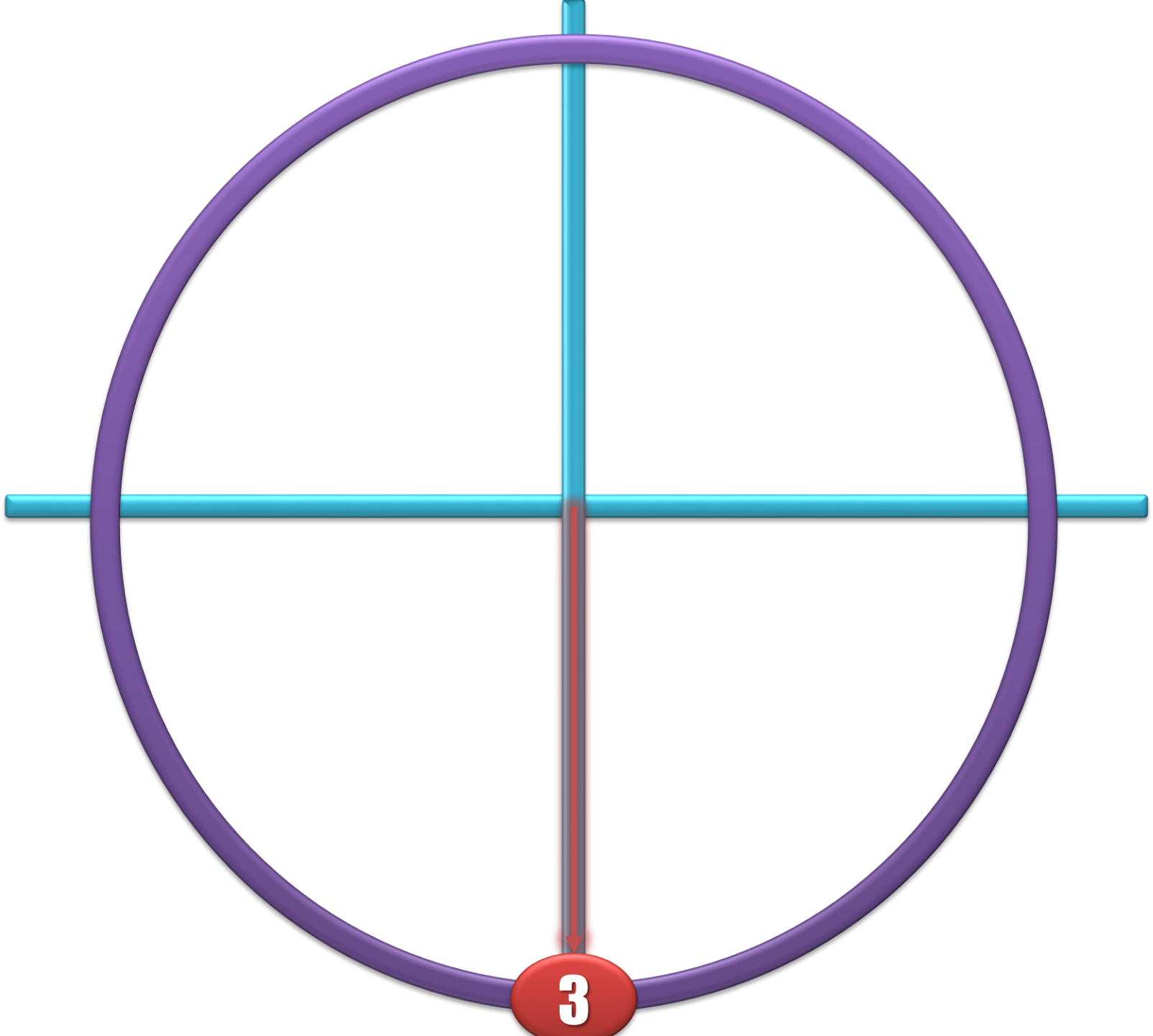
- Most important tools:
  - **Points:** P, Q, …
  - **Vectors:** u, v, …
  - **Rays:** $\mathbf{P}(t) = \mathbf{P}_0 + t\,\mathbf{PQ}$
  - **Normal Vectors:**

$$\mathbf{u} \times \mathbf{v} = \mathbf{n}$$

$$\mathbf{u} \cdot \mathbf{n} = 0$$

$$\mathbf{v} \cdot \mathbf{n} = 0$$



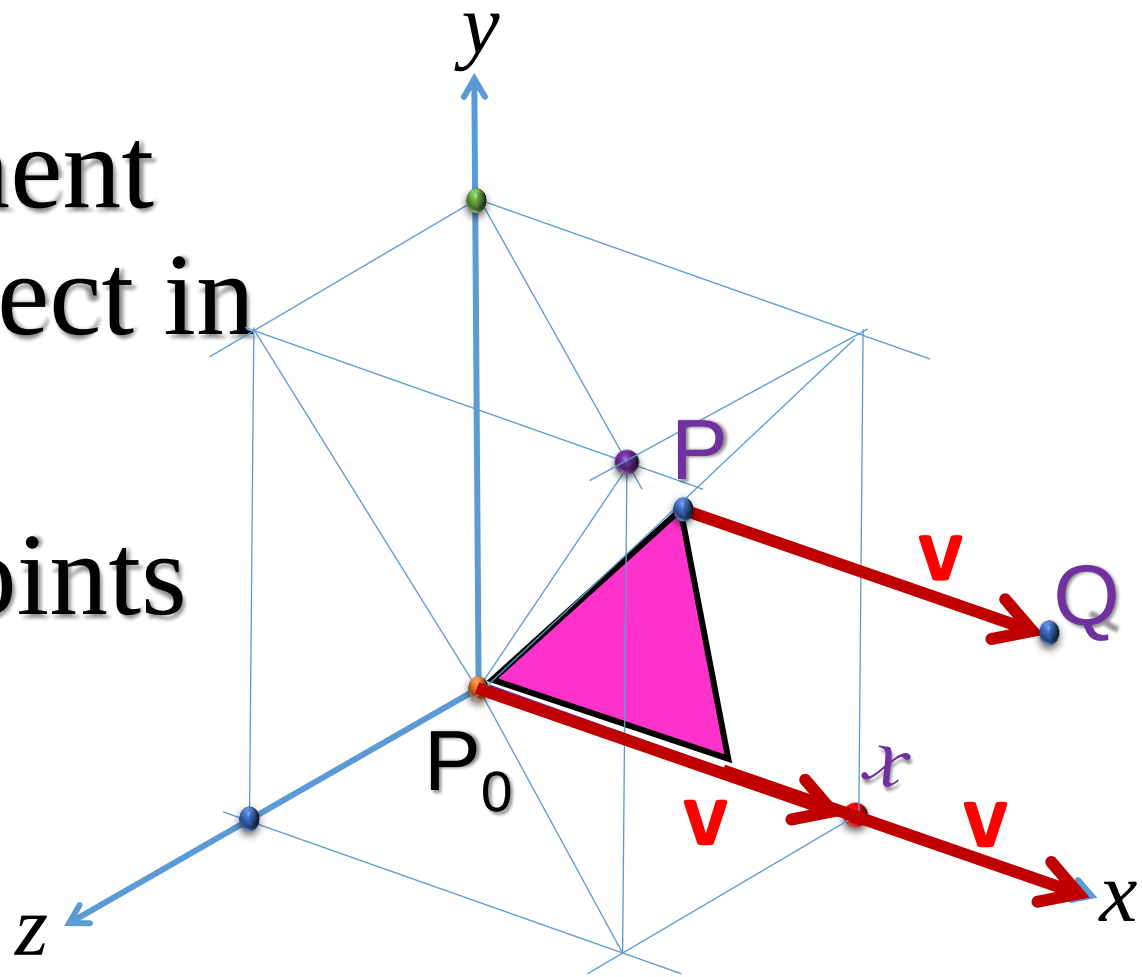

3

Basic Movement Transf.

Translation

- Simultaneous movement of all points of an object in the same direction
- This means, for all points on the object, P

$$\mathbf{P} + \mathbf{v} = \mathbf{Q}$$



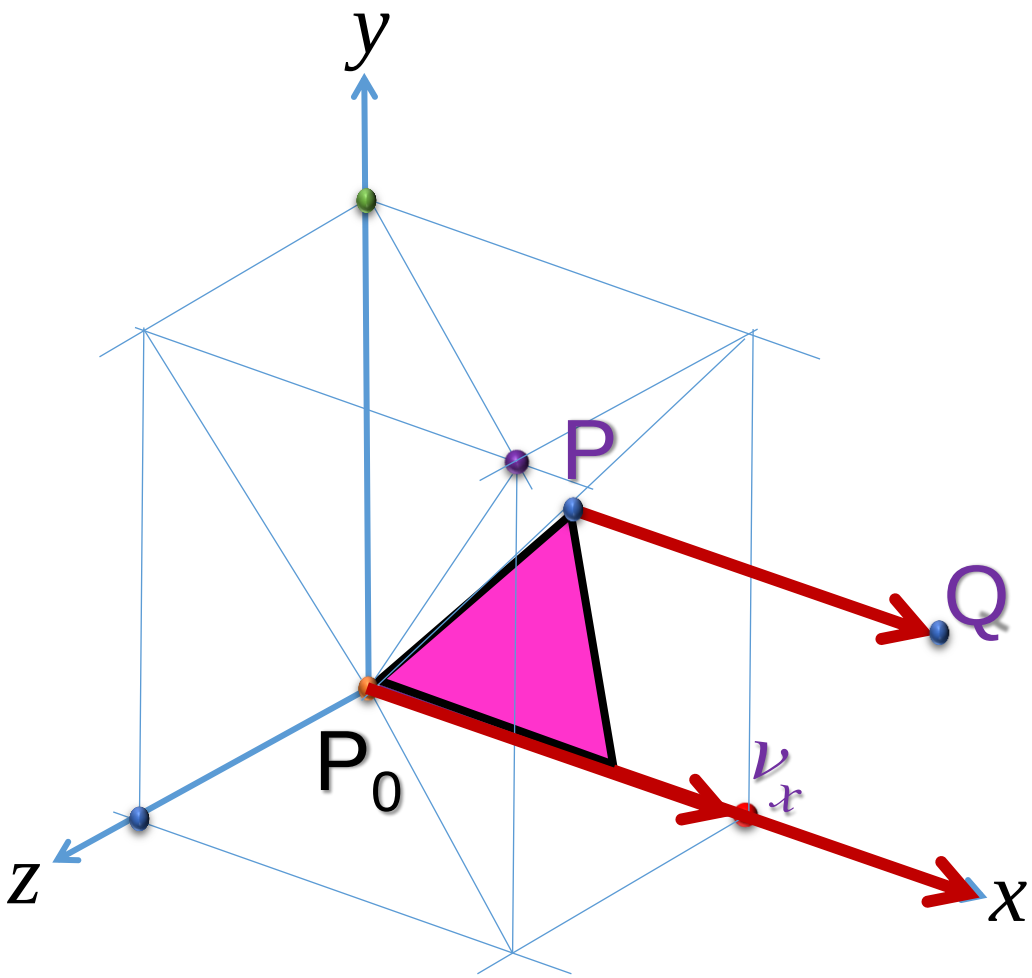
Translation

- Translation

$$\mathbf{P} + \mathbf{v} = \mathbf{Q}$$

- We represent this by

$$\begin{bmatrix} P_x \\ P_y \\ P_z \end{bmatrix} + \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix} = \begin{bmatrix} Q_x \\ Q_y \\ Q_z \end{bmatrix}$$

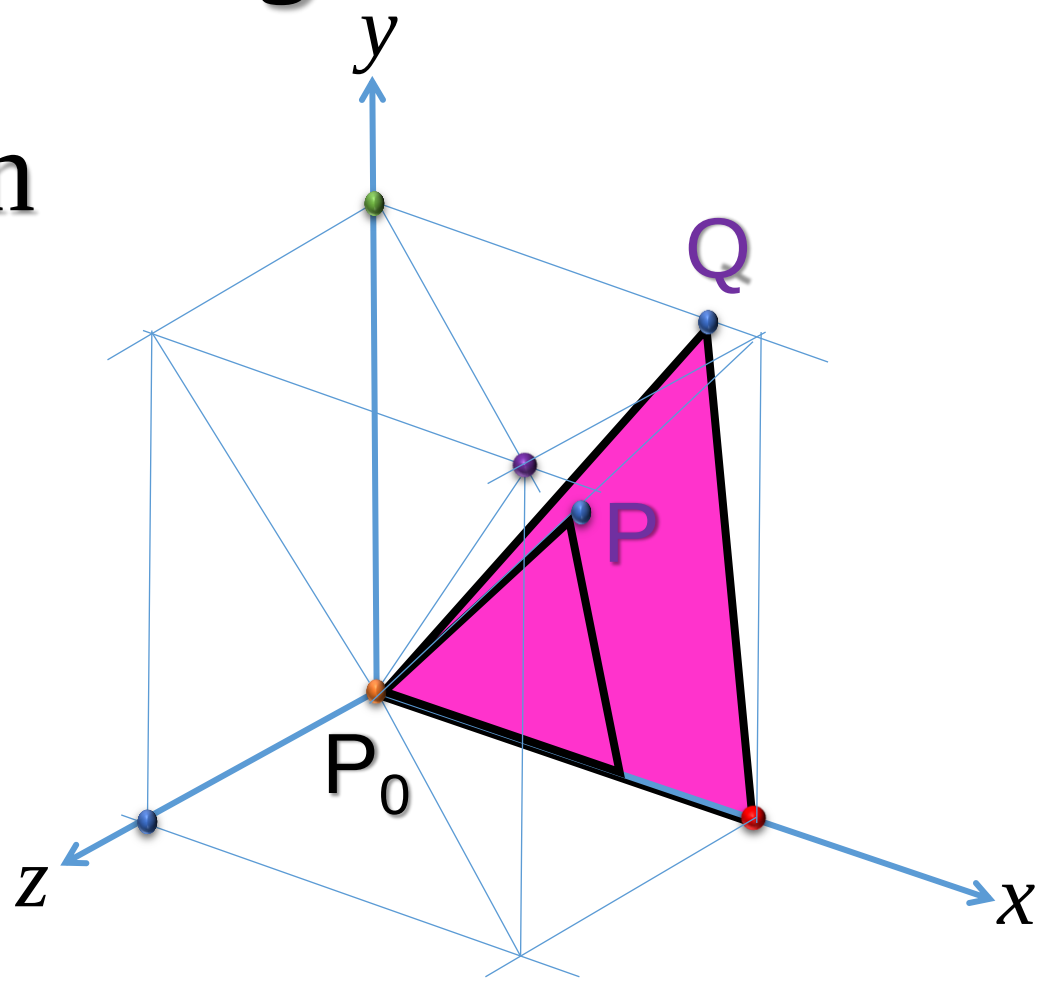


Scaling Geometry

- Multiply every direction by a scalar

$$\begin{bmatrix} P_x \cdot s_x \\ P_y \cdot s_y \\ P_z \cdot s_z \end{bmatrix} = \begin{bmatrix} Q_x \\ Q_y \\ Q_z \end{bmatrix}$$

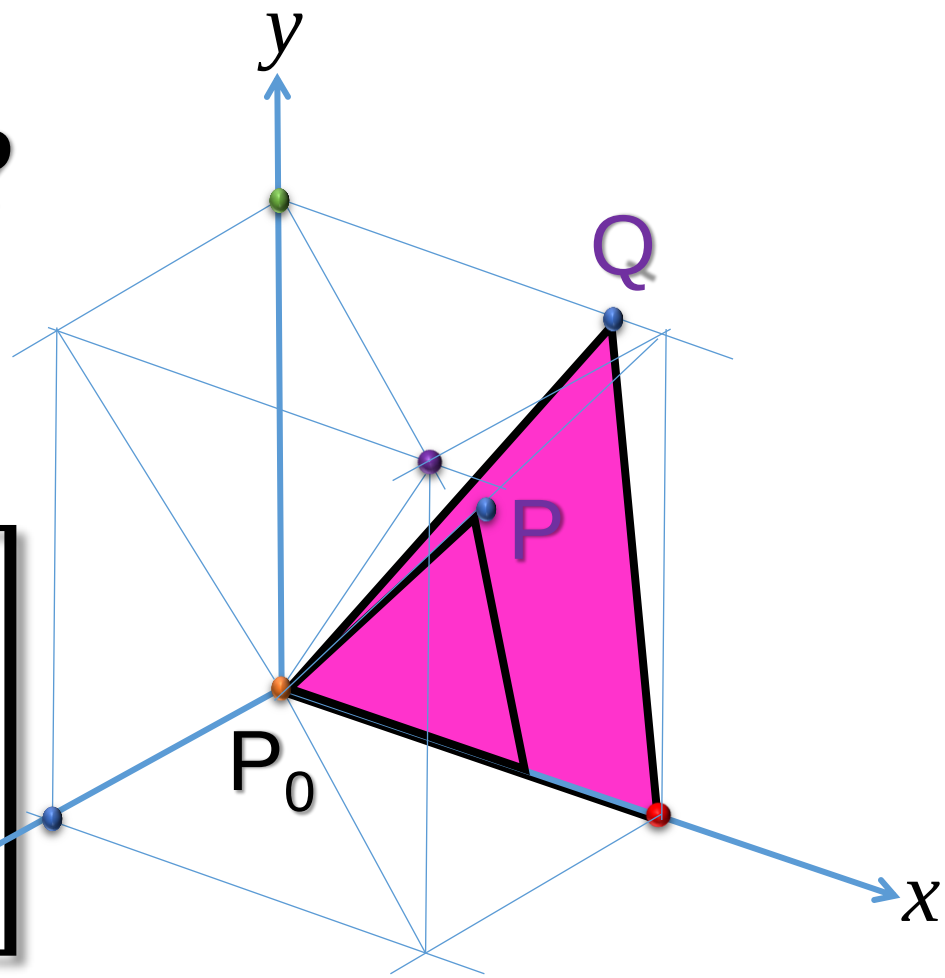
- Note, S is not a vector



Scaling Operation

- How to represent scaling?
- How about a matrix op:

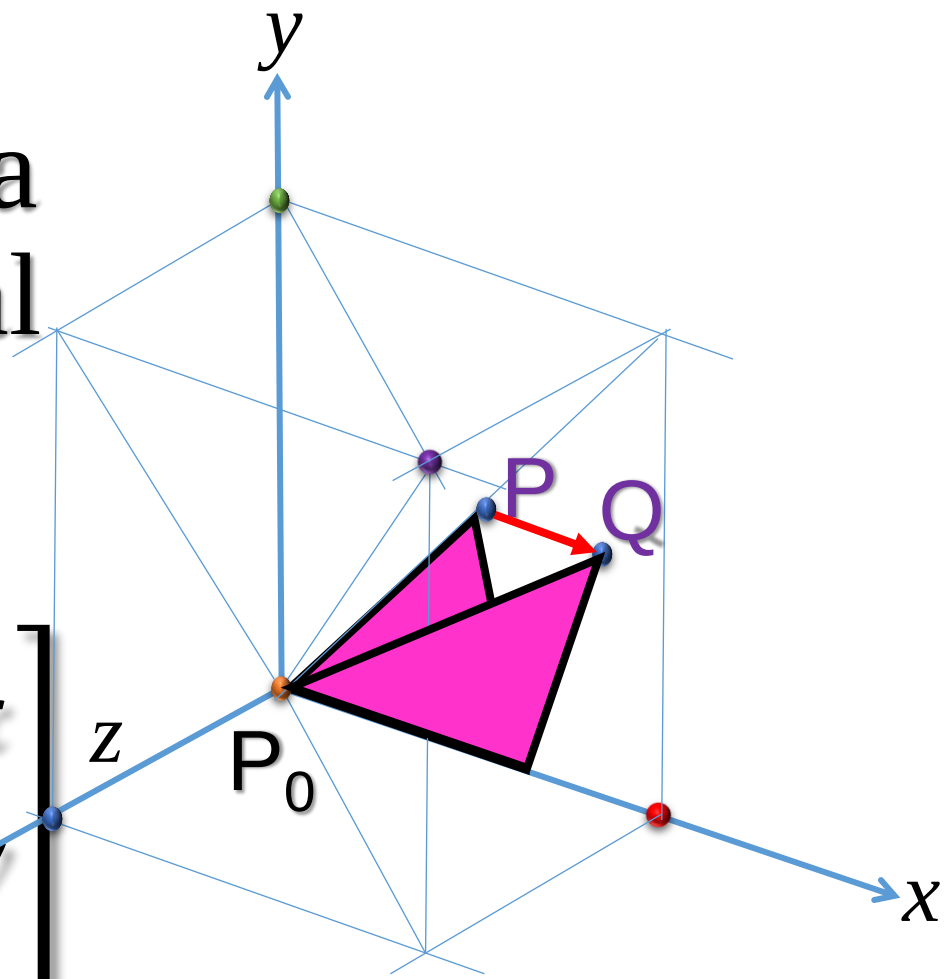
$$\begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & s_z \end{bmatrix} \begin{bmatrix} P_x \\ P_y \\ P_z \end{bmatrix} = \begin{bmatrix} Q_x \\ Q_y \\ Q_z \end{bmatrix}$$



Shearing

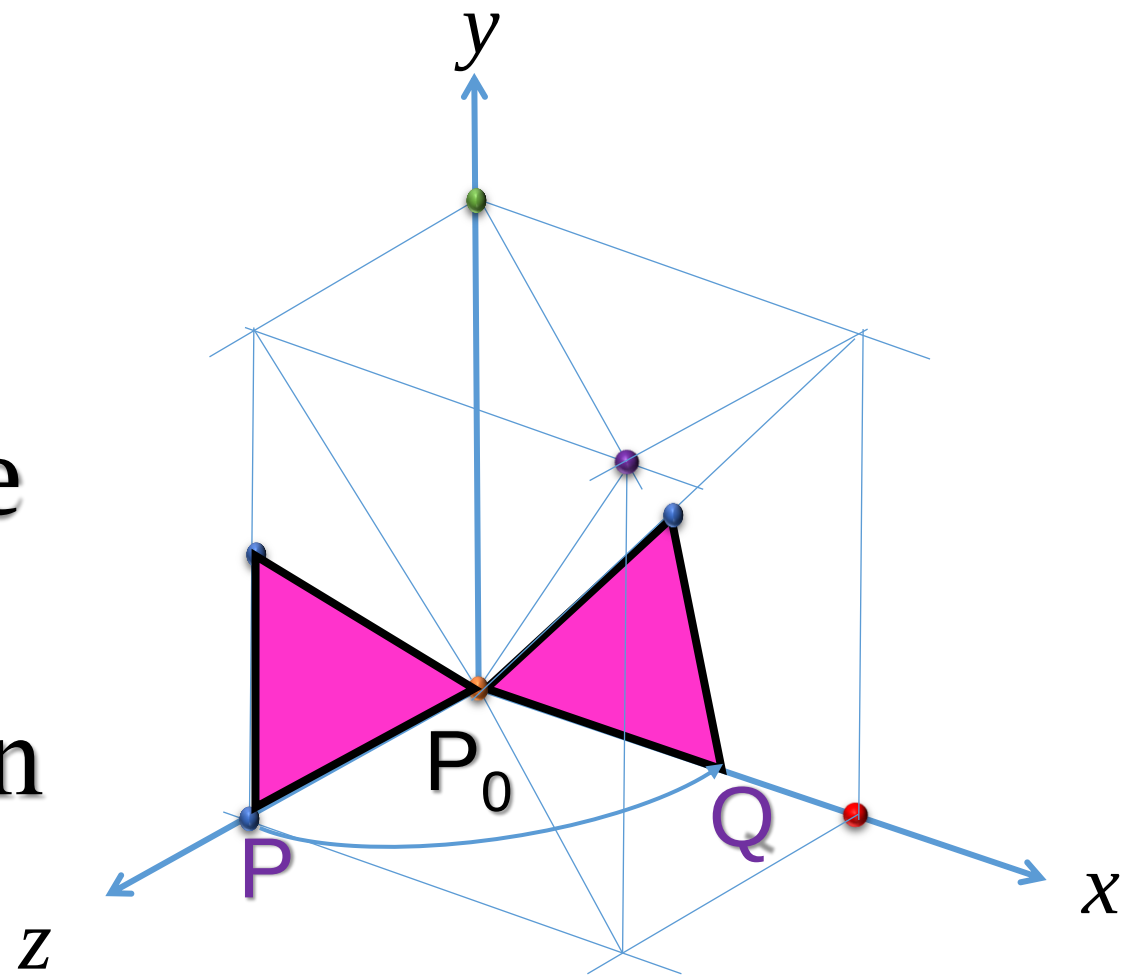
- Shearing is equivalent to a scaling plus a proportional translation:

$$\begin{bmatrix} s_x & h_{xy} & 0 \\ 0 & s_y & 0 \\ 0 & 0 & s_z \end{bmatrix} \begin{bmatrix} P_x \\ P_y \\ P_z \end{bmatrix} = \begin{bmatrix} Q_x \\ Q_y \\ Q_z \end{bmatrix}$$

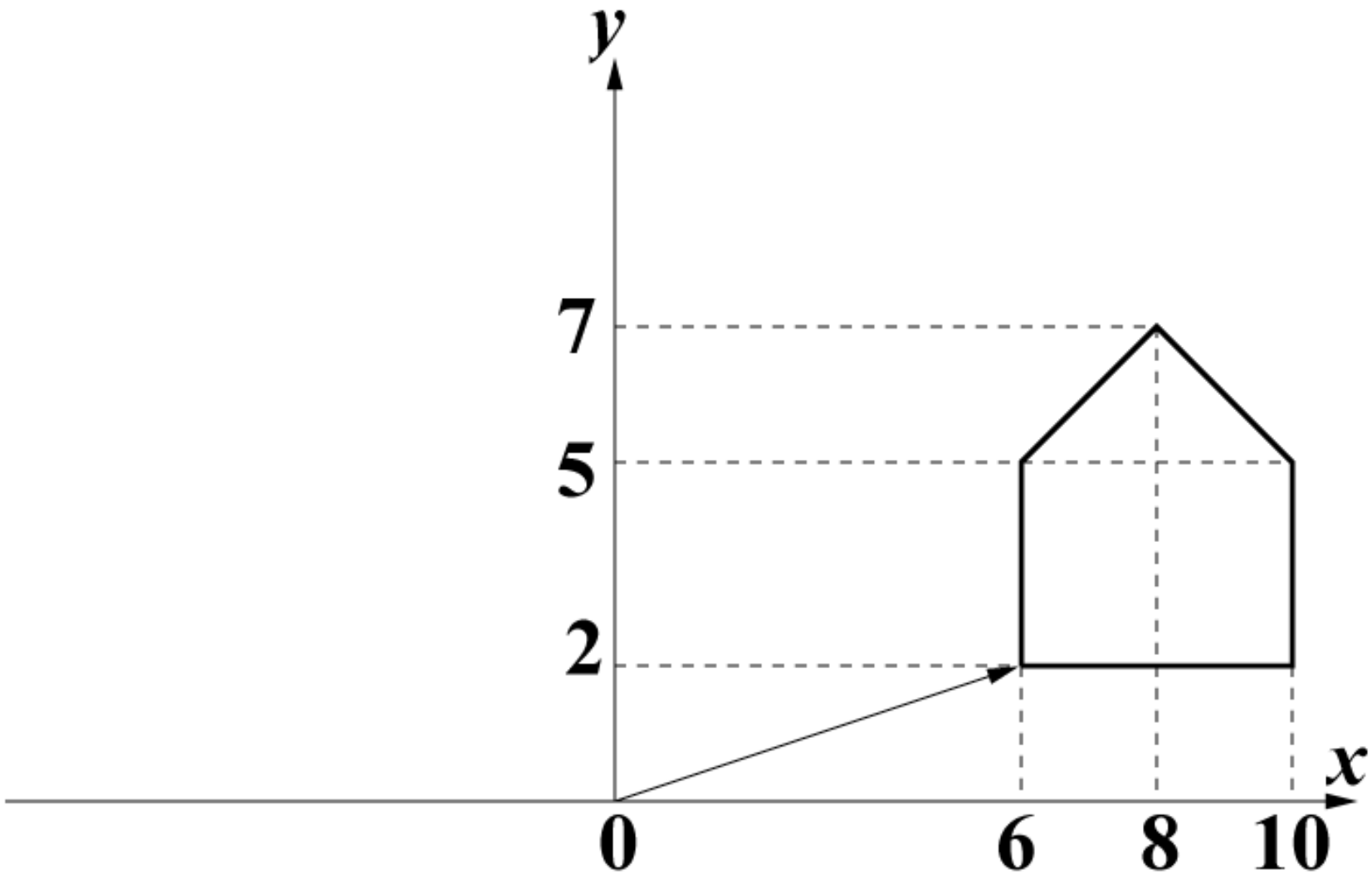


Rotation

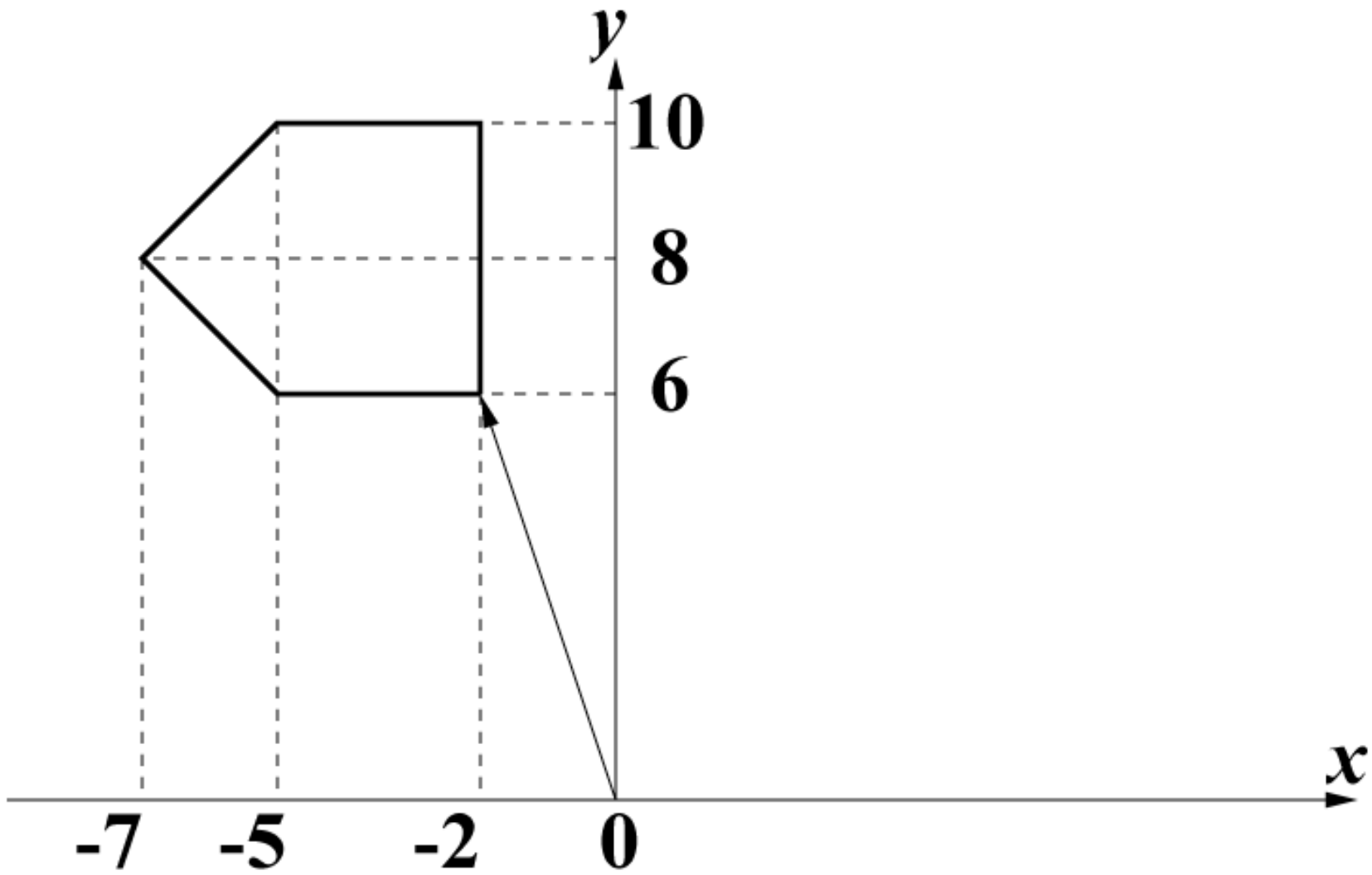
- Rotations are hard!
- Why?
- Because, rotations are inherently 3D!
- So, let's break it down



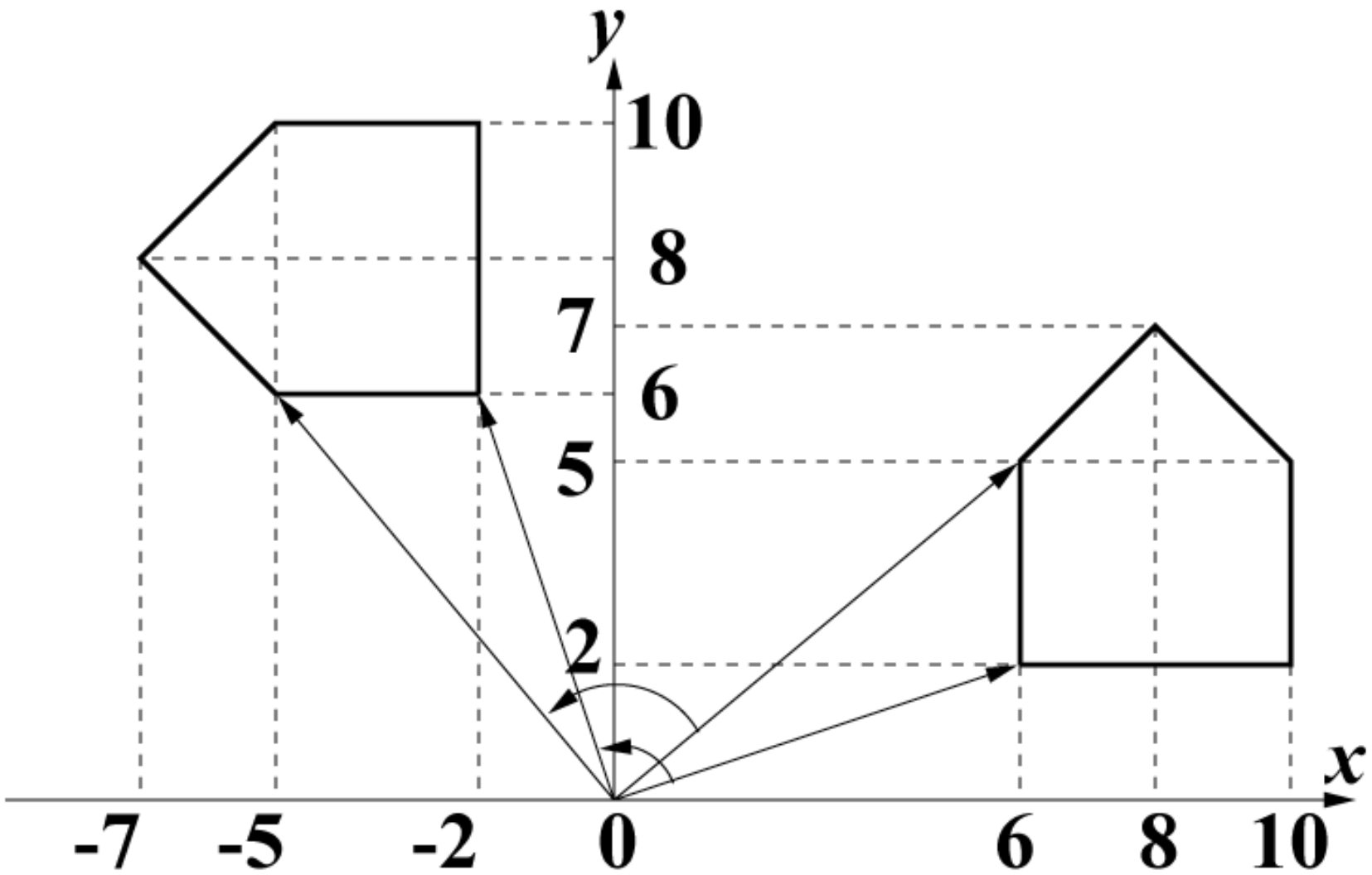
Rotation: Initial



Rotation: Final



Rotation: Operation



Rotation Operation

Coordinates:

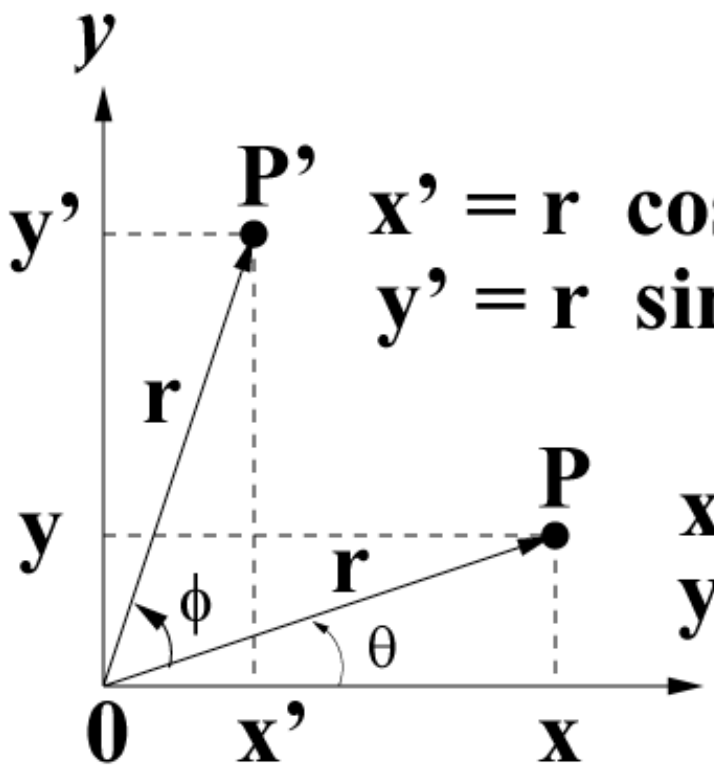
$$x' = \cos(\phi)x - \sin(\phi)y$$

$$y' = \sin(\phi)x + \cos(\phi)y$$

Matrix Form:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos(\phi) & -\sin(\phi) \\ \sin(\phi) & \cos(\phi) \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Where does it come from?



$$x' = r\cos(\theta + \phi);$$
$$y' = r\sin(\theta + \phi);$$

$$x = r\cos(\theta);$$
$$y = r\sin(\theta);$$

$$\cos(\theta + \phi) = \cos(\phi)\cos(\theta) - \sin(\phi)\sin(\theta);$$
$$\sin(\theta + \phi) = \cos(\phi)\sin(\theta) + \sin(\phi)\cos(\theta);$$

Homogeneous Coordinates

Homogeneous Coordinates

REAL ***HOMOGENEOUS***

$$\begin{bmatrix} x & y \end{bmatrix} \rightarrow \begin{bmatrix} x_h & y_h & w_h \end{bmatrix}$$

$$such\ \ that:\ \ x = \frac{x_h}{w_h}\ \ and\ \ y = \frac{y_h}{w_h}$$

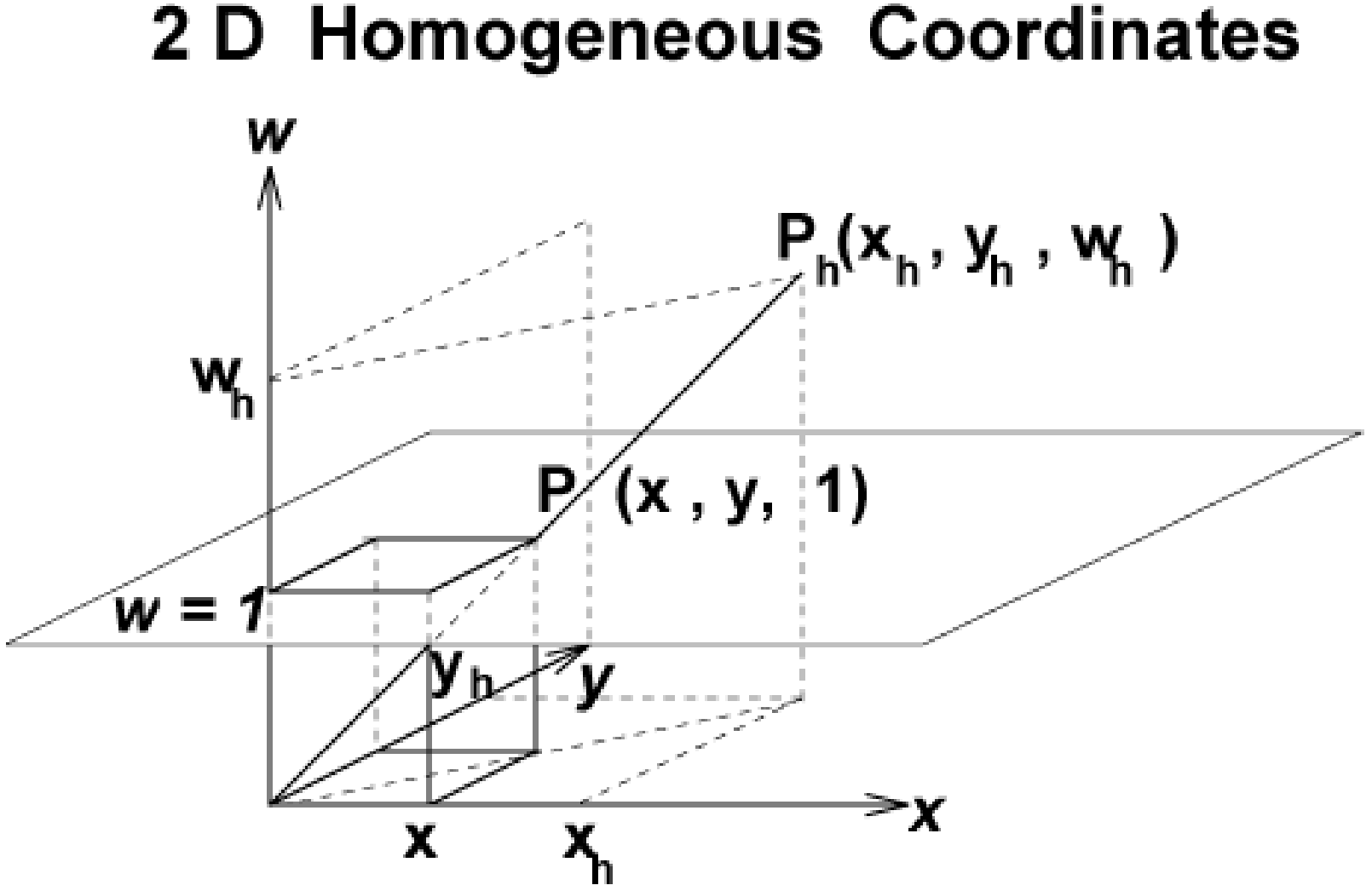
Why?...

Recall: translation ⇨ *addition*

rotation ⇨ *multiplication*

In order to unify the form of all the transformation operations.

H-coord. Interpretation



Homogeneous Rotation

$$R = \begin{bmatrix} \cos(\phi) & -\sin(\phi) & 0 \\ \sin(\phi) & \cos(\phi) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Homogenous Scaling

$$S = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Homogeneous Translation

$$T = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

Example

Example: P' = T P

$$\begin{bmatrix} x'_h \\ y'_h \\ w'_h \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_h \\ y_h \\ w_h \end{bmatrix} = \begin{bmatrix} x_h + t_x w_h \\ y_h + t_y w_h \\ w_h \end{bmatrix}$$

$$for \quad w_h = 1$$

$$\begin{bmatrix} x'_h \\ y'_h \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_h \\ y_h \\ 1 \end{bmatrix} = \begin{bmatrix} x_h + t_x \\ y_h + t_y \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \\ 0 \end{bmatrix}$$

Windows and Viewports

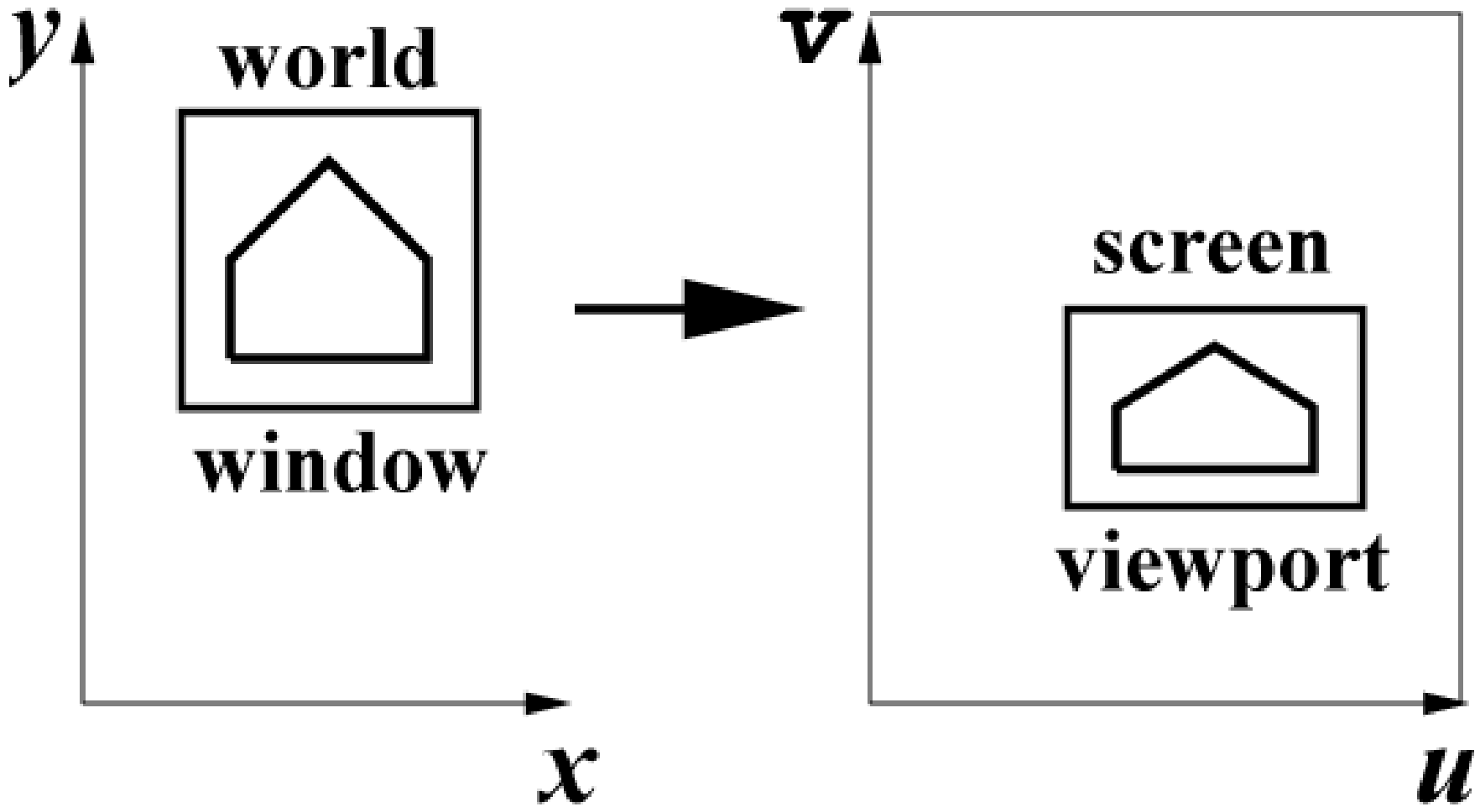
Viewport mapping:

world coordinates must be mapped into screen coordinates

What is this mapping?

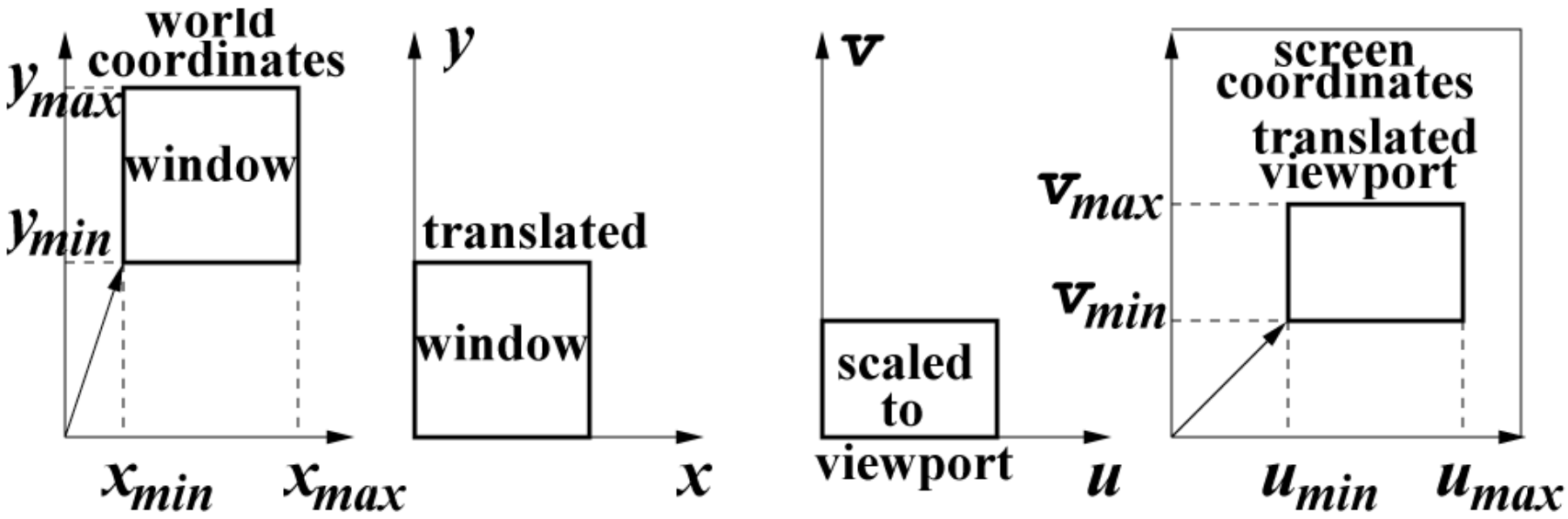
Homogeneous transformation

Viewport Mapping



Viewport Mapping

Basic Operations



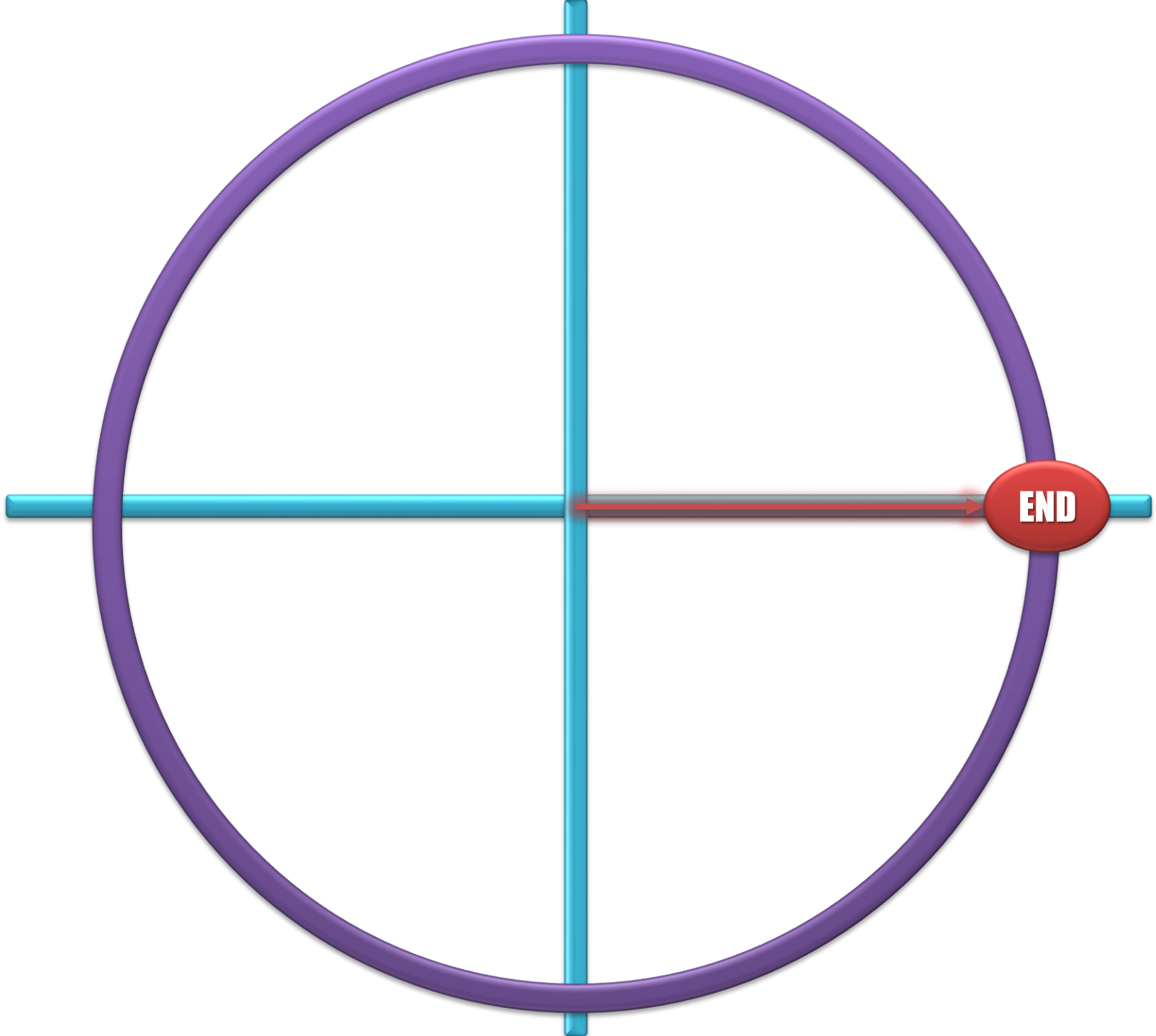
Viewport Mapping

$$M_{wv} = T(u_{\min}, v_{\min})$$

$$\bullet\, S\left(\frac{u_{\max} - u_{\min}}{x_{\max} - x_{\min}}, \frac{v_{\max} - v_{\min}}{y_{\max} - y_{\min}}\right)$$

$$\bullet\, T(-x_{\min}, -y_{\min})$$

$$\Rightarrow P_{viewport} = M_{wv} \bullet P_{window}$$


END

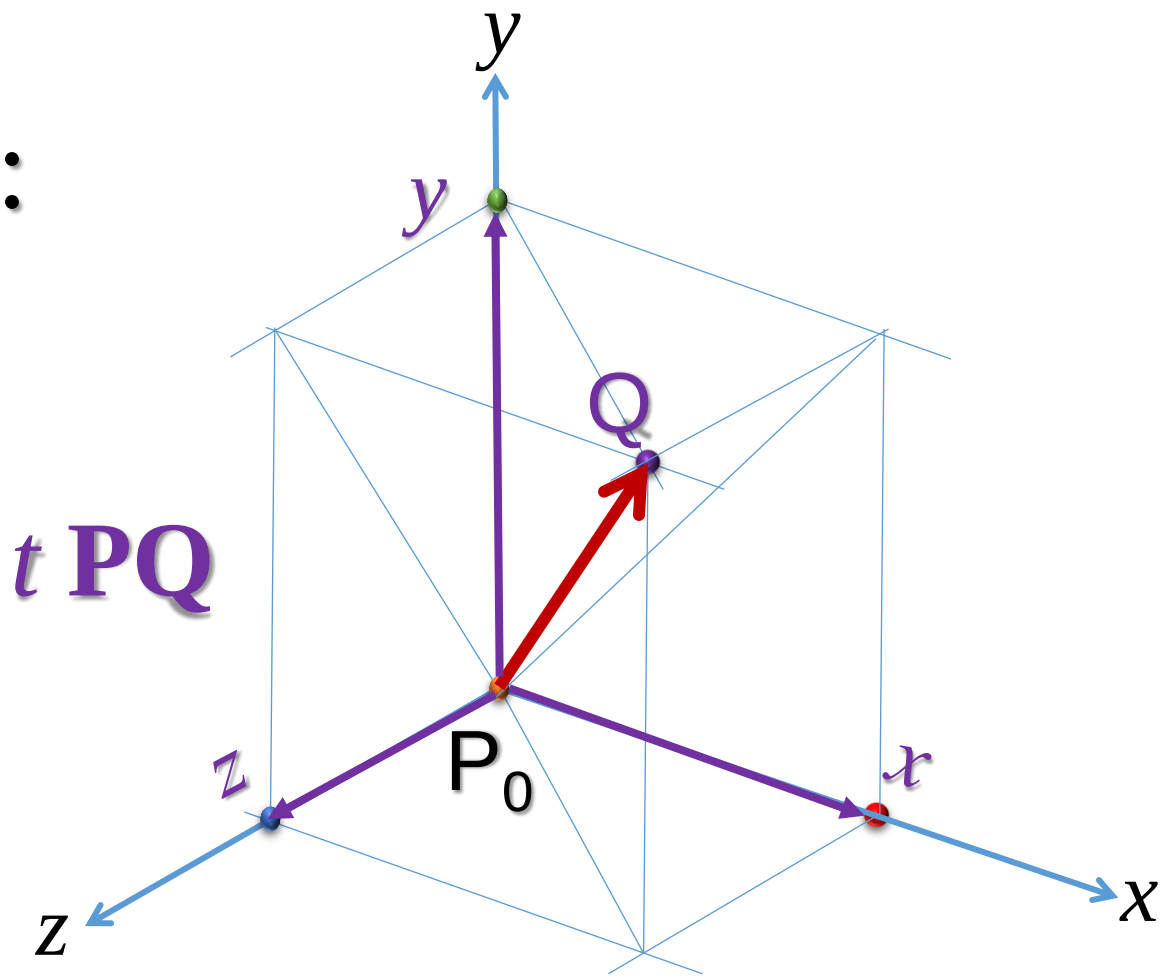
Summary

- Most important tools:
  - **Points:** P, Q, …
  - **Vectors:** u, v, …
  - **Rays:** $\mathbf{P}(t) = \mathbf{P}_0 + t\,\mathbf{PQ}$
  - **Normal Vectors:**

$$\mathbf{u} \times \mathbf{v} = \mathbf{n}$$

$$\mathbf{u} \cdot \mathbf{n} = 0$$

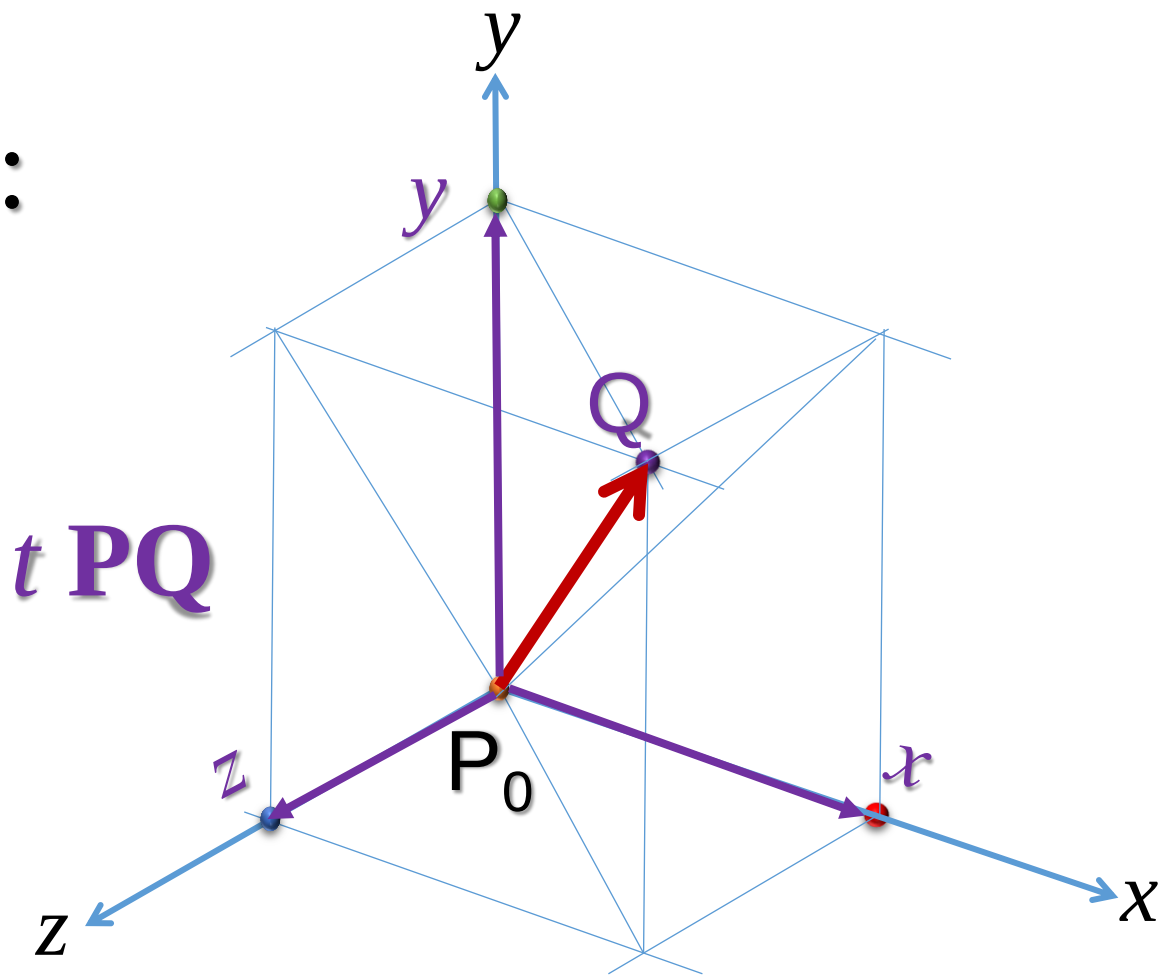
$$\mathbf{v} \cdot \mathbf{n} = 0$$



Summary

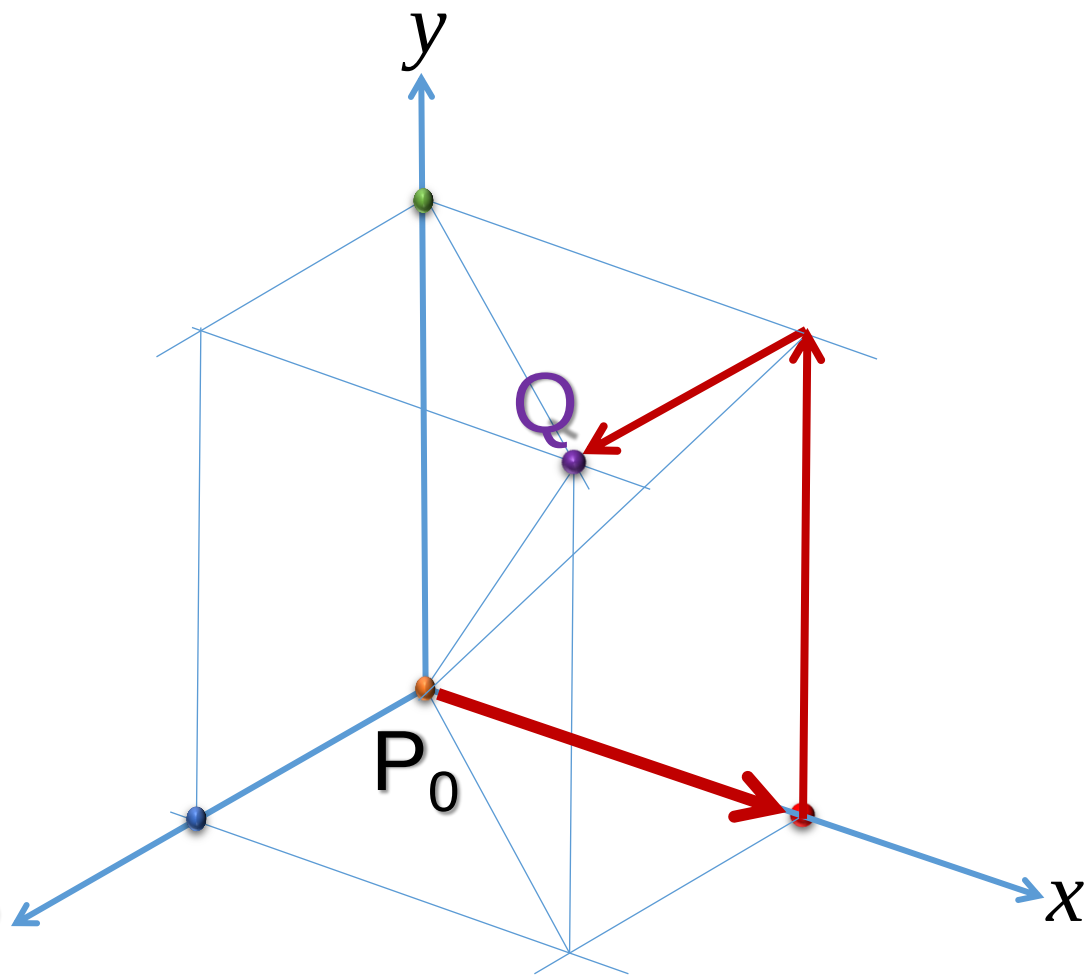
- Most important tools:
  - **Points:** P, Q, …
  - **Vectors:** u, v, …
  - **Rays:** $\mathbf{P}(t) = \mathbf{P}_0 + t\,\mathbf{PQ}$
  - **Normal Vectors:**

$$\mathbf{u} \times \mathbf{v} = \mathbf{n}$$
$$\mathbf{u} \cdot \mathbf{n} = 0$$
$$\mathbf{v} \cdot \mathbf{n} = 0$$



Set up Measurements

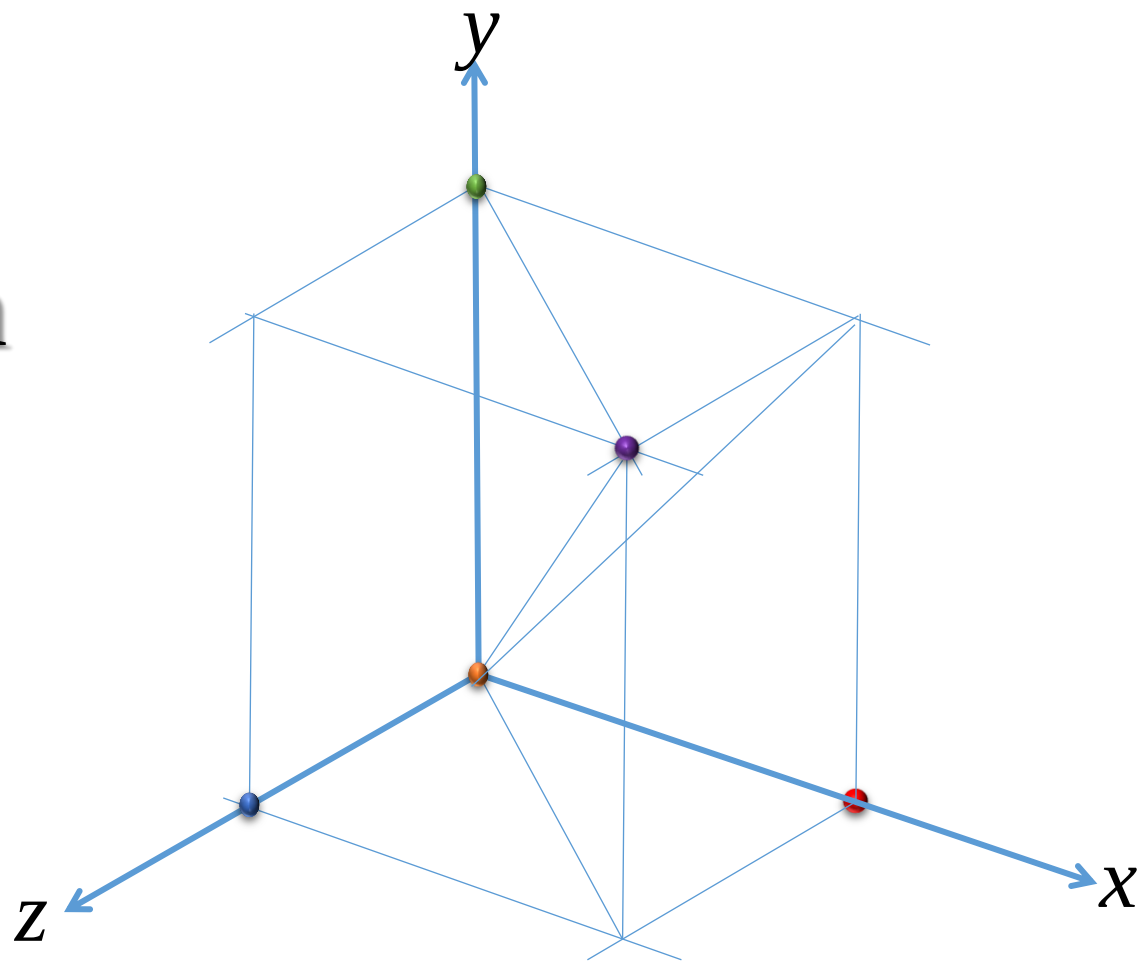
- We need
  - Three non-collinear vectors
  - Units of measure
- We call these numbers coordinates



Basics: Points vs Vectors

- A point is …

  a position or location in space.



Some points to note..

Composing Transforms: Order Matters

- Apply A then B to P: $P' = B \cdot (A \cdot P) = (B \cdot A) \cdot P$ — pre-multiplied, rightmost applies FIRST
- Composite once, apply to thousands of vertices: $M = T \cdot R \cdot S$ computed a single time
- Worked example: square at origin — rotate-by-45°-then-translate vs translate-then-rotate

Inverse Transformations

- Every basic transform has an inverse: $T(t)^{-1} = T(-t)$; $S(s)^{-1} = S(1/s)$
- Rotation: $R(\varphi)^{-1} = R(-\varphi) = R(\varphi)^{T}$ — the inverse of a rotation is its TRANSPOSE
- Composite inverse reverses order: $(A \cdot B)^{-1} = B^{-1} \cdot A^{-1}$

3D Rotations: One Per Axis

- $R_z(\varphi)$: rotates x,y; z fixed — the 2D matrix embedded in 3D (this is M1's rotateZ)
- $R_x(\varphi)$: $y' = y\cos\varphi - z\sin\varphi$; $z' = y\sin\varphi + z\cos\varphi$
- $R_y(\varphi)$: $z' = z\cos\varphi - x\sin\varphi$; $x' = z\sin\varphi + x\cos\varphi$ (watch the sign flip!)
- General 3D rotation = compose axis rotations; direction given by the right-hand rule
- Exam type question: write $R_z(90°)$ and apply it to $(1, 0, 0, 1)^T$

Matrices in Code

- WebGL2/GLSL convention: column-major, column vectors (refer to the beginning of this lesson)
- mat4 as Float32Array(16): index of (row r, col c) = c*4 + r
- Translation lands at indices 12, 13, 14 — the LAST column

Mat4 Skeleton in your code

- multiply(a, b) — 4×4 product (composition, appeared in previous part!)
- translate(x, y, z) / scale(x, y, z) / rotateZ(φ) — The matrices, 4×4 form
- perspective(fov, aspect, near, far) — preview only; W4 derives it fully
- **Advice: try to implement and unit-test TODAY's parts (multiply/translate/scale/rotateZ) while the math is fresh**

Week 3: Takeaways

- Points + vectors: $V = P1 - P0$; dot = projection, cross = normal
- Homogeneous coordinates unify translation/rotation/scaling as matrix products
- Compose with care: rightmost transform applies first; inverses reverse order

Next Week (W4): Viewing

- Where do these matrices plug in? The vertex transformation pipeline:
- Model → World → Eye → Clip → NDC → Screen (with perspective divide!)
- Window-to-viewport mapping — the section REMOVED from today's deck lands here, in full
- **Lab 3: your triangle gets a camera. Bring your Mat4 code.**